

OpenClaw 中文完整教程（总结版）

按学习路径重组 · 去除导航与跳转文本 · 覆盖全部中文资料

目录

- 第一章 入门与总览 (11 节)
- 第二章 安装与部署 (19 节)
- 第三章 核心概念 (28 节)
- 第四章 渠道接入 (28 节)
- 第五章 模型与提供商 (21 节)
- 第六章 工具体系 (24 节)
- 第七章 命令行实战 (41 节)
- 第八章 自动化与定时任务 (8 节)
- 第九章 节点能力 (9 节)
- 第十章 Web 控制台 (5 节)
- 第十一章 网关与系统配置 (29 节)
- 第十二章 平台专项 (27 节)
- 第十三章 插件扩展 (4 节)
- 第十四章 诊断与排障 (1 节)
- 第十五章 调试 (1 节)
- 第十六章 安全 (1 节)
- 第十七章 参考资料 (24 节)
- 第十八章 实验与进阶 (6 节)

第一章 入门与总览

1. 智能体引导

关键点：

- 该页面是英文文档的中文占位版本，完整内容请先参考英文版：Agent Bootstrapping。

2. docs directory

本节覆盖：

- 从这里开始
- 提供商与用户体验
- 配套应用

关键点：

- [文档中心（所有页面链接）](/start/hubs)
- [帮助](/help)
- [配置](/gateway/configuration)
- [配置示例](/gateway/configuration-examples)
- [斜杠命令](/tools/slash-commands)
- [多智能体路由](/concepts/multi-agent)
- [更新与回滚](/install/updating)
- [配对（私信和节点）](/channels/pairing)

3. 入门指南

本节覆盖：

- 0) 前置条件
- 1) 安装 CLI（推荐）
- 2) 运行新手引导向导（并安装服务）

关键点：

- 模型/认证（推荐 OAuth）
- Gateway 网关设置
- 渠道（WhatsApp/Telegram/Discord/Mattermost（插件）/...）
- 配对默认值（安全私信）
- 工作区引导 + Skills
- 可选的后台服务

- Node `>=22`
- `pnpm`（可选；如果从源代码构建则推荐）

操作示例：

- "workspace": "~/openclaw/workspace",
- `curl -fsSL https://openclaw.ai/install.sh | bash`
- `iwr -useb https://openclaw.ai/install.ps1 | iex`
- `npm install -g openclaw@latest`
- `openclaw onboard --install-daemon`
- `openclaw gateway status`

4. hubs

本节覆盖：

- 从这里开始
- 安装 + 更新
- 核心概念

关键点：

- [索引](/)
- [入门指南](/start/getting-started)
- [快速开始](/start/quickstart)
- [新手引导](/start/onboarding)
- [向导](/start/wizard)
- [安装配置](/start/setup)
- [仪表盘（本地 Gateway 网关）](http://127.0.0.1:18789/)
- [帮助](/help)

5. OpenClaw 的传说

本节覆盖：

- 起源故事
- 第一次蜕壳（2026 年 1 月 27 日）
- 名字的含义

关键点：

- GitHub 更名：`github.com/openclaw/openclaw``

- X 账号 `@openclaw` 获得金色认证标记
- npm 包以新名称发布
- 文档迁移到 `docs.openclaw.ai`
- 公告在 90 分钟内获得 20 万以上浏览量
- **ELU** 创作了令人惊叹的 logo，包括"THE CLAW IS THE LAW"西部横幅
- **Whurley**（是的，就是那个 William Hurley，量子计算先驱）制作了 ASCII 艺术
- **Onur** 处理了 GitHub，第一个获得合作徽章

6. 新手引导 (macOS 应用)

本节覆盖：

- 1) 欢迎 + 安全提示
- 2) 本地 vs 远程
- 3) 仅限本地的认证 (Anthropic OAuth)

关键点：

- 欢迎 + 安全提示
- **Gateway 网关选择** (本地 / 远程 / 稍后配置)
- **认证 (Anthropic OAuth)** — 仅限本地
- **设置向导** (Gateway 网关驱动)
- **权限** (TCC 提示)
- **CLI** (可选)
- **新手引导聊天** (专用会话)
- **本地 (此 Mac)：** 新手引导可以在本地运行 OAuth 流程并写入凭证。

操作示例：

- `openclaw webhooks gmail setup --account you@gmail.com`

7. 使用 OpenClaw 构建个人助手

本节覆盖：

- 安全第一
- 先决条件
- 双手机设置 (推荐)

关键点：

- 在你的机器上运行命令 (取决于你的 Pi 工具设置)

- 在你的工作区读/写文件
- 通过 WhatsApp/Telegram/Discord/Mattermost（插件）发送消息
- 始终设置 `channels.whatsapp.allowFrom`（永远不要在你的个人 Mac 上对全世界开放）。
- 为助手使用专用的 WhatsApp 号码。
- 心跳现在默认每 30 分钟一次。在你信任设置之前，通过设置 `agents.defaults.heartbeat.every: "0m"` 来禁用。
- Node **22+**
- OpenClaw 在 PATH 中可用（推荐：全局安装）

操作示例：

- `npm install -g openclaw@latest`
- `git clone https://github.com/openclaw/openclaw.git`
- `pnpm link --global`
- `openclaw channels login`
- `openclaw gateway --port 18789`
- `openclaw setup`

8. quickstart

本节覆盖：

- 安装
- 新手引导并运行 Gateway 网关
- 从源码安装（开发）

关键点：

- `<Note>`
- OpenClaw 需要 Node 22 或更新版本。
- `</Note>`
- `<Steps>`
- `<Step title="新手引导并安装服务">`
- `</Step>`
- `<Step title="配对 WhatsApp">`
- `<Step title="启动 Gateway 网关">`

操作示例：

- `npm install -g openclaw@latest`

- openclaw onboard --install-daemon
- openclaw channels login
- openclaw gateway --port 18789
- git clone https://github.com/openclaw/openclaw.git
- openclaw onboard --install-daemon

9. 设置

本节覆盖：

- 太长不看
- 先决条件（从源码）
- 个性化策略（让更新不会造成问题）

关键点：

- **个性化设置**存放在仓库之外：**~/.openclaw/workspace`（工作区） + ~/.openclaw/openclaw.json`（配置）。**
- **稳定工作流：**安装 macOS 应用；让它运行内置的 Gateway 网关。
- **前沿工作流：**通过 `pnpm gateway:watch`` 自己运行 Gateway 网关，然后让 macOS 应用以本地模式连接。
- Node `>=22``
- `pnpm``
- Docker（可选；仅用于容器化设置/e2e — 参阅 [Docker](/install/docker))
- **配置：**`~/.openclaw/openclaw.json``（JSON/JSON5 格式）
- **工作区：**`~/.openclaw/workspace``（Skills、提示、记忆；将其设为私有 git 仓库）

操作示例：

- openclaw setup
- openclaw setup
- openclaw channels login
- openclaw health
- ./scripts/restart-mac.sh
- openclaw health

10. 案例展示

本节覆盖：

- OpenClaw 实战演示

- Discord 最新分享
- 自动化与 workflows

关键点：

- 来自社区的真实项目。看看大家正在用 OpenClaw 构建什么。
- <Info>
- 想要展示你的项目？在 Discord 的 #showcase 频道 分享或在 X 上 @openclaw。
- </Info>
- VelvetShark 的完整设置演练（28 分钟）。
- style={{
- position: "relative",
- paddingBottom: "56.25%",

11. 新手引导向导 (CLI)

本节覆盖：

- 快速开始 vs 高级
- 向导做了什么
- 流程详情（本地）

关键点：

- 本地 Gateway 网关 (loopback)
- 默认工作区（或现有工作区）
- Gateway 网关端口 ****18789****
- Gateway 网关认证 ****Token****（自动生成，即使在 loopback 上）
- Tailscale 暴露 ****关闭****
- Telegram + WhatsApp 私信默认使用****允许列表****（系统会提示你输入电话号码）
- 模型/认证（OpenAI Code (Codex) 订阅 OAuth、Anthropic API 密钥（推荐）或 setup-token（粘贴），以及 MiniMax/GLM/Moonshot/AI Gateway 选项）
- 工作区位置 + 引导文件

操作示例：

- openclaw onboard
- openclaw configure
- openclaw agents add <name>
- openclaw onboard --non-interactive \

- openclaw onboard --non-interactive \
- openclaw onboard --non-interactive \

第二章 安装与部署

1. Ansible 安装

本节覆盖：

- 快速开始
- 你将获得
- 要求

关键点：

- ****防火墙优先安全****：UFW + Docker 隔离（仅 SSH + Tailscale 可访问）
- ****Tailscale VPN****：安全远程访问，无需公开暴露服务
- ****Docker****：隔离的沙箱容器，仅绑定 localhost
- ****纵深防御****：4 层安全架构
- ****一条命令设置****：几分钟内完成部署
- ****Systemd 集成****：带加固的开机自启动
- ****操作系统****：Debian 11+ 或 Ubuntu 20.04+
- ****访问权限****：Root 或 sudo 权限

操作示例：

- `curl -fsSL https://raw.githubusercontent.com/openclaw/openclaw-ansible/main/install.sh | bash`
- `openclaw channels login`
- `git clone https://github.com/openclaw/openclaw-ansible.git`
- `./run-playbook.sh`
- # 或直接运行（然后手动执行 `/tmp/openclaw-setup.sh`）
- # `ansible-playbook playbook.yml --ask-become-pass`

2. Bun（实验性）

本节覆盖：

- 状态
- 安装
- 构建/测试（Bun）

关键点：

- Bun 是一个可选的本地运行时，用于直接运行 TypeScript（``bun run ...``、``bun --watch ...``）。

- ``pnpm`` 是构建的默认工具，仍然完全支持（并被一些文档工具使用）。
- Bun 无法使用 ``pnpm-lock.yaml`` 并会忽略它。
- `@whiskeysockets/baileys`preinstall``：检查 Node 主版本 ≥ 20 （我们运行 Node 22+）。
- ``protobuffs`postinstall``：发出关于不兼容版本方案的警告（无构建产物）。
- 一些脚本仍然硬编码 pnpm（例如 ``docs:build``、``ui:*``、``protocol:check``）。目前请通过 pnpm 运行这些脚本。
- 目标：使用 Bun 运行此仓库（可选，不推荐用于 WhatsApp/Telegram），同时不偏离 pnpm 工作流。
- 不推荐用于 Gateway 网关运行时（WhatsApp/Telegram 存在 bug）。生产环境请使用 Node。

操作示例：

- `bun install`
- `bun install --no-save`
- `bun run build`
- `bun run vitest run`
- `bun pm trust @whiskeysockets/baileys protobuffs`

3. 开发渠道

本节覆盖：

- 切换渠道
- 插件和渠道
- 标签最佳实践

关键点：

- **beta**：`npm dist-tag beta``（测试中的构建）。
- **dev**：``main`` 的移动头（git）。`npm dist-tag : dev``（发布时）。
- ``dev`` 切换到 ``main`` 并在上游基础上 rebase。
- ``dev`` 确保有一个 git checkout（默认 `~/openclaw``，可通过 `OPENCLAW_GIT_DIR`` 覆盖），
- ``dev`` 优先使用 git checkout 中的内置插件。
- 为你希望 git checkout 落在的发布版本打标签（``vYYYY.M.D`` 或 ``vYYYY.M.D-<patch>``）。
- 保持标签不可变：永远不要移动或重用标签。
- `npm dist-tag`` 仍然是 npm 安装的数据源：

操作示例：

- `openclaw update --channel beta`
- `openclaw update --channel dev`
- `openclaw update --channel beta`
- `openclaw update --channel dev`

4. Docker（可选）

本节覆盖：

- Docker 适合我吗？
- 要求
- 容器化 Gateway 网关（Docker Compose）

关键点：

- ****是****：你想要一个隔离的、可丢弃的 Gateway 网关环境，或在没有本地安装的主机上运行 OpenClaw。
- ****否****：你在自己的机器上运行，只想要最快的开发循环。请改用正常的安装流程。
- ****沙箱注意事项****：智能体沙箱隔离也使用 Docker，但它****不需要****完整的 Gateway 网关在 Docker 中运行。参阅[沙箱隔离](/gateway/sandboxing)。
- 容器化 Gateway 网关（完整的 OpenClaw 在 Docker 中）
- 每会话智能体沙箱（主机 Gateway 网关 + Docker 隔离的智能体工具）
- Docker Desktop（或 Docker Engine）+ Docker Compose v2
- 足够的磁盘空间用于镜像 + 日志
- 构建 Gateway 网关镜像

操作示例：

- `./docker-setup.sh`
- `docker build -t openclaw:local -f Dockerfile .`
- `docker compose run --rm openclaw-cli onboard`
- `docker compose up -d openclaw-gateway`
- `docker compose -f docker-compose.yml -f docker-compose.extra.yml <command>`
- `docker compose run --rm openclaw-cli dashboard --no-open`

5. exe.dev

本节覆盖：

- 新手快速路径
- 你需要什么

- 使用 Shelley 自动安装

关键点：

- https://exe.new/openclaw
- 根据需要填写你的认证密钥/令牌
- 点击 VM 旁边的"Agent"，然后等待...
- exe.dev 账户
- `ssh exe.dev` 访问 exe.dev 虚拟机（可选）
- 目标：OpenClaw Gateway 网关运行在 exe.dev VM 上，可从你的笔记本电脑通过以下地址访问：`

• 本页假设使用 exe.dev 的默认 exeuntu 镜像。如果你选择了不同的发行版，请相应地映射软件包。

- Shelley，exe.dev 的智能体，可以使用我们的提示立即安装 OpenClaw。使用的提示如下：

操作示例：

- Set up OpenClaw (<https://docs.openclaw.ai/install>) on this VM. Use the non-interactive and accept-risk flags for openclaw onboarding. Add the supplied auth or token as needed. Configure nginx to forward from the default port 18789 to the root location on the default enabled site config, making sure to enable Websocket support. Pairing is done by "openclaw devices list" and "openclaw device approve <request id>". Make sure the dashboard shows that OpenClaw's health is OK. exe.dev handles forwarding from port 8000 to port 80/443 and HTTPS for us, so the final "reachable" should be <vm-name>.exe.xyz, without port specification.
- `curl -fsSL https://openclaw.ai/install.sh | bash`
- `location / {`
- `proxy_pass http://127.0.0.1:18789;`
- `npm i -g openclaw@latest`
- `openclaw doctor`

6. Fly.io 部署

本节覆盖：

- 你需要什么
- 初学者快速路径
- 1) 创建 Fly 应用

关键点：

- 已安装 [flyctl CLI](https://fly.io/docs/hands-on/install-flyctl/)

- Fly.io 账户（免费套餐可用）
- 模型认证：Anthropic API 密钥（或其他提供商密钥）
- 渠道凭证：Discord bot token、Telegram token 等
- 克隆仓库 → 自定义 `fly.toml`
- 创建应用 + 卷 → 设置密钥
- 使用 `fly deploy` 部署
- SSH 进入创建配置或使用 Control UI

操作示例：

- `git clone https://github.com/openclaw/openclaw.git`
- `fly volumes create openclaw_data --size 1 --region iad`
- `dockerfile = "Dockerfile"`
- `OPENCLAW_STATE_DIR = "/data"`
- `app = "node dist/index.js gateway --allow-unconfigured --port 3000 --bind lan"`
- `destination = "/data"`

7. 在 GCP Compute Engine 上运行 OpenClaw（Docker，生产 VPS 指南）

本节覆盖：

- 目标
- 我们在做什么（简单说明）？
- 快速路径（有经验的运维人员）

关键点：

- 创建 GCP 项目并启用计费
- 创建 Compute Engine VM
- 安装 Docker（隔离的应用运行时）
- 在 Docker 中启动 OpenClaw Gateway 网关
- 在主机上持久化 `~/openclaw` + `~/openclaw/workspace`（重启/重建后仍保留）
- 通过 SSH 隧道从你的笔记本电脑访问控制 UI
- 从你的笔记本电脑进行 SSH 端口转发
- 如果你自己管理防火墙和令牌，可以直接暴露端口

操作示例：

- `gcloud projects create my-openclaw-project --name="OpenClaw Gateway"`
- `gcloud compute ssh openclaw-gateway --zone=us-central1-a`

- `curl -fsSL https://get.docker.com | sudo sh`
- `gcloud compute ssh openclaw-gateway --zone=us-central1-a`
- `docker --version`
- `docker compose version`

8. 在 Hetzner 上运行 OpenClaw (Docker, 生产 VPS 指南)

本节覆盖：

- 目标
- 我们在做什么（简单说明）？
- 快速路径（有经验的运维人员）

关键点：

- 租用一台小型 Linux 服务器（Hetzner VPS）
- 安装 Docker（隔离的应用运行时）
- 在 Docker 中启动 OpenClaw Gateway 网关
- 在主机上持久化 `~/openclaw` + `~/openclaw/workspace``（重启/重建后保留）
- 通过 SSH 隧道从你的笔记本电脑访问控制 UI
- 从你的笔记本电脑进行 SSH 端口转发
- 如果你自己管理防火墙和令牌，可以直接暴露端口
- 配置 Hetzner VPS

操作示例：

- `curl -fsSL https://get.docker.com | sh`
- `docker --version`
- `docker compose version`
- `git clone https://github.com/openclaw/openclaw.git`
- `mkdir -p /root/.openclaw`
- `mkdir -p /root/.openclaw/workspace`

9. 安装

本节覆盖：

- 快速安装（推荐）
- 系统要求
- 选择安装路径

关键点：

- `**Node >=22**`
- macOS、Linux 或通过 WSL2 的 Windows
- ``pnpm`` 仅在从源代码构建时需要
- Docker：[Docker](/install/docker)
- Nix：[Nix](/install/nix)
- Ansible：[Ansible](/install/ansible)
- Bun（仅 CLI）：[Bun](/install/bun)
- 运行新手引导：``openclaw onboard --install-daemon``

操作示例：

- `curl -fsSL https://openclaw.ai/install.sh | bash`
- `iwr -useb https://openclaw.ai/install.ps1 | iex`
- `openclaw onboard --install-daemon`
- `curl -fsSL https://openclaw.ai/install.sh | bash`
- `curl -fsSL https://openclaw.ai/install.sh | bash -s -- --help`
- `curl -fsSL https://openclaw.ai/install.sh | bash -s -- --no-onboard`

10. 安装器内部机制

本节覆盖：

- `install.sh`（推荐）
- 可发现性 / "git 安装"提示
- 为什么需要 Git

关键点：

- ``https://openclaw.ai/install.sh`` — "推荐"安装器（默认全局 npm 安装；也可从 GitHub 检出安装）
- ``https://openclaw.ai/install-cli.sh`` — 无需 root 权限的 CLI 安装器（安装到带有独立 Node 的前缀目录）
- ``https://openclaw.ai/install.ps1`` — Windows PowerShell 安装器（默认 npm；可选 git 安装）
- 检测操作系统（macOS / Linux / WSL）。
- 确保 Node.js `**22+**`（macOS 通过 Homebrew；Linux 通过 NodeSource）。
- 选择安装方式：
- ``npm``（默认）：``npm install -g openclaw@latest``

- ``git`` : 克隆/构建源码检出并安装包装脚本

操作示例：

- `curl -fsSL https://openclaw.ai/install.sh | bash -s -- --help`
- `& ([scriptblock]::Create((iwr -useb https://openclaw.ai/install.ps1))) -?`
- `SHARP_IGNORE_GLOBAL_LIBVIPS=0 curl -fsSL https://openclaw.ai/install.sh | bash`
- `curl -fsSL https://openclaw.ai/install-cli.sh | bash -s -- --help`
- `iwr -useb https://openclaw.ai/install.ps1 | iex`
- `iwr -useb https://openclaw.ai/install.ps1 | iex -InstallMethod git`

11. 在 macOS 虚拟机上运行 OpenClaw（沙箱隔离）

本节覆盖：

- 推荐默认方案（大多数用户）
- macOS VM 选项
- 在你的 Apple Silicon Mac 上运行本地 VM（Lume）

关键点：

- **小型 Linux VPS** 用于永久在线的 Gateway 网关，成本低。参见 [VPS 托管](/vps)。
- **专用硬件**（Mac mini 或 Linux 机器）如果你想要完全控制和**住宅 IP**用于浏览器自动化。许多网站会屏蔽数据中心 IP，所以本地浏览通常效果更好。
- **混合方案：** 将 Gateway 网关保持在廉价 VPS 上，当你需要浏览器/UI 自动化时，将你的 Mac 作为**节点**连接。参见[节点](/nodes)和 [Gateway 网关远程](/gateway/remote)。
- 隔离的完整 macOS 环境（你的主机保持干净）
- 通过 BlueBubbles 支持 iMessage（在 Linux/Windows 上不可能）
- 通过克隆 VM 即时重置
- 无需额外硬件或云成本
- [MacStadium](https://www.macstadium.com/)（托管 Mac）

操作示例：

- `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/trycua/cua/main/libs/lume/scripts/install.sh)"`
- `echo 'export PATH="$PATH:$HOME/.local/bin"' >> ~/.zshrc && source ~/.zshrc`
- `lume --version`
- `lume create openclaw --os macos --ipsw latest`
- `npm install -g openclaw@latest`
- `openclaw onboard --install-daemon`

12. 将 OpenClaw 迁移到新机器

本节覆盖：

- 开始之前（你要迁移什么）
- 1) 确定你的状态目录
- 2) 确定你的工作区

关键点：

- 复制**状态目录**（`\$OPENCLAW\_STATE\_DIR`，默认：`~/openclaw/`） — 这包括配置、认证、会话和渠道状态。
- 复制你的**工作区**（默认 `~/openclaw/workspace/`） — 这包括你的智能体文件（记忆、提示等）。
- **状态目录**：`~/openclaw/`
- `--profile <name>`（通常变成 `~/openclaw-<profile>/`）
- `OPENCLAW\_STATE\_DIR=/some/path`
- `~/openclaw/workspace/`（推荐的工作区）
- 你创建的自定义文件夹
- Gateway 网关配置（`openclaw.json`）

操作示例：

- `openclaw status`
- `openclaw gateway stop`
- `tar -czf openclaw-workspace.tgz openclaw/workspace`
- `openclaw doctor`
- `openclaw gateway restart`
- `openclaw status`

13. Nix 安装

本节覆盖：

- 快速开始
- 你将获得
- Nix 模式运行时行为

关键点：

- Gateway 网关 + macOS 应用 + 工具（whisper、spotify、cameras） — 全部固定版本
- 重启后仍能运行的 Launchd 服务

- 带有声明式配置的插件系统
- 即时回滚：`home-manager switch --rollback`
- `OPENCLAW\_STATE\_DIR`（默认：`~/openclaw`）
- `OPENCLAW\_CONFIG\_PATH`（默认：`\$OPENCLAW\_STATE\_DIR/openclaw.json`）
- 自动安装和自我修改流程被禁用
- 缺失的依赖会显示 Nix 特定的修复消息

操作示例：

- Repository: `github:openclaw/nix-openclaw`
- 2. Create a local flake at `~/code/openclaw-local` using `templates/agent-first/flake.nix`
- 4. Set up secrets (bot token, Anthropic key) - plain files at `~/secrets/` is fine
- `apps/macos/Sources/OpenClaw/Resources/Info.plist`

14. Node.js

关键点：

- 该页面是英文文档的中文占位版本，完整内容请先参考英文版：Node.js。

15. northflank

本节覆盖：

- 如何开始
- 你将获得
- 设置流程

关键点：

- 点击 [Deploy OpenClaw](https://northflank.com/stacks/deploy-openclaw) 打开模板。
- 如果你还没有账户，请创建一个 [Northflank 账户](https://app.northflank.com/signup)。
- 点击 **Deploy OpenClaw now**。
- 设置必需的环境变量：`SETUP\_PASSWORD`。
- 点击 **Deploy stack** 构建并运行 OpenClaw 模板。
- 等待部署完成，然后点击 **View resources**。
- 打开 OpenClaw 服务。
- 打开公开的 OpenClaw URL，在 `/setup` 完成设置。

16. railway

本节覆盖：

- 快速检查清单（新用户）
- 一键部署
- 你将获得

关键点：

- 点击下方的 **Deploy on Railway**。
- 添加一个挂载到 `/data`` 的 **Volume**。
- 设置必需的**变量**（至少需要 `SETUP_PASSWORD``）。
- 在端口 `8080`` 上启用 **HTTP Proxy**。
- 打开 `https://<your-railway-domain>/setup`` 并完成向导。
- 为你生成一个域名（通常是 `https://<something>.up.railway.app``），或者
- 使用你绑定的自定义域名。
- `https://<your-railway-domain>/setup`` — 设置向导（需密码保护）

17. render

本节覆盖：

- 前提条件
- 使用 Render Blueprint 部署
- 了解 Blueprint

关键点：

- 一个 [Render 账户](https://render.com)（提供免费套餐）
- 来自你首选[模型提供商](/providers)的 API 密钥
- 根据本仓库根目录下的 `render.yaml`` Blueprint 创建一个新的 Render 服务。
- 提示你设置 `SETUP_PASSWORD``
- 构建 Docker 镜像并部署
- 访问 `https://<your-service>.onrender.com/setup``
- 输入你的 `SETUP_PASSWORD``
- 选择模型提供商并粘贴你的 API 密钥

操作示例：

- `healthCheckPath: /health``
- `value: /data/.openclaw``
- `value: /data/workspace``
- `mountPath: /data``

- <https://<your-service>.onrender.com/setup/export>

18. 卸载

本节覆盖：

- 简单方式（CLI 仍已安装）
- 手动服务移除（CLI 未安装）
- macOS (launchd)

关键点：

- 如果 `openclaw` 仍已安装，使用**简单方式**。
- 如果 CLI 已删除但服务仍在运行，使用**手动服务移除**。
- 停止 Gateway 网关服务：
- 卸载 Gateway 网关服务 (launchd/systemd/schtasks)：
- 删除状态 + 配置：
- 删除你的工作区（可选，移除智能体文件）：
- 移除 CLI 安装（选择你使用的那个）：
- 如果你安装了 macOS 应用：

操作示例：

- `openclaw uninstall`
- `openclaw uninstall --all --yes --non-interactive`
- `npx -y openclaw uninstall --all --yes --non-interactive`
- `openclaw gateway stop`
- `openclaw gateway uninstall`
- `rm -rf "${OPENCLAW_STATE_DIR:-$HOME/.openclaw}"`

19. 更新

本节覆盖：

- 推荐：重新运行网站安装程序（原地升级）
- 更新之前
- 更新（全局安装）

关键点：

- 如果你不想再次运行新手引导向导，添加 `--no-onboard`。
- 对于**源码安装**，使用：

- 对于**全局安装**，脚本底层使用`npm install -g openclaw@latest`。
- 旧版说明：`clawdbot` 仍可作为兼容性垫片使用。
- 了解你的安装方式：**全局**（npm/pnpm）还是**源码**（git clone）。
- 了解你的 Gateway 网关运行方式：**前台终端**还是**受管理服务**（launchd/systemd）。
- 快照你的定制内容：
- 配置：`~/.openclaw/openclaw.json`

操作示例：

- `curl -fsSL https://openclaw.ai/install.sh | bash`
- `curl -fsSL https://openclaw.ai/install.sh | bash -s -- --install-method git --no-onboard`
- `npm i -g openclaw@latest`
- `openclaw update --channel beta`
- `openclaw update --channel dev`
- `openclaw doctor`

第三章 核心概念

1. 智能体循环 (OpenClaw)

本节覆盖：

- 入口点
- 工作原理（高层次）
- 队列 + 并发

关键点：

- Gateway 网关 RPC：`agent` 和 `agent.wait`。
- CLI：`agent` 命令。
- `agent` RPC 验证参数，解析会话（sessionKey/sessionId），持久化会话元数据，立即返回 `{runId, acceptedAt}`。
- `agentCommand` 运行智能体：
- 解析模型 + 思考/详细模式默认值
- 加载 Skills 快照
- 调用 `runEmbeddedPiAgent`（pi-agent-core 运行时）
- 如果嵌入式循环未发出**生命周期结束/错误**事件，则发出该事件

2. 智能体工作区

本节覆盖：

- 默认位置
- 额外的工作区文件夹
- 工作区文件映射（每个文件的含义）

关键点：

- 默认：`~/openclaw/workspace`
- 如果设置了 `OPENCLAW\_PROFILE` 且不是 `"default"`，默认值变为
- 在 `~/openclaw/openclaw.json` 中覆盖：
- `AGENTS.md`
- 智能体的操作指南以及它应该如何使用记忆。
- 在每个会话开始时加载。
- 适合放置规则、优先级和"如何行为"的详细信息。
- `SOUL.md`

操作示例：

- workspace: "~/.openclaw/workspace",
- cd ~/.openclaw/workspace
- git init
- git add AGENTS.md SOUL.md TOOLS.md IDENTITY.md USER.md HEARTBEAT.md memory/
- git commit -m "Add agent workspace"
- git branch -M main

3. 智能体运行时

本节覆盖：

- 工作区（必需）
- 引导文件（注入）
- 内置工具

关键点：

- `AGENTS.md` — 操作指令 + "记忆"
- `SOUL.md` — 人设、边界、语气
- `TOOLS.md` — 用户维护的工具说明（例如 `img`、`sag`、约定）
- `BOOTSTRAP.md` — 一次性首次运行仪式（完成后删除）
- `IDENTITY.md` — 智能体名称/风格/表情
- `USER.md` — 用户档案 + 偏好称呼
- 内置（随安装包提供）
- 托管/本地：`~/.openclaw/skills`

4. Gateway 网关架构

本节覆盖：

- 概述
- 组件和流程
- Gateway 网关（守护进程）

关键点：

- 单个长期运行的 **Gateway 网关** 拥有所有消息平台（通过 Baileys 的 WhatsApp、通过 grammY 的 Telegram、Slack、Discord、Signal、iMessage、WebChat）。
- 控制平面客户端（macOS 应用、CLI、Web 界面、自动化）通过配置的绑定主机（默认 `127.0.0.1:18789`）上的 **WebSocket** 连接到 Gateway 网关。

- **节点** (macOS/iOS/Android/无头设备) 也通过 **WebSocket** 连接，但声明 `role: node`` 并带有明确的能力/命令。
- 每台主机一个 Gateway 网关；它是唯一打开 WhatsApp 会话的位置。
- **canvas 主机** (默认 `18793``) 提供智能体可编辑的 HTML 和 A2UI。
- 维护提供商连接。
- 暴露类型化的 WS API (请求、响应、服务器推送事件)。
- 根据 JSON Schema 验证进站帧。

操作示例：

- `|---- req:connect ----->|`
- `|<----- res (ok) -----| (or res error + close)`
- `|<----- event:presence ---|`
- `|<----- event:tick -----|`
- `|----- req:agent ----->|`
- `|<----- res:agent -----| (ack: {runId,status:"accepted"})`

5. 上下文窗口与压缩

本节覆盖：

- 什么是压缩
- 配置
- 自动压缩 (默认开启)

关键点：

- 压缩点之后的近期消息
- 详细模式下显示 ``Auto-compaction complete``
- ``/status`` 显示 ``Compactions: <count>``
- **压缩**：总结并**持久化**到 JSONL 中。
- **会话修剪**：仅裁剪旧的**工具结果**，**在内存中**按请求进行。
- 当会话感觉过时或上下文臃肿时，使用 ``/compact``。
- 大型工具输出已被截断；修剪可以进一步减少工具结果的堆积。
- 如果你需要全新开始，``/new`` 或 ``/reset`` 会启动一个新的会话 ID。

操作示例：

- `/compact Focus on decisions and open questions`

6. 上下文

本节覆盖：

- 快速开始（检查上下文）
- 示例输出
- ``/context list``

关键点：

- ****系统提示词****（OpenClaw 构建）：规则、工具、Skills 列表、时间/运行时，以及注入的工作区文件。
- ****对话历史****：你的消息 + 助手在此会话中的消息。
- ****工具调用/结果 + 附件****：命令输出、文件读取、图片/音频等。
- ``/status`` → 快速查看"我的窗口有多满？" + 会话设置。
- ``/context list`` → 注入了什么 + 大致大小（每个文件 + 总计）。
- ``/context detail`` → 更深入的分解：每个文件、每个工具 schema 大小、每个 Skills 条目大小和系统提示词大小。
- ``/usage tokens`` → 在正常回复后附加每次回复的使用量页脚。
- ``/compact`` → 将较旧的历史总结为紧凑条目以释放窗口空间。

操作示例：

- Bootstrap max/file: 20,000 chars
- Session tokens (cached): 14,250 total / ctx=32,000

7. features

本节覆盖：

- 亮点
- 完整列表

关键点：

- 通过 WhatsApp Web (Baileys) 集成 WhatsApp
- Telegram 机器人支持 (grammY)
- Discord 机器人支持 (channels.discord.js)
- Mattermost 机器人支持 (插件)
- 通过本地 imsg CLI 集成 iMessage (macOS)
- Pi 的智能体桥接，支持 RPC 模式和工具流式传输
- 长响应的流式传输和分块处理
- 多智能体路由，按工作区或发送者隔离会话

8. Markdown 格式化

本节覆盖：

- 目标
- 管道
- IR 示例

关键点：

- **一致性：**一次解析，多个渲染器。
- **安全分块：**在渲染前拆分文本，确保行内格式不会跨块断裂。
- **渠道适配：**将同一 IR 映射到 Slack mrkdown、Telegram HTML 和 Signal 样式范围，无需重新解析 Markdown。
- **解析 Markdown -> IR**
- IR 是纯文本加上样式跨度（粗体/斜体/删除线/代码/剧透）和链接跨度。
- 偏移量使用 UTF-16 代码单元，以便 Signal 样式范围与其 API 对齐。
- 仅当渠道启用了表格转换时才会解析表格。
- **分块 IR（格式优先）**

操作示例：

- Hello **world** — see [docs](https://docs.openclaw.ai).
- "links": [{ "start": 19, "end": 23, "href": "https://docs.openclaw.ai" }]

9. 记忆

本节覆盖：

- 记忆文件（Markdown）
- 何时写入记忆
- 自动记忆刷新（压缩前触发）

关键点：

- ``memory/YYYY-MM-DD.md``
- 每日日志（仅追加）。
- 在会话开始时读取今天和昨天的内容。
- ``MEMORY.md``（可选）
- 精心整理的长期记忆。
- **仅在主要的私人会话中加载**（绝不在群组上下文中加载）。
- 决策、偏好和持久性事实写入 ``MEMORY.md``。

- 日常笔记和运行上下文写入 `memory/YYYY-MM-DD.md` 。

操作示例：

- prompt: "Write any lasting notes to memory/YYYY-MM-DD.md; reply with NO_REPLY if nothing to store.",
- extraPaths: ["../team-docs", "/srv/shared-notes/overview.md"]
- baseUrl: "https://api.example.com/v1/",
- deltaMessages: 50 // JSONL 行数
- extensionPath: "/path/to/sqlite-vec"
- baseUrl: "https://api.example.com/v1/",

10. 消息

本节覆盖：

- 消息流程（高层概述）
- 进站去重
- 进站防抖

关键点：

- `messages.\*` 用于前缀、队列和群组行为。
- `agents.defaults.\*` 用于分块流式传输和分块默认值。
- 渠道覆盖（`channels.whatsapp.\*`、`channels.telegram.\*` 等）用于上限和流式传输开关。
- 防抖仅适用于**纯文本**消息；媒体/附件会立即刷新。
- 控制命令会绕过防抖，保持独立。
- 直接聊天合并到智能体主会话密钥。
- 群组/渠道获得各自的会话密钥。
- 会话存储和记录保存在 Gateway 网关主机上。

操作示例：

- -> 路由/绑定 -> 会话密钥

11. 模型故障转移

本节覆盖：

- 认证存储（密钥 + OAuth）
- 配置文件 ID
- 会话粘性（缓存友好）

关键点：

- 在当前提供商内进行**认证配置文件轮换**。
- **模型回退**到 ``agents.defaults.model.fallbacks`` 中的下一个模型。
- 密钥存储在 ``~/.openclaw/agents/<agentId>/agent/auth-profiles.json`` (旧版: ``~/.openclaw/agent/auth-profiles.json``)。
- 配置 ``auth.profiles`` / ``auth.order`` **仅用于元数据和路由** (不含密钥)。
- 旧版仅导入 OAuth 文件: ``~/.openclaw/credentials/oauth.json`` (首次使用时导入到 ``auth-profiles.json``)。
- ``type: "api_key"`` → ``{ provider, key }``
- ``type: "oauth"`` → ``{ provider, access, refresh, expires, email? }`` (某些提供商还有 ``projectId`/`enterpriseUrl``)
- 默认: 当没有电子邮件可用时为 ``provider:default``。

12. 模型提供商

本节覆盖:

- 快速规则
- 内置提供商 (pi-ai 目录)
- OpenAI

关键点:

- 模型引用使用 ``provider/model`` 格式 (例如: ``opencode/claude-opus-4-5``)。
- 如果设置了 ``agents.defaults.models``, 它将成为允许列表。
- CLI 辅助工具: ``openclaw onboard``、``openclaw models list``、``openclaw models set <provider/model>``。
- 提供商: ``openai``
- 认证: ``OPENAI_API_KEY``
- 示例模型: ``openai/gpt-5.2``
- CLI: ``openclaw onboard --auth-choice openai-api-key``
- 提供商: ``anthropic``

操作示例:

- `agents: { defaults: { model: { primary: "openai/gpt-5.2" } } },`
- `agents: { defaults: { model: { primary: "anthropic/claude-opus-4-5" } } },`
- `agents: { defaults: { model: { primary: "openai-codex/gpt-5.2" } } },`
- `agents: { defaults: { model: { primary: "opencode/claude-opus-4-5" } } },`
- `defaults: { model: { primary: "moonshot/kimi-k2.5" } },`

- baseUrl: "https://api.moonshot.ai/v1",

13. 模型 CLI

本节覆盖：

- 模型选择工作原理
- 快速模型推荐（经验之谈）
- 设置向导（推荐）

关键点：

- **主要模型**（``agents.defaults.model.primary`` 或 ``agents.defaults.model``）。
- **提供商认证故障转移**在移动到下一个模型之前在提供商内部发生。
- ``agents.defaults.models`` 是 OpenClaw 可使用的模型白名单/目录（加上别名）。
- ``agents.defaults.imageModel`` **仅在主要模型无法接受图像时使用**。
- 每个智能体的默认值可以通过 ``agents.list[].model`` 加绑定覆盖 ``agents.defaults.model``（参见 </concepts/multi-agent/>）。
- **GLM**：在编程/工具调用方面稍好。
- **MiniMax**：在写作和氛围方面更好。
- ``agents.defaults.model.primary`` 和 ``agents.defaults.model.fallbacks``

操作示例：

- openclaw onboard
- Model "provider/model" is not allowed. Use /model to list available models.
- model: { primary: "anthropic/claude-sonnet-4-5" },
- "anthropic/claude-sonnet-4-5": { alias: "Sonnet" },
- "anthropic/claude-opus-4-5": { alias: "Opus" },
- /model

14. 多智能体路由

本节覆盖：

- 什么是"一个智能体"？
- 路径（快速映射）
- 单智能体模式（默认）

关键点：

- **工作区**（文件、AGENTS.md/SOUL.md/USER.md、本地笔记、人设规则）。
- **状态目录**（``agentDir``）用于认证配置文件、模型注册表和每智能体配置。

- **会话存储**（聊天历史 + 路由状态）位于 `~/openclaw/agents/<agentId>/sessions`` 下。
- 配置：`~/openclaw/openclaw.json``（或 `OPENCLAW_CONFIG_PATH``）
- 状态目录：`~/openclaw``（或 `OPENCLAW_STATE_DIR``）
- 工作区：`~/openclaw/workspace``（或 `~/openclaw/workspace-<agentId>``）
- 智能体目录：`~/openclaw/agents/<agentId>/agent``（或 `agents.list[].agentDir``）
- 会话：`~/openclaw/agents/<agentId>/sessions``

操作示例：

- `~/openclaw/agents/<agentId>/agent/auth-profiles.json`
- `openclaw agents add work`
- `openclaw agents list --bindings`
- `{ id: "alex", workspace: "~/openclaw/workspace-alex" },`
- `{ id: "mia", workspace: "~/openclaw/workspace-mia" },`
- `workspace: "~/openclaw/workspace-home",`

15. OAuth

本节覆盖：

- 令牌汇聚点（为什么需要它）
- 存储（令牌存放位置）
- Anthropic setup-token（订阅认证）

关键点：

- OAuth **令牌交换**的工作原理（PKCE）
- 令牌**存储**在哪里（以及原因）
- 如何处理**多账户**（配置文件 + 按会话覆盖）
- 你通过 OpenClaw 和 Claude Code / Codex CLI 登录 → 其中一个稍后会随机“登出”
- 运行时从**同一个位置**读取凭据
- 我们可以保留多个配置文件并确定性地路由它们

- 认证配置文件（OAuth + API

密钥）：`~/openclaw/agents/<agentId>/agent/auth-profiles.json``

- 运行时缓存（自动管理；请勿编辑）：`~/openclaw/agents/<agentId>/agent/auth.json``

操作示例：

- `openclaw models auth login --provider <id>`
- `openclaw models auth setup-token --provider anthropic`

- openclaw models auth paste-token --provider anthropic
- openclaw models status
- openclaw agents add work
- openclaw agents add personal

16. 在线状态

本节覆盖：

- 在线状态字段（显示的内容）
- 生产者（在线状态来源）
- 1) Gateway 网关自身条目

关键点：

- **Gateway 网关**本身，以及
- **连接到 Gateway 网关的客户端**（mac 应用、WebChat、CLI 等）
- `instanceId``（可选但强烈推荐）：稳定的客户端身份（通常是 `connect.client.instanceId``）
- `host``：人类友好的主机名
- `ip``：尽力而为的 IP 地址
- `version``：客户端版本字符串
- `deviceFamily`` / `modelIdentifier``：硬件提示
- `mode``：`ui``、`webchat``、`cli``、`backend``、`probe``、`test``、`node``，...

17. 命令队列 (2026-01-16)

本节覆盖：

- 为什么需要
- 工作原理
- 队列模式（按渠道）

关键点：

- 自动回复运行可能开销很大（LLM 调用），当多条入站消息接近同时到达时可能发生冲突。
- 序列化可以避免竞争共享资源（会话文件、日志、CLI stdin），并降低上游速率限制的可能性。
- 一个支持通道感知的 FIFO 队列以可配置的并发上限排空每个通道（未配置的通道默认为 1；main 默认为 4，subagent 为 8）。
- `runEmbeddedPiAgent`` 按**会话键**入队（通道 `session:<key>``），以保证每个会话只有一个活动运行。

- 然后每个会话运行被排入**全局通道**（默认为`main`），因此整体并行度受`agents.defaults.maxConcurrent`限制。
- 启用详细日志时，如果排队运行在开始前等待超过约 2 秒，会发出简短通知。
- 输入指示器仍在入队时立即触发（当渠道支持时），因此在等待轮次时用户体验不受影响。
- `steer`：立即注入当前运行（在下一个工具边界后取消待处理的工具调用）。如果未在流式传输，则回退到 followup。

18. 重试策略

本节覆盖：

- 目标
- 默认值
- 行为

关键点：

- 按 HTTP 请求重试，而非按多步骤流程重试。
- 避免重复执行非幂等操作。
- 尝试次数：3
- 最大延迟上限：30000 毫秒
- 抖动：0.1（10%）
- 提供商默认值：
- Telegram 最小延迟：400 毫秒
- Discord 最小延迟：500 毫秒

19. 会话剪枝

本节覆盖：

- 运行时机
- 智能默认值（Anthropic）
- 改进内容（成本 + 缓存行为）

关键点：

- 当启用`mode: "cache-ttl"`且该会话的最后一次 Anthropic 调用早于`ttl`时。
- 仅影响该请求发送给模型的消息。
- 仅对 Anthropic API 调用（和 OpenRouter Anthropic 模型）生效。
- 为获得最佳效果，请将`ttl`与你的模型`cacheControlTtl`匹配。
- 剪枝后，TTL 窗口会重置，因此后续请求会保持缓存直到`ttl`再次过期。

- ****OAuth 或 setup-token**** 配置文件：启用 `cache-ttl` 剪枝并将心跳设置为 `1h`。
- ****API 密钥****配置文件：启用 `cache-ttl` 剪枝，将心跳设置为 `30m`，并将 Anthropic 模型的 `cacheControlTtl` 默认为 `1h`。
- 如果你显式设置了这些值中的任何一个，OpenClaw ****不会****覆盖它们。

20. 会话工具

本节覆盖：

- 工具名称
- 键模型
- sessions_list

关键点：

- `sessions\_list`
- `sessions\_history`
- `sessions\_send`
- `sessions\_spawn`
- 主直接聊天桶始终是字面键 `"main"`（解析为当前智能体的主键）。
- 群聊使用 `agent:<agentId>:<channel>:group:<id>` 或 `agent:<agentId>:<channel>:channel:<id>`（传递完整键）。
- 定时任务使用 `cron:<job.id>`。
- Hooks 使用 `hook:<uuid>`，除非明确设置。

操作示例：

- // 默认："spawned"
- sessionToolsVisibility: "spawned", // 或 "all"

21. 会话管理

本节覆盖：

- Gateway 网关是唯一数据源
- 状态存储位置
- 会话修剪

关键点：

- `main`（默认）：所有私信共享主会话以保持连续性。
- `per-peer`：跨渠道按发送者 ID 隔离。
- `per-channel-peer`：按渠道 + 发送者隔离（推荐用于多用户收件箱）。

- ``per-account-channel-peer``：按账户 + 渠道 + 发送者隔离（推荐用于多账户收件箱）。
- 在**远程模式**下，你关心的会话存储位于远程 Gateway 网关主机上，而不是你的 Mac 上。
- UI 中显示的令牌计数来自 Gateway 网关的存储字段（``inputTokens``、``outputTokens``、``totalTokens``、``contextTokens``）。客户端不会解析 JSONL 对话记录来"修正"总数。
- 在 **Gateway 网关主机**上：
- 存储文件：``~/openclaw/agents/<agentId>/sessions/sessions.json``（每个智能体）。

操作示例：

- `// ~/openclaw/openclaw.json`
- `scope: "per-sender", // keep group keys separate`
- `dmScope: "main", // DM continuity (set per-channel-peer/per-account-channel-peer for shared inboxes)`
- `// Defaults: mode=daily, atHour=4 (gateway host local time).`
- `// If you also set idleMinutes, whichever expires first wins.`
- `resetTriggers: ["/new", "/reset"],`

22. 会话

关键点：

- 规范的会话管理文档位于会话管理。

23. 流式传输 + 分块

本节覆盖：

- 分块流式传输（渠道消息）
- 分块算法（低/高边界）
- 合并（合并流式块）

关键点：

- ****分块流式传输（渠道）：**
在助手写入时发出已完成的**块**。这些是普通的渠道消息（不是令牌增量）。
- ****类令牌流式传输（仅限 Telegram）：**
在生成时用部分文本更新**草稿气泡**；最终消息在结束时发送。
- ``text_delta/events``：模型流事件（对于非流式模型可能稀疏）。
- ``chunker``：应用最小/最大边界 + 断点偏好的 ``EmbeddedBlockChunker``。
- ``channel send``：实际的出站消息（块回复）。
- ``agents.defaults.blockStreamingDefault``：``"on"/"off"``（默认关闭）。

- 渠道覆盖：`\*.blockStreaming`（以及每账户变体）可为每个渠道强制设置`"on"/"off"`。
- `agents.defaults.blockStreamingBreak`：`"text\_end"` 或 `"message\_end"`。

操作示例：

- └─ text_delta/events

24. 系统提示词

本节覆盖：

- 结构
- 提示词模式
- 工作区引导注入

关键点：

- **Tooling**：当前工具列表 + 简短描述。
- **Safety**：简短的防护提醒，避免追求权力的行为或绕过监督。
- **Skills**（如果可用）：告诉模型如何按需加载 Skill 指令。
- **OpenClaw Self-Update**：如何运行`config.apply`和`update.run`。
- **Workspace**：工作目录（`agents.defaults.workspace`）。
- **Documentation**：OpenClaw 文档的本地路径（仓库或 npm 包）以及何时阅读它们。
- **Workspace Files (injected)**：表示下方包含引导文件。
- **Sandbox**（启用时）：表示沙箱隔离运行时、沙箱路径，以及是否可用提权执行。

操作示例：

- <name>...</name>
- <description>...</description>
- <location>...</location>
- </skill>
- </available_skills>

25. 时区

本节覆盖：

- 消息信封（默认为本地时间）
- 示例
- 工具负载（原始提供商数据 + 规范化字段）

关键点：

- ``envelopeTimezone: "utc"``` 使用 UTC。
- ``envelopeTimezone: "user"``` 使用 ``agents.defaults.userTimezone``（回退到主机时区）。
- 使用显式 IANA 时区（例如 ``"Europe/Vienna"```）可设置固定偏移量。
- ``envelopeTimestamp: "off"``` 从信封头中移除绝对时间戳。
- ``envelopeElapsed: "off"``` 移除已用时间后缀（``+2m`` 样式）。
- ``timestampMs``（UTC 纪元毫秒数）
- ``timestampUtc``（ISO 8601 UTC 字符串）
- ``Current Date & Time`` 部分，显示本地时间和时区

操作示例：

- `envelopeTimezone: "local", // "utc" | "local" | "user" | IANA 时区`
- `envelopeTimestamp: "on", // "on" | "off"`
- `envelopeElapsed: "on", // "on" | "off"`
- `agents: { defaults: { userTimezone: "America/Chicago" } },`

26. TypeBox 作为协议的事实来源

本节覆盖：

- 心智模型（30 秒）
- 模式所在位置
- 当前流程

关键点：

- `**Request**` : ``{ type: "req", id, method, params }``
- `**Response**` : ``{ type: "res", id, ok, payload | error }``
- `**Event**` : ``{ type: "event", event, payload, seq?, stateVersion? }``
- 源码：``src/gateway/protocol/schema.ts``
- 运行时验证器（AJV）：``src/gateway/protocol/index.ts``
- 服务器握手 + 方法分发：``src/gateway/server.ts``
- 节点客户端：``src/gateway/client.ts``
- 生成的 JSON Schema：``dist/protocol.schema.json``

操作示例：

- `|---- req:connect ----->|`
- `|<---- res:hello-ok -----|`
- `|<---- event:tick -----|`

- |---- req:health ----->|
- |<---- res:health -----|
- `const ws = new WebSocket("ws://127.0.0.1:18789");`

27. 输入指示器

本节覆盖：

- 默认行为
- 模式
- 配置

关键点：

- ****私聊****：模型循环开始后立即显示输入指示器。
- ****群聊中被提及****：立即显示输入指示器。
- ****群聊中未被提及****：仅在消息文本开始流式传输时显示输入指示器。
- ****心跳运行****：输入指示器禁用。
- ``never`` — 永远不显示输入指示器。
- ``instant`` — ****模型循环开始后立即****显示输入指示器，即使运行最终只返回静默回复令牌。
- ``thinking`` — 在****第一个推理增量****时开始显示输入指示器（需要运行时设置
- ``message`` — 在****第一个非静默文本增量****时开始显示输入指示器（忽略

28. 使用量跟踪

本节覆盖：

- 功能简介
- 展示位置
- 提供商及凭据

关键点：

- 直接从提供商的使用量端点拉取使用量/配额数据。
- 不提供估算费用；仅展示提供商报告的时间窗口数据。
- 聊天中的 ``/status``：包含会话 token 数和估算费用的表情符号丰富的状态卡片（仅限 API 密钥）。当可用时，会显示****当前模型提供商****的使用量。
- 聊天中的 ``/usage off|tokens|full``：每次响应的使用量页脚（OAuth 仅显示 token 数）。
- 聊天中的 ``/usage cost``：从 OpenClaw 会话日志汇总的本地费用摘要。
- CLI：``openclaw status --usage`` 打印完整的按提供商分类的详细信息。

- CLI : ``openclaw channels list`` 在提供商配置旁打印相同的使用量快照（使用 ``--no-usage`` 跳过）。
- macOS 菜单栏：上下文菜单下的"使用量"部分（仅在可用时显示）。

第四章 渠道接入

1. BlueBubbles (macOS REST)

本节覆盖：

- 概述
- 快速开始
- 新手引导

关键点：

- 通过 BlueBubbles 辅助应用在 macOS 上运行（bluebubbles.app）。
- 推荐/已测试版本：macOS Sequoia (15)。macOS Tahoe (26) 可用；但在 Tahoe 上编辑功能目前不可用，群组图标更新可能显示成功但实际未同步。
- OpenClaw 通过其 REST API 与之通信（`GET /api/v1/ping`、`POST /message/text`、`POST /chat/:id/\*`）。
- 传入消息通过 webhook 到达；发出的回复、输入指示器、已读回执和 tapback 均为 REST 调用。
- 附件和贴纸作为进站媒体被接收（并在可能时呈现给智能体）。
- 配对/白名单的工作方式与其他渠道相同（`/channels/pairing` 等），使用 `channels.bluebubbles.allowFrom` + 配对码。
- 回应作为系统事件呈现，与 Slack/Telegram 类似，智能体可以在回复前提及它们。
- 高级功能：编辑、撤回、回复线程、消息效果、群组管理。

操作示例：

- `serverUrl: "http://192.168.1.100:1234",`
- `webhookPath: "/bluebubbles-webhook",`
- `openclaw onboard`
- `openclaw channels add bluebubbles --http-url http://192.168.1.100:1234 --password <password>`
- `"*": { requireMention: true }, // 所有群组的默认设置`
- `"iMessage;-;chat123": { requireMention: false }, // 特定群组的覆盖设置`

2. 广播群组

本节覆盖：

- 概述
- 使用场景

- 1. 专业智能体团队

关键点：

- 群聊：群组 JID（例如 `120363403215116621@g.us`）
- 私信：E.164 格式的电话号码（例如 `+15551234567`）
- ****接收消息**** 到达 WhatsApp 群组
- ****广播检查****：系统检查 peer ID 是否在 `broadcast` 中
- ****如果在广播列表中****：
- 所有列出的智能体处理该消息
- 每个智能体有自己的会话键和隔离的上下文
- ****如果不在广播列表中****：

操作示例：

- "workspace": "/path/to/code-reviewer",
- "workspace": "/path/to/security-auditor",
- "workspace": "/path/to/docs-generator",
- Workspace: /Users/pascal/openclaw-alfred/
- Workspace: /Users/pascal/openclaw-baerbel/
- "tools": { "allow": ["read", "exec"] } // Read-only

3. 渠道与路由

本节覆盖：

- 关键术语
- 会话键格式（示例）
- 路由规则（如何选择智能体）

关键点：

- ****渠道****：`whatsapp`、`telegram`、`discord`、`slack`、`signal`、`imessage`、`webchat`。
- ****AccountId****：每个渠道的账户实例（在支持的情况下）。
- ****AgentId****：隔离的工作区 + 会话存储（"大脑"）。
- ****SessionKey****：用于存储上下文和控制并发的桶键。
- `agent:<agentId>:<mainKey>`（默认：`agent:main:main`）
- 群组：`agent:<agentId>:<channel>:group:<id>`
- 渠道/房间：`agent:<agentId>:<channel>:channel:<id>`

- Slack/Discord 线程会在基础键后追加 `:thread:<threadId>` 。

操作示例：

- list: [{ id: "support", name: "Support", workspace: "~/.openclaw/workspace-support" }],

4. Discord (Bot API)

本节覆盖：

- 快速设置（新手）
- 目标
- 工作原理

关键点：

- 创建 Discord 机器人并复制机器人令牌。
- 在 Discord 应用设置中启用 **Message Content Intent**（如果你计划使用允许列表或名称查找，还需启用 **Server Members Intent**）。
- 为 OpenClaw 设置令牌：
- 环境变量：`DISCORD\_BOT\_TOKEN=...`
- 或配置：`channels.discord.token: "..."`。
- 如果两者都设置，配置优先（环境变量回退仅适用于默认账户）。
- 使用消息权限邀请机器人到你的服务器（如果你只想使用私信，可以创建一个私人服务器）。
- 启动 Gateway 网关。

操作示例：

- policy: "pairing", // pairing | allowlist | open | disabled
- token: "pk_live_...", // 可选；私有系统需要

5. 飞书机器人

本节覆盖：

- 需要插件
- 快速开始
- 方式一：通过安装向导添加（推荐）

关键点：

- 创建飞书应用并获取凭证
- 配置应用凭证
- `openclaw gateway status` - 查看网关运行状态
- `openclaw logs --follow` - 查看实时日志

- `openclaw gateway restart` - 重启网关以应用新配置
- 点击 ****创建企业自建应用****
- 填写应用名称和描述
- 选择应用图标

操作示例：

- `openclaw plugins install @openclaw/feishu`
- `openclaw plugins install ./extensions/feishu`
- `openclaw onboard`
- `openclaw channels add`
- `openclaw channels add`
- `openclaw gateway`

6. Google Chat (Chat API)

本节覆盖：

- 快速设置（新手）
- 添加到 Google Chat
- 公网 URL（仅 Webhook）

关键点：

- 创建一个 Google Cloud 项目并启用 ****Google Chat API****。
- 前往：[Google Chat API Credentials](https://console.cloud.google.com/apis/api/chat.googleapis.com/credentials)
- 如果 API 尚未启用，请启用它。
- 创建一个****服务账号****：
- 点击 ****Create Credentials**** > ****Service Account****。
- 随意命名（例如 `openclaw-chat`）。
- 权限留空（点击 ****Continue****）。
- 有访问权限的主账号留空（点击 ****Done****）。

操作示例：

- `tailscale serve --bg --https 8443 http://127.0.0.1:18789`
- `tailscale serve --bg --https 8443 http://100.106.161.80:18789`
- `tailscale funnel --bg --set-path /googlechat http://127.0.0.1:18789/googlechat`
- `tailscale funnel --bg --set-path /googlechat http://100.106.161.80:18789/googlechat`

- `reverse_proxy /googlechat* localhost:18789`
- `serviceAccountFile: "/path/to/service-account.json",`

7. grammY 集成 (Telegram Bot API)

关键点：

- 以 TS 为核心的 Bot API 客户端，内置长轮询 + webhook 辅助工具、中间件、错误处理和速率限制器。
- 媒体处理辅助工具比手动编写 `fetch + FormData` 更简洁；支持所有 Bot API 方法。
- 可扩展：通过自定义 `fetch` 支持代理，可选的会话中间件，类型安全的上下文。
- **单一客户端路径：**移除了基于 `fetch` 的实现；grammY 现在是唯一的 Telegram 客户端（发送 + Gateway 网关），默认启用 `grammY throttler`。
- **Gateway 网关：**`monitorTelegramProvider`` 构建 `grammY `Bot``，接入 `mention/allowlist` 网关控制，通过 ``getFile`/`download`` 下载媒体，并使用 ``sendMessage/sendPhoto/sendVideo/sendAudio/sendDocument`` 发送回复。通过 ``webhookCallback`` 支持长轮询或 webhook。
- **代理：**可选的 ``channels.telegram.proxy`` 通过 grammY 的 ``client.baseFetch`` 使用 ``undici.ProxyAgent``。
- **Webhook 支持：**``webhook-set.ts`` 封装了 ``setWebhook/deleteWebhook``；``webhook.ts`` 托管回调，支持健康检查和优雅关闭。当设置了 ``channels.telegram.webhookUrl`` + ``channels.telegram.webhookSecret`` 时，Gateway 网关启用 webhook 模式（否则使用长轮询）。
- **会话：**私聊折叠到智能体主会话（``agent:<agentId>:<mainKey>``）；群组使用 ``agent:<agentId>:telegram:group:<chatId>``；回复路由回同一渠道。

8. 群组消息 (WhatsApp 网页渠道)

本节覆盖：

- 已实现的功能 (2025-12-03)
- 配置示例 (WhatsApp)
- 激活命令 (仅所有者)

关键点：

- 激活模式：``mention``（默认）或 ``always``。``mention`` 需要被提及（通过 ``mentionedJids`` 的真实 WhatsApp @提及、正则表达式模式，或文本中任意位置的机器人 E.164 号码）。``always`` 会在每条消息时唤醒智能体，但它应该只在能提供有意义价值时才回复；否则返回静默令牌 ``NO_REPLY``。默认值可在配置中设置（``channels.whatsapp.groups``），并可通过

`/activation` 为每个群组单独覆盖。当设置了 `channels.whatsapp.groups` 时，它同时充当群组允许列表（包含 `""` 以允许所有群组）。

- 群组策略：`channels.whatsapp.groupPolicy` 控制是否接受群组消息（`open|disabled|allowlist`）。`allowlist` 使用 `channels.whatsapp.groupAllowFrom`（回退：显式的 `channels.whatsapp.allowFrom`）。默认为 `allowlist`（在你添加发送者之前被阻止）。
- 独立群组会话：会话键格式为 `agent:<agentId>:whatsapp:group:<jid>`，因此 `/verbose on` 或 `/think high`（作为独立消息发送）等命令仅作用于该群组；个人私信状态不受影响。群组线程会跳过心跳。
- 上下文注入：**仅待处理**的群组消息（默认 50 条），即*未*触发运行的消息，会以 `[Chat messages since your last reply - for context]` 为前缀注入，触发行在 `[Current message - respond to this]` 下。已在会话中的消息不会重复注入。
- 发送者显示：每个群组批次现在以 `[from: Sender Name (+E164)]` 结尾，让 Pi 知道是谁在说话。

• 阅后即焚/一次性查看：我们在提取文本/提及之前会先解包这些消息，因此其中的提及仍会触发。

- 群组系统提示：在群组会话的第一轮（以及每当 `/activation` 更改模式时），我们会向系统提示注入一段简短说明，如 `You are replying inside the WhatsApp group "<subject>". Group members: Alice (+44...), Bob (+43...), ... Activation: trigger-only ... Address the specific sender noted in the message context.` 如果元数据不可用，我们仍会告知智能体这是一个群聊。
- 正则表达式不区分大小写；它们涵盖了像 `@openclaw` 这样的显示名称提及，以及带或不带 `+`/空格的原始号码。

9. 群组

本节覆盖：

- 新手入门（2 分钟）
- 会话键
- 模式：个人私信 + 公开群组（单智能体）

关键点：

- 群组受限（`groupPolicy: "allowlist"`）。
- 除非你明确禁用提及限制，否则回复需要 @ 提及。
- 群组会话使用 `agent:<agentId>:<channel>:group:<id>` 会话键（房间/频道使用 `agent:<agentId>:<channel>:channel:<id>`）。
- Telegram 论坛话题在群组 ID 后添加 `:topic:<threadId>`，因此每个话题都有自己的会话。
- 私聊使用主会话（或按发送者配置时使用各自的会话）。

- 群组会话跳过心跳。
- ****私信****：完整工具（主机）
- ****群组****：沙箱 + 受限工具（Docker）

操作示例：

- mode: "non-main", // 群组/频道是非主 -> 沙箱隔离
- scope: "session", // 最强隔离（每个群组/频道一个容器）
- // 如果 allow 非空，其他所有工具都被阻止（deny 仍然优先）。
- docker: {
- // hostPath: containerPath: mode
- "~/FriendsShared:/data:ro",

10. iMessage (img)

本节覆盖：

- 快速设置（新手）
- 简介
- 配置写入

关键点：

- 确保在此 Mac 上已登录"信息"。
- 安装 `img`：
- `brew install steipete/tap/img`
- 配置 OpenClaw 的 `channels.imessage.cliPath` 和 `channels.imessage.dbPath`。
- 启动 Gateway 网关并批准所有 macOS 提示（自动化 + 完全磁盘访问权限）。
- 基于 macOS 上 `img` 的 iMessage 渠道。
- 确定性路由：回复始终返回到 iMessage。
- 私信共享智能体的主会话；群组是隔离的（`agent:<agentId>:imessage:group:<chat\_id>`）。

操作示例：

- cliPath: "/usr/local/bin/img",
- dbPath: "/Users/<you>/Library/Messages/chat.db",
- #!/usr/bin/env bash
- exec /usr/bin/ssh -o BatchMode=yes -o ConnectTimeout=5 -T
- <bot-macos-user>@localhost \
- "/usr/local/bin/img" "\$@"

- cliPath: "/path/to/ormsg-bot",

11. 聊天渠道

本节覆盖：

- 支持的渠道
- 注意事项

关键点：

- [WhatsApp](/channels/whatsapp) — 最受欢迎；使用 Baileys，需要二维码配对。
- [Telegram](/channels/telegram) — 通过 grammY 使用 Bot API；支持群组。
- [Discord](/channels/discord) — Discord Bot API + Gateway；支持服务器、频道和私信。
- [Slack](/channels/slack) — Bolt SDK；工作区应用。
- [飞书](/channels/feishu) — 飞书（Lark）机器人（插件，需单独安装）。
- [Google Chat](/channels/googlechat) — 通过 HTTP webhook 的 Google Chat API 应用。
- [Mattermost](/channels/mattermost) — Bot API + WebSocket；频道、群组、私信（插件，需单独安装）。
- [Signal](/channels/signal) — signal-cli；注重隐私。

12. LINE（插件）

本节覆盖：

- 需要安装插件
- 配置步骤
- 配置

关键点：

- 创建 LINE Developers 账户并打开控制台：
- 创建（或选择）一个 Provider 并添加 **Messaging API** 渠道。
- 从渠道设置中复制 **Channel access token** 和 **Channel secret**。
- 在 Messaging API 设置中启用 **Use webhook**。
- 将 webhook URL 设置为你的 Gateway 网关端点（必须使用 HTTPS）：
- ``LINE_CHANNEL_ACCESS_TOKEN``
- ``LINE_CHANNEL_SECRET``
- ``channels.line.dmPolicy``：`pairing | allowlist | open | disabled`

操作示例：

- openclaw plugins install @openclaw/line
- openclaw plugins install ./extensions/line
- https://gateway-host/line/webhook
- tokenFile: "/path/to/line-token.txt",
- secretFile: "/path/to/line-secret.txt",
- webhookPath: "/line/marketing",

13. 渠道位置解析

本节覆盖：

- 文本格式
- 上下文字段
- 渠道说明

关键点：

- 附加到进站消息体的可读文本，以及
- 自动回复上下文负载中的结构化字段。
- **Telegram**（位置图钉 + 地点 + 实时位置）
- **WhatsApp**（locationMessage + liveLocationMessage）
- **Matrix**（`m.location` 配合 `geo\_uri`）
- `48.858844, 2.294351 ±12m`
- `Eiffel Tower — Champ de Mars, Paris (48.858844, 2.294351 ±12m)`
- `Live location: 48.858844, 2.294351 ±12m`

14. Matrix（插件）

本节覆盖：

- 需要插件
- 设置
- 加密（E2EE）

关键点：

- 安装 Matrix 插件：
- 从 npm：`openclaw plugins install @openclaw/matrix`
- 从本地检出：`openclaw plugins install ./extensions/matrix`
- 在主服务器上创建 Matrix 账户：

- 在 https://matrix.org/ecosystem/hosting/ 浏览托管选项
- 或自行托管。
- 获取机器人账户的访问令牌：
- 在你的主服务器上使用 `curl` 调用 Matrix 登录 API：

操作示例：

- `openclaw plugins install @openclaw/matrix`
- `openclaw plugins install ./extensions/matrix`
- `curl --request POST \`
- `--url https://matrix.example.org/_matrix/client/v3/login \`
- `--header 'Content-Type: application/json' \`
- `homeserver: "https://matrix.example.org",`

15. Mattermost（插件）

本节覆盖：

- 需要插件
- 快速设置
- 环境变量（默认账户）

关键点：

- 安装 Mattermost 插件。
- 创建 Mattermost bot 账户并复制 **bot token**。
- 复制 Mattermost **基础 URL**（例如 `https://chat.example.com``）。
- 配置 OpenClaw 并启动 Gateway 网关。
- `MATTERMOST_BOT_TOKEN=...``
- `MATTERMOST_URL=https://chat.example.com``
- `oncall``（默认）：仅在频道中被 @提及时响应。
- `onmessage``：响应每条频道消息。

操作示例：

- `openclaw plugins install @openclaw/mattermost`
- `openclaw plugins install ./extensions/mattermost`
- `baseUrl: "https://chat.example.com",`
- `default: { name: "Primary", botToken: "mm-token", baseUrl: "https://chat.example.com" },`

- alerts: { name: "Alerts", botToken: "mm-token-2", baseUrl: "https://alerts.example.com" },

16. Microsoft Teams（插件）

本节覆盖：

- 需要插件
- 快速设置（初学者）
- 目标

关键点：

- 安装 Microsoft Teams 插件。
- 创建一个 **Azure Bot**（App ID + 客户端密钥 + 租户 ID）。
- 使用这些凭证配置 OpenClaw。
- 通过公共 URL 或隧道暴露 `/api/messages``（默认端口 3978）。
- 安装 Teams 应用包并启动 Gateway 网关。
- 通过 Teams 私信、群聊或频道与 OpenClaw 交流。
- 保持路由确定性：回复始终返回到消息到达的渠道。
- 默认使用安全的渠道行为（除非另有配置，否则需要提及）。

操作示例：

- `openclaw plugins install @openclaw/msteams`
- `openclaw plugins install ./extensions/msteams`
- `webhook: { port: 3978, path: "/api/messages" },`
- # 复制 https URL，例如 `https://abc123.ngrok.io`
- # 将消息端点设置为：`https://abc123.ngrok.io/api/messages`
- `"webhook": { "port": 3978, "path": "/api/messages" }`

17. Nextcloud Talk（插件）

本节覆盖：

- 需要插件
- 快速设置（新手）
- 注意事项

关键点：

- 安装 Nextcloud Talk 插件。
- 在你的 Nextcloud 服务器上创建机器人：

- 在目标房间设置中启用机器人。
- 配置 OpenClaw :
- 配置项: ``channels.nextcloud-talk.baseUrl` + `channels.nextcloud-talk.botSecret``
- 或环境变量: ``NEXTCLOUD_TALK_BOT_SECRET`` (仅默认账户)
- 重启 Gateway 网关 (或完成新手引导)。
- 机器人无法主动发起私信。用户必须先向机器人发送消息。

操作示例:

- `openclaw plugins install @openclaw/nextcloud-talk`
- `openclaw plugins install ./extensions/nextcloud-talk`
- `./occ talk:bot:install "OpenClaw"<shared-secret>"<webhook-url>" --feature reaction`
- `baseUrl: "https://cloud.example.com",`

18. Nostr

本节覆盖:

- 安装 (按需)
- 新手引导 (推荐)
- 手动安装

关键点:

- 新手引导向导 (``openclaw onboard``) 和 ``openclaw channels add`` 会列出可选的渠道插件。
- 选择 Nostr 会提示你按需安装插件。
- ****Dev 渠道 + git checkout 可用: **** 使用本地插件路径。
- 生成 Nostr 密钥对 (如需要):
- 添加到配置:
- 重启 Gateway 网关。
- 个人资料 URL 必须使用 ``https://``。
- 从中继导入会合并字段并保留本地覆盖。

操作示例:

- `openclaw plugins install @openclaw/nostr`
- `openclaw plugins install --link <path-to-openclaw>/extensions/nostr`
- `"picture": "https://example.com/avatar.png",`
- `"banner": "https://example.com/banner.png",`

- "website": "https://example.com",
- "relays": ["wss://relay.damus.io", "wss://relay.primal.net", "wss://nostr.wine"]

19. 配对

本节覆盖：

- 1) 私信配对（进站聊天访问）
- 批准发送者
- 状态存储位置

关键点：

- **私信配对**（谁被允许与机器人对话）
- **节点配对**（哪些设备/节点被允许加入 Gateway 网关网络）
- 8 个字符，大写，无歧义字符（`0O1l`）。
- **1 小时后过期**。机器人仅在创建新请求时发送配对消息（大约每个发送者每小时一次）。
- 待处理的私信配对请求默认上限为**每个渠道 3 个**；在一个过期或被批准之前，额外的请求将被忽略。
- 待处理请求：`<channel>-pairing.json`
- 已批准允许列表存储：`<channel>-allowFrom.json`
- `pending.json`（短期；待处理请求会过期）

操作示例：

- `openclaw pairing list telegram`
- `openclaw pairing approve telegram <CODE>`
- `openclaw devices list`
- `openclaw devices approve <requestId>`
- `openclaw devices reject <requestId>`

20. Signal (signal-cli)

本节覆盖：

- 快速设置（初学者）
- 它是什么
- 配置写入

关键点：

- 为 bot 使用**单独的 Signal 号码**（推荐）。
- 安装 `signal-cli`（需要 Java）。

- 链接 bot 设备并启动守护进程：
- ``signal-cli link -n "OpenClaw"``
- 配置 OpenClaw 并启动 Gateway 网关。
- 通过 ``signal-cli`` 的 Signal 渠道（非嵌入式 libsignal）。
- 确定性路由：回复始终返回到 Signal。
- 私信共享智能体的主会话；群组是隔离的（``agent:<agentId>:signal:group:<groupId>``）。

操作示例：

- `httpUrl: "http://127.0.0.1:8080",`

21. Slack

本节覆盖：

- Socket 模式（默认）
- 快速设置（新手）
- 设置

关键点：

- 创建一个 Slack 应用并启用 **Socket Mode**。
- 创建一个 **App Token**（``xapp-...``）和 **Bot Token**（``xoxb-...``）。
- 为 OpenClaw 设置令牌并启动 Gateway 网关。
- 在 <https://api.slack.com/apps> 创建一个 Slack 应用（从头开始）。
- **Socket Mode** → 开启。然后前往 **Basic Information** → **App-Level Tokens** → **Generate Token and Scopes**，添加 ``connections:write`` 权限范围。复制 **App Token**（``xapp-...``）。
- **OAuth & Permissions** → 添加 bot token 权限范围（使用下面的 manifest）。点击 **Install to Workspace**。复制 **Bot User OAuth Token**（``xoxb-...``）。
- 可选：**OAuth & Permissions** → 添加 **User Token Scopes**（参见下面的只读列表）。重新安装应用并复制 **User OAuth Token**（``xoxp-...``）。
- **Event Subscriptions** → 启用事件并订阅：

操作示例：

- `webhookPath: "/slack/events",`
- `"command": "/openclaw",`
- `replyToMode: "off",` // 频道的默认值
- `direct: "all",` // 私信始终使用线程
- `group: "first",` // 群组私信/MPIM 第一条回复使用线程

22. Telegram (Bot API)

本节覆盖：

- 快速设置（入门）
- 这是什么
- 设置（快速路径）

关键点：

- 通过 `**@BotFather**`（[直达链接](https://t.me/BotFather)）创建机器人。确认用户名确实是 ``@BotFather``，然后复制 token。
- 设置 token：
- 环境变量：``TELEGRAM_BOT_TOKEN=...``
- 或配置：``channels.telegram.botToken: "...``。
- 如果两者都设置了，配置优先（环境变量回退仅适用于默认账户）。
- 启动 Gateway 网关。
- 私信访问默认使用配对模式；首次联系时需要批准配对码。
- 一个由 Gateway 网关拥有的 Telegram Bot API 渠道。

操作示例：

- `"-1001234567890": { requireMention: false }, // 在此群组中始终响应`
- `"*": { requireMention: false }, // 所有群组，始终响应`
- `"*": { requireMention: true }, // 或完全省略 groups`
- `curl "https://api.telegram.org/bot<bot_token>/getUpdates"`
- `media: "https://example.com/voice.ogg",`
- `reactionNotifications: "all", // 查看所有反应`

23. Tlon（插件）

本节覆盖：

- 需要插件
- 设置
- 群组频道

关键点：

- 安装 Tlon 插件。
- 获取你的 ship URL 和登录代码。
- 配置 ``channels.tlon``。

- 重启 Gateway 网关。
- 私信机器人或在群组频道中提及它。
- 私信：``~sampel-palnet`` 或 ``dm/~sampel-palnet``
- 群组：``chat/~host-ship/channel`` 或 ``group::~host-ship/channel``
- 群组回复需要提及（例如 ``~your-bot-ship``）才能响应。

操作示例：

- `openclaw plugins install @openclaw/tlon`
- `openclaw plugins install ./extensions/tlon`
- `url: "https://your-ship-host",`
- `groupChannels: ["chat/~host-ship/general", "chat/~host-ship/support"],`
- `"chat/~host-ship/general": {`
- `"chat/~host-ship/announcements": {`

24. 渠道故障排除

本节覆盖：

- 渠道
- Telegram 快速修复

关键点：

- Discord：`[/channels/discord#troubleshooting] (/channels/discord#troubleshooting)`

•

Telegram：`[/channels/telegram#troubleshooting] (/channels/telegram#troubleshooting)`

- WhatsApp：`[/channels/whatsapp#troubleshooting-quick] (/channels/whatsapp#troubleshooting-quick)`

• 日志显示 ``HttpError: Network request for 'sendMessage' failed`` 或 ``sendChatAction`` → 检查 IPv6 DNS。如果 ``api.telegram.org`` 优先解析为 IPv6 而主机缺少 IPv6

出站连接，请强制使用 IPv4 或启用 IPv6。参见

`[/channels/telegram#troubleshooting] (/channels/telegram#troubleshooting)`。

• 日志显示 ``setMyCommands failed`` → 检查到 ``api.telegram.org`` 的出站 HTTPS 和 DNS 可达性（常见于限制严格的 VPS 或代理环境）。

- `channels status --probe`

会在检测到常见渠道配置错误时输出警告，并包含小型实时检查（凭据、部分权限/成员资格）。

操作示例：

- `openclaw doctor`
- `openclaw channels status --probe`

25. Twitch（插件）

本节覆盖：

- 需要插件
- 快速设置（新手）
- 它是什么

关键点：

- 为机器人创建一个专用的 Twitch 账户（或使用现有账户）。
- 生成凭证：[Twitch Token Generator](https://twitchtokengenerator.com/)
- 选择 **Bot Token**
- 确认已选择 `chat:read` 和 `chat:write` 权限范围
- 复制 **Client ID** 和 **Access Token**
- 查找你的 Twitch 用户

ID：https://www.streamweasels.com/tools/convert-twitch-username-to-user-id/

- 环境变量：`OPENCLAW_TWITCH_ACCESS_TOKEN=...`（仅限默认账户）
- 或配置：`channels.twitch.accessToken`

操作示例：

- `openclaw plugins install @openclaw/twitch`
- `openclaw plugins install ./extensions/twitch`
- `username: "openclaw",` // 机器人的 Twitch 账户
- `accessToken: "oauth:abc123...",` // OAuth Access Token（或使用 `OPENCLAW_TWITCH_ACCESS_TOKEN` 环境变量）
- `clientId: "xyz789...",` // Token Generator 中的 Client ID
- `channel: "vevisk",` // 要加入的 Twitch 频道聊天（必填）

26. WhatsApp（网页渠道）

本节覆盖：

- 快速设置（新手）
- 目标
- 配置写入

关键点：

- 如果可能，使用**单独的手机号码**（推荐）。
- 在 `~/openclaw/openclaw.json` 中配置 WhatsApp。

- 运行 `openclaw channels login` 扫描二维码（关联设备）。
- 启动 Gateway 网关。
- 在一个 Gateway 网关进程中支持多个 WhatsApp 账户（多账户）。
- 确定性路由：回复返回到 WhatsApp，无模型路由。
- 模型能看到足够的上下文来理解引用回复。
- ****Gateway 网关****拥有 Baileys socket 和收件箱循环。

操作示例：

- [/Replying]

27. Zalo (Bot API)

本节覆盖：

- 需要插件
- 快速设置（初学者）
- 它是什么

关键点：

- 通过 CLI 安装：`openclaw plugins install @openclaw/zalo`
- 或在新手引导期间选择 ****Zalo**** 并确认安装提示
- 详情：[插件](/tools/plugin)
- 安装 Zalo 插件：
- 从源代码检出：`openclaw plugins install ./extensions/zalo`
- 从 npm（如果已发布）：`openclaw plugins install @openclaw/zalo`
- 或在新手引导中选择 ****Zalo**** 并确认安装提示
- 设置 token：

28. Zalo Personal（非官方）

本节覆盖：

- 需要插件
- 前置条件：zca-cli
- 快速设置（新手）

关键点：

- 通过 CLI 安装：`openclaw plugins install @openclaw/zalouser`
- 或从源码检出安装：`openclaw plugins install ./extensions/zalouser`

- 详情：[\[插件\] \(/tools/plugin\)](#)
- 验证：``zca --version``
- 如果缺失，请安装 zca-cli（参见 ``extensions/zalouser/README.md`` 或上游 zca-cli 文档）。
- 安装插件（见上文）。
- 登录（QR，在 Gateway 网关机器上）：
- ``openclaw channels login --channel zalouser``

操作示例：

- `openclaw directory self --channel zalouser`
- `openclaw directory peers list --channel zalouser --query "name"`
- `openclaw directory groups list --channel zalouser --query "work"`

第五章 模型与提供商

1. Anthropic (Claude)

本节覆盖：

- 选项 A：Anthropic API 密钥
- CLI 设置
- 配置片段

关键点：

- `"5m"` 映射到 `short`
- `"1h"` 映射到 `long`
- 使用 `claude setup-token` 生成 setup-token 并粘贴，或在 Gateway 网关主机上运行 `openclaw models auth setup-token`。
- 如果你在 Claude 订阅上看到"OAuth token refresh failed ..."，请使用 setup-token 重新认证。参见 [\[/gateway/troubleshooting#oauth-token-refresh-failed-anthropic-claude-subscription\]](/gateway/troubleshooting#oauth-token-refresh-failed-anthropic-claude-subscription) (</gateway/troubleshooting#oauth-token-refresh-failed-anthropic-claude-subscription>)。
- 认证详情 + 重用规则在 [\[/concepts/oauth\]](/concepts/oauth) (</concepts/oauth>)。
- Claude 订阅认证可能过期或被撤销。重新运行 `claude setup-token`
- 如果 Claude CLI 登录在不同的机器上，在 Gateway 网关主机上使用
- 认证是**按智能体**的。新智能体不会继承主智能体的密钥。

操作示例：

- `openclaw onboard`
- `openclaw onboard --anthropic-api-key "$ANTHROPIC_API_KEY"`
- `agents: { defaults: { model: { primary: "anthropic/claude-opus-4-5" } } },`
- `"anthropic/claude-opus-4-5": {`
- `openclaw models auth setup-token --provider anthropic`
- `openclaw models auth paste-token --provider anthropic`

2. Amazon Bedrock

本节覆盖：

- piai 支持的功能
- 自动模型发现
- 设置（手动）

关键点：

- 提供商：`amazon-bedrock`
- API：`bedrock-converse-stream`
- 认证：AWS 凭证（环境变量、共享配置或实例角色）
- 区域：`AWS\_REGION` 或 `AWS\_DEFAULT\_REGION`（默认：`us-east-1`）
- `enabled` 在存在 AWS 凭证时默认为 `true`。
- `region` 默认为 `AWS\_REGION` 或 `AWS\_DEFAULT\_REGION`，然后是 `us-east-1`。
- `providerFilter` 匹配 Bedrock 提供商名称（例如 `anthropic`）。
- `refreshInterval` 单位为秒；设置为 `0` 可禁用缓存。

操作示例：

- # 可选（Bedrock API 密钥/Bearer 令牌）：
- baseUrl: "https://bedrock-runtime.us-east-1.amazonaws.com",
- model: { primary: "amazon-bedrock/anthropic.claude-opus-4-5-20251101-v1:0" },
- # 添加到 ~/.bashrc 或你的 shell 配置文件
- aws iam create-role --role-name EC2-Bedrock-Access \
- aws iam attach-role-policy --role-name EC2-Bedrock-Access \

3. Claude Max API 代理

本节覆盖：

- 为什么使用它？
- 工作原理
- 安装

关键点：

- 在 `http://localhost:3456/v1/chat/completions` 接受 OpenAI 格式的请求
- 将其转换为 Claude Code CLI 命令
- 以 OpenAI 格式返回响应（支持流式传输）
- ****npm:**** <https://www.npmjs.com/package/claude-max-api-proxy>
- ****GitHub:**** <https://github.com/atalovesyou/claude-max-api-proxy>
- ****Issues:**** <https://github.com/atalovesyou/claude-max-api-proxy/issues>
- 这是一个****社区工具****，并非由 Anthropic 或 OpenClaw 官方支持
- 需要有效的 Claude Max/Pro 订阅并已认证 Claude Code CLI

操作示例：

- npm install -g claude-max-api-proxy

- `claude --version`
- # 服务器运行在 `http://localhost:3456`
- `curl http://localhost:3456/health`
- `curl http://localhost:3456/v1/models`
- `curl http://localhost:3456/v1/chat/completions \`

4. Deepgram（音频转录）

本节覆盖：

- 快速开始
- 选项
- 注意事项

关键点：

- 设置你的 API 密钥：
- 启用提供商：
- ``model``：Deepgram 模型 ID（默认：``nova-3``）
- ``language``：语言提示（可选）
- ``tools.media.audio.providerOptions.deepgram.detect_language``：启用语言检测（可选）
- ``tools.media.audio.providerOptions.deepgram.punctuate``：启用标点符号（可选）
- ``tools.media.audio.providerOptions.deepgram.smart_format``：启用智能格式化（可选）
- 使用代理时，可通过 ``tools.media.audio.baseUrl`` 和 ``tools.media.audio.headers`` 覆盖端点或请求头。

5. GitHub Copilot

本节覆盖：

- 什么是 GitHub Copilot？
- 在 OpenClaw 中使用 Copilot 的两种方式
- 1) 内置 GitHub Copilot 提供商（``github-copilot``）

关键点：

- 需要交互式 TTY；请直接在终端中运行。
- Copilot 模型的可用性取决于你的订阅计划；如果某个模型被拒绝，请尝试其他 ID（例如 ``github-copilot/gpt-4.1``）。

- 登录会将 GitHub 令牌存储在认证配置文件中，并在 OpenClaw 运行时将其兑换为 Copilot API 令牌。
- GitHub Copilot 是 GitHub 的 AI 编程助手。它为你的 GitHub 账户和订阅计划提供 Copilot 模型的访问权限。OpenClaw 可以通过两种不同的方式将 Copilot 用作模型提供商。
- 使用原生设备登录流程获取 GitHub 令牌，然后在 OpenClaw 运行时将其兑换为 Copilot API 令牌。这是默认且最简单的方式，因为它不需要 VS Code。
- 使用 Copilot Proxy VS Code 扩展作为本地桥接。OpenClaw 与代理的 /v1 端点通信，并使用你在其中配置的模型列表。当你已经在 VS Code 中运行 Copilot Proxy 或需要通过它进行路由时，选择此方式。你必须启用该插件并保持 VS Code 扩展运行。
- 使用 GitHub Copilot 作为模型提供商 (github-copilot)。登录命令运行 GitHub 设备流程，保存认证配置文件，并更新你的配置以使用该配置文件。
- 系统会提示你访问一个 URL 并输入一次性代码。请保持终端打开直到流程完成。

操作示例：

- `openclaw models auth login-github-copilot`
- `openclaw models auth login-github-copilot --profile-id github-copilot:work`
- `openclaw models auth login-github-copilot --yes`
- `openclaw models set github-copilot/gpt-4o`
- `agents: { defaults: { model: { primary: "github-copilot/gpt-4o" } } },`

6. GLM 模型

本节覆盖：

- CLI 设置
- 配置片段
- 注意事项

关键点：

- GLM 版本和可用性可能会变化；请查阅 Z.AI 的文档获取最新信息。
- 示例模型 ID 包括 `glm-4.7` 和 `glm-4.6`。
- 有关提供商的详细信息，请参阅 `[/providers/zai]([/providers/zai])`。
- GLM 是一个模型系列（而非公司），通过 Z.AI 平台提供。在 OpenClaw 中，GLM 模型通过 zai 提供商访问，模型 ID 格式如 `zai/glm-4.7`。

操作示例：

- `openclaw onboard --auth-choice zai-api-key`
- `agents: { defaults: { model: { primary: "zai/glm-4.7" } } },`

7. 模型提供商

本节覆盖：

- 亮点：Venice (Venice AI)
- 快速开始
- 提供商文档

关键点：

- 默认：`venice/llama-3.3-70b`
- 最佳综合：`venice/claude-opus-45` (Opus 仍然是最强的)
- 与提供商进行认证 (通常通过 `openclaw onboard`)。
- 设置默认模型：
- [OpenAI (API + Codex)](/providers/openai)
- [Anthropic (API + Claude Code CLI)](/providers/anthropic)
- [Qwen (OAuth)](/providers/qwen)
- [OpenRouter](/providers/openrouter)

操作示例：

- `agents: { defaults: { model: { primary: "anthropic/claude-opus-4-5" } } },`

8. MiniMax

本节覆盖：

- 模型概述 (M2.1)
- MiniMax M2.1 vs MiniMax M2.1 Lightning
- 选择设置方式

关键点：

- 更强的**多语言编程**能力 (Rust、Java、Go、C++、Kotlin、Objective-C、TS/JS)。
- 更好的**Web/应用开发**和美观输出质量 (包括原生移动端)。
- 改进的**复合指令**处理，适用于办公风格的工作流程，基于交错思考和集成约束执行。
- **更简洁的响应**，更低的 token 使用量和更快的迭代循环。
- 更强的**工具/智能体框架**兼容性和上下文管理 (Claude Code、Droid/Factory AI、Cline、Kilo Code、Roo Code、BlackBox)。
- 更高质量的**对话和技术写作**输出。
- **速度**：Lightning 是 MiniMax 定价文档中的"快速"变体。
- **成本**：定价显示相同的输入成本，但 Lightning 的输出成本更高。

操作示例：

- `openclaw plugins enable minimax-portal-auth` # 如果已加载则跳过
- `openclaw gateway restart` # 如果 Gateway 网关已在运行则重启
- `openclaw onboard --auth-choice minimax-portal`
- `agents: { defaults: { model: { primary: "minimax/MiniMax-M2.1" } } },`
- `baseUrl: "https://api.minimax.io/anthropic",`
- `"anthropic/claude-opus-4-5": { alias: "opus" },`

9. 模型提供商

本节覆盖：

- 推荐：Venice (Venice AI)
- 快速开始（两个步骤）
- 支持的提供商（入门集）

关键点：

- 默认：``venice/llama-3.3-70b``
- 最佳综合：``venice/claude-opus-45`` (Opus 仍然是最强的)
- 与提供商认证（通常通过 ``openclaw onboard``）。
- 设置默认模型：
- `[OpenAI (API + Codex) ](/providers/openai)`
- `[Anthropic (API + Claude Code CLI) ](/providers/anthropic)`
- `[OpenRouter](/providers/openrouter)`
- `[Vercel AI Gateway](/providers/vercel-ai-gateway)`

操作示例：

- `agents: { defaults: { model: { primary: "anthropic/claude-opus-4-5" } } },`

10. Moonshot AI (Kimi)

本节覆盖：

- 配置片段 (Moonshot API)
- Kimi Coding
- 注意事项

关键点：

- ``kimi-k2.5``

- ``kimi-k2-0905-preview``
- ``kimi-k2-turbo-preview``
- ``kimi-k2-thinking``
- ``kimi-k2-thinking-turbo``
- Moonshot 模型引用使用 ``moonshot/<modelId>``。Kimi Coding 模型引用使用 ``kimi-coding/<modelId>``。
- 如有需要，可在 ``models.providers`` 中覆盖定价和上下文元数据。
- 如果 Moonshot 发布了某个模型的不同上下文限制，请相应调整 ``contextWindow``。

操作示例：

- `openclaw onboard --auth-choice moonshot-api-key`
- `openclaw onboard --auth-choice kimi-code-api-key`
- `model: { primary: "moonshot/kimi-k2.5" },`
- `// moonshot-kimi-k2-aliases:start`
- `"moonshot/kimi-k2.5": { alias: "Kimi K2.5" },`
- `"moonshot/kimi-k2-0905-preview": { alias: "Kimi K2" },`

11. Ollama

本节覆盖：

- 快速开始
- 模型发现（隐式提供商）
- 配置

关键点：

- 安装 Ollama：<https://ollama.ai>
- 为 OpenClaw 启用 Ollama（任意值即可；Ollama 不需要真实密钥）：
- 使用 Ollama 模型：
- 查询 ``/api/tags`` 和 ``/api/show``
- 仅保留报告了 ``tools`` 能力的模型
- 当模型报告 ``thinking`` 时标记为 ``reasoning``
- 在可用时从 ``model_info["<arch>.context_length"]`` 读取 ``contextWindow``
- 将 ``maxTokens`` 设置为上下文窗口的 10 倍

操作示例：

- `openclaw config set models.providers.ollama.apiKey "ollama-local"`

- model: { primary: "ollama/llama3.3" },
- openclaw models list
- // 使用包含 /v1 的主机地址以兼容 OpenAI API
- baseUrl: "http://ollama-host:11434/v1",
- baseUrl: "http://ollama-host:11434/v1",

12. OpenAI

本节覆盖：

- 方式 A：OpenAI API 密钥（OpenAI Platform）
- CLI 设置
- 配置片段

关键点：

- 模型引用始终使用 `provider/model` 格式（参见 [\[/concepts/models\]\(/concepts/models\)](#)）。
- 认证详情和复用规则请参阅 [\[/concepts/oauth\]\(/concepts/oauth\)](#)。
- OpenAI 提供 GPT 模型的开发者 API。Codex 支持 ChatGPT 登录进行订阅访问，或 API 密钥登录进行按量计费访问。Codex 云端需要 ChatGPT 登录。
- 适用于：直接 API 访问和按量计费。
- 从 OpenAI 控制台获取你的 API 密钥。
- 适用于：使用 ChatGPT/Codex 订阅访问而非 API 密钥。
- Codex 云端需要 ChatGPT 登录，而 Codex CLI 支持 ChatGPT 或 API 密钥登录。

操作示例：

- openclaw onboard --auth-choice openai-api-key
- openclaw onboard --openai-api-key "\$OPENAI_API_KEY"
- agents: { defaults: { model: { primary: "openai/gpt-5.2" } } },
- openclaw onboard --auth-choice openai-codex
- openclaw models auth login --provider openai-codex
- agents: { defaults: { model: { primary: "openai-codex/gpt-5.2" } } },

13. OpenCode Zen

本节覆盖：

- CLI 设置
- 配置片段

- 注意事项

关键点：

- 也支持 `OPENCODE\_ZEN\_API\_KEY` 。
- 你需要登录 Zen，添加账单信息，然后复制你的 API 密钥。
- OpenCode Zen 按请求计费；详情请查看 OpenCode 控制台。
- OpenCode Zen 是由 OpenCode 团队推荐的一组精选模型列表，适用于编程智能体。它是一个可选的托管模型访问路径，使用 API 密钥和 opencode 提供商。Zen 目前处于测试阶段。

操作示例：

- `openclaw onboard --auth-choice opencode-zen`
- `openclaw onboard --opencode-zen-api-key "$OPENCODE_API_KEY"`
- `agents: { defaults: { model: { primary: "opencode/claude-opus-4-5" } } },`

14. OpenRouter

本节覆盖：

- CLI 设置
- 配置片段
- 注意事项

关键点：

- 模型引用格式为 `openrouter/<provider>/<model>` 。
- 更多模型/提供商选项，请参阅[模型提供商](/concepts/model-providers) 。
- OpenRouter 底层使用 Bearer 令牌和你的 API 密钥进行认证。
- OpenRouter 提供了一个统一 API，通过单一端点和 API 密钥将请求路由到多种模型。它兼容 OpenAI，因此大多数 OpenAI SDK 只需切换 base URL 即可使用。

操作示例：

- `openclaw onboard --auth-choice apiKey --token-provider openrouter --token "$OPENROUTER_API_KEY"`
- `model: { primary: "openrouter/anthropic/claude-sonnet-4-5" },`

15. 千帆 (Qianfan)

关键点：

- 该页面是英文文档的中文占位版本，完整内容请先参考英文版：Qianfan。

16. Qwen

本节覆盖：

- 启用插件
- 认证
- 模型 ID

关键点：

- ``qwen-portal/coder-model``
- ``qwen-portal/vision-model``
- 令牌自动刷新；如果刷新失败或访问被撤销，请重新运行登录命令。
- 默认基础 URL：``https://portal.qwen.ai/v1``（如果 Qwen 提供不同的端点，使用 ``models.providers.qwen-portal.baseUrl`` 覆盖）。
- 参阅[模型提供商](/concepts/model-providers)了解提供商级别的规则。
- Qwen 为 Qwen Coder 和 Qwen Vision 模型提供免费层 OAuth 流程（每天 2,000 次请求，受 Qwen 速率限制约束）。
- 启用后重启 Gateway 网关。
- 这会运行 Qwen 设备码 OAuth 流程并将提供商条目写入你的 `models.json`（加上一个 `qwen` 别名以便快速切换）。

操作示例：

- `openclaw plugins enable qwen-portal-auth`
- `openclaw models auth login --provider qwen-portal --set-default`
- `openclaw models set qwen-portal/coder-model`

17. Synthetic

本节覆盖：

- 快速设置
- 配置示例
- 模型目录

关键点：

- 设置 ``SYNTHETIC_API_KEY``（或运行以下向导）。
- 运行新手引导：
- 模型引用格式为 ``synthetic/<modelId>``。
- 如果启用了模型允许列表（``agents.defaults.models``），请添加你计划使用的所有模型。
- 参阅[模型提供商](/concepts/model-providers)了解提供商规则。

- Synthetic 提供兼容 Anthropic 的端点。OpenClaw 将其注册为 synthetic 提供商，并使用 Anthropic Messages API。
- 默认模型设置为：
- 注意：OpenClaw 的 Anthropic 客户端会自动在 base URL 后追加 /v1，因此请使用 /anthropic/v1)。如果 Synthetic 更改了其 base URL，请覆盖 `models.providers.synthetic.baseUrl``。

操作示例：

- `openclaw onboard --auth-choice synthetic-api-key`
- `synthetic/hf:MiniMaxAI/MiniMax-M2.1`
- `model: { primary: "synthetic/hf:MiniMaxAI/MiniMax-M2.1" },`
- `models: { "synthetic/hf:MiniMaxAI/MiniMax-M2.1": { alias: "MiniMax M2.1" } },`
- `baseUrl: "https://api.synthetic.new/anthropic",`
- `id: "hf:MiniMaxAI/MiniMax-M2.1",`

18. Venice AI (Venice 精选)

本节覆盖：

- 为什么在 OpenClaw 中使用 Venice
- 隐私模式
- 功能特性

关键点：

- ****私密推理****，适用于开源模型（无日志记录）。
- 需要时可使用****无审查模型****。
- 在质量重要时，可****匿名访问****专有模型（Opus/GPT/Gemini）。
- 兼容 OpenAI 的 ``/v1`` 端点。
- ****注重隐私****：可选择"私密"（完全私密）和"匿名化"（代理转发）模式
- ****无审查模型****：访问无内容限制的模型
- ****主流模型访问****：通过 Venice 匿名代理使用 Claude、GPT-5.2、Gemini、Grok
- ****兼容 OpenAI API****：标准 ``/v1`` 端点，易于集成

操作示例：

- `openclaw onboard --auth-choice venice-api-key`
- `openclaw onboard --non-interactive \`
- `openclaw chat --model venice/llama-3.3-70b "Hello, are you working?"`
- `openclaw models set venice/claude-opus-45`

- openclaw models set venice/llama-3.3-70b
- openclaw models list | grep venice

19. Vercel AI Gateway

本节覆盖：

- 快速开始
- 非交互式示例
- 环境变量说明

关键点：

- 提供商：`vercel-ai-gateway`
- 认证：`AI\_GATEWAY\_API\_KEY`
- API：兼容 Anthropic Messages
- 设置 API 密钥（推荐：为 Gateway 网关存储它）：
- 设置默认模型：
- Vercel AI Gateway 提供了一个统一的 API，通过单一端点访问数百个模型。
- 如果 Gateway 网关作为守护进程运行（launchd/systemd），请确保 AI_GATEWAY_API_KEY
- 对该进程可用（例如，在 ~/.openclaw/.env 中或通过

操作示例：

- openclaw onboard --auth-choice ai-gateway-api-key
- model: { primary: "vercel-ai-gateway/anthropic/claude-opus-4.5" },
- openclaw onboard --non-interactive \

20. Xiaomi MiMo

本节覆盖：

- 模型概览
- CLI 设置
- 配置片段

关键点：

- **mimo-v2-flash**：262144 token 上下文窗口，兼容 Anthropic Messages API。
- 基础 URL：`https://api.xiaomimimo.com/anthropic`
- 授权方式：`Bearer \$XIAOMI\_API\_KEY`
- 模型引用：`xiaomi/mimo-v2-flash`。

- 当设置了 `XIAOMI\_API\_KEY`（或存在身份验证配置文件）时，该提供商会自动注入。
- 有关提供商规则，请参阅 /concepts/model-providers。
- Xiaomi MiMo 是 MiMo 模型的 API 平台。它提供与 OpenAI 和 Anthropic 格式兼容的 REST API，并使用 API 密钥进行身份验证。请在 Xiaomi MiMo 控制台 中创建你的 API 密钥。OpenClaw 使用 xiaomi 提供商配合 Xiaomi MiMo API 密钥。

操作示例：

- `openclaw onboard --auth-choice xiaomi-api-key`
- `openclaw onboard --auth-choice xiaomi-api-key --xiaomi-api-key "$XIAOMI_API_KEY"`
- `agents: { defaults: { model: { primary: "xiaomi/mimo-v2-flash" } } },`
- `baseUrl: "https://api.xiaomimimo.com/anthropic",`

21. Z.AI

本节覆盖：

- CLI 设置
- 配置片段
- 注意事项

关键点：

- GLM 模型以 `zai/<model>` 的形式提供（例如：`zai/glm-4.7`）。
- 参阅 /providers/glm 了解模型系列概览。
- Z.AI 使用 Bearer 认证方式配合你的 API 密钥。
- Z.AI 是 GLM 模型的 API 平台。它为 GLM 提供 REST API，并使用 API 密钥进行身份验证。请在 Z.AI 控制台中创建你的 API 密钥。OpenClaw 通过 zai 提供商配合 Z.AI API 密钥使用。

操作示例：

- `openclaw onboard --auth-choice zai-api-key`
- `openclaw onboard --zai-api-key "$ZAI_API_KEY"`
- `agents: { defaults: { model: { primary: "zai/glm-4.7" } } },`

第六章 工具体系

1. `openclaw agent`（直接智能体运行）

本节覆盖：

- 行为
- 示例
- 标志

关键点：

- 必需：`--message <text>`
- `--to <dest>` 派生会话键（群组/频道目标保持隔离；直接聊天折叠到 `main`），**或**
- `--session-id <id>` 通过 ID 重用现有会话，**或**
- `--agent <id>` 直接定位已配置的智能体（使用该智能体的 `main` 会话键）
- 运行与正常入站回复相同的嵌入式智能体运行时。
- 思考/详细标志持久化到会话存储中。
- 默认：打印回复文本（加上 `MEDIA:<url>` 行）
- `--json`：打印结构化负载 + 元数据

操作示例：

- `openclaw agent --to +15555550123 --message "status update"`
- `openclaw agent --agent ops --message "Summarize logs"`
- `openclaw agent --session-id 1234 --message "Summarize inbox" --thinking medium`
- `openclaw agent --to +15555550123 --message "Trace logs" --verbose on --json`
- `openclaw agent --to +15555550123 --message "Summon reply" --deliver`
- `openclaw agent --agent ops --message "Generate report" --deliver --reply-channel slack --reply-to "#reports"`

2. apply_patch 工具

本节覆盖：

- 参数
- 说明
- 示例

关键点：

- `input`（必需）：完整的补丁内容，包括 `\*\*\* Begin Patch` 和 `\*\*\* End Patch`。
- 路径相对于工作区根目录解析。

- 在 `\*\*\* Update File:` 段中使用 `\*\*\* Move to:` 可重命名文件。
- 需要时使用 `\*\*\* End of File` 标记仅在文件末尾的插入。
- 实验性功能，默认禁用。通过 `tools.exec.applyPatch.enabled` 启用。
- 仅限 OpenAI（包括 OpenAI Codex）。可选通过
- 配置仅在 `tools.exec` 下。
- 使用结构化补丁格式应用文件更改。这非常适合多文件

操作示例：

- `*** Add File: path/to/file.txt`
- `*** Update File: src/app.ts`
- `"input": "*** Begin Patch\n*** Update File: src/index.ts\n@@\n-const foo = 1\n+const foo = 2\n*** End Patch"`

3. 浏览器故障排除（Linux）

本节覆盖：

- 问题："Failed to start Chrome CDP on port 18800"
- 根本原因
- 解决方案 1：安装 Google Chrome（推荐）

关键点：

- 手动启动 Chromium：
- 可选创建 systemd 用户服务以自动启动 Chrome：
- `**使用托管浏览器：**`openclaw browser start --browser-profile openclaw``
- `**使用扩展中继：**` 安装扩展，打开一个标签页，然后点击 OpenClaw 扩展图标来附加它。
- ``chrome`` 配置文件在可能时使用你的`**系统默认 Chromium 浏览器**`。
- 本地 ``openclaw`` 配置文件自动分配 ``cdpPort``/``cdpUrl``；仅为远程 CDP 设置这些。
- OpenClaw 的浏览器控制服务器无法启动

Chrome/Brave/Edge/Chromium，出现以下错误：

- 在 Ubuntu（和许多 Linux 发行版）上，默认的 Chromium 安装是 snap 包。Snap 的 AppArmor 限制会干扰 OpenClaw 启动和监控浏览器进程的方式。

操作示例：

- `sudo apt --fix-broken install -y # if there are dependency errors`
- `chromium-browser --headless --no-sandbox --disable-gpu \`
- `--user-data-dir=$HOME/.openclaw/browser/openclaw/user-data \`
- `# ~/.config/systemd/user/openclaw-browser.service`

- `ExecStart=/snap/bin/chromium --headless --no-sandbox --disable-gpu`
`--remote-debugging-port=18800`
`--user-data-dir=%h/.openclaw/browser/openclaw/user-data about:blank`
- `curl -s http://127.0.0.1:18791/ | jq '{running, pid, chosenBrowser}'`

4. 浏览器登录 + X/Twitter 发帖

本节覆盖：

- 手动登录（推荐）
- 使用哪个 Chrome 配置文件？
- X/Twitter：推荐流程

关键点：

- **让智能体打开浏览器**，然后你自己登录。
- **通过 CLI 打开**：
- **阅读/搜索/话题**：使用 **bird** CLI Skills（无浏览器，稳定）。
- 仓库：<https://github.com/steipete/bird>
- **发布更新**：使用**主机**浏览器（手动登录）。
- 当网站需要登录时，请在主机浏览器配置文件（openclaw 浏览器）中手动登录。
- 不要将你的凭证提供给模型。自动登录通常会触发反机器人防御并可能锁定账户。
- 返回主浏览器文档：浏览器。

操作示例：

- `openclaw browser start`
- `openclaw browser open https://x.com`
- `openclaw browser open https://x.com --browser-profile openclaw --target host`

5. 浏览器（openclaw 托管）

本节覆盖：

- 功能概览
- 快速开始
- 配置文件：`openclaw` 与 `chrome`

关键点：

- 把它想象成一个**独立的、仅供智能体使用的浏览器**。
- `openclaw` 配置文件**不会**触及你的个人浏览器配置文件。
- 智能体可以在安全的通道中**打开标签页、读取页面、点击和输入**。

- 默认的 `chrome` 配置文件通过扩展中继使用**系统默认的 Chromium 浏览器**；切换到 `openclaw` 可使用隔离的托管浏览器。
- 一个名为 **openclaw** 的独立浏览器配置文件（默认橙色主题）。
- 确定性标签页控制（列出/打开/聚焦/关闭）。
- 智能体操作（点击/输入/拖动/选择）、快照、截图、PDF。
- 可选的多配置文件支持（`openclaw`、`work`、`remote` 等）。

操作示例：

- `openclaw browser --browser-profile openclaw status`
- `openclaw browser --browser-profile openclaw start`
- `openclaw browser --browser-profile openclaw open https://example.com`
- `openclaw browser --browser-profile openclaw snapshot`
- `enabled: true, // default: true`
- `// cdpUrl: "http://127.0.0.1:18792", // legacy single-profile override`

6. Chrome 扩展（浏览器中继）

本节覆盖：

- 它是什么（概念）
- 安装/加载（未打包）
- 更新（无构建步骤）

关键点：

- **浏览器控制服务**（Gateway 网关或节点）：智能体/工具调用的 API（通过 Gateway 网关）
- **本地中继服务器**（loopback CDP）：在控制服务器和扩展之间桥接（默认 `http://127.0.0.1:18792`）
- **Chrome MV3 扩展**：使用 `chrome.debugger` 附加到活动标签页，并将 CDP 消息传送到中继
- 将扩展安装到稳定的本地路径：
- 打印已安装扩展的目录路径：
- Chrome → `chrome://extensions`
- 启用"开发者模式"
- "加载已解压的扩展程序" → 选择上面打印的目录

操作示例：

- `openclaw browser extension install`

- openclaw browser extension path
- openclaw browser create-profile \
- --cdp-url http://127.0.0.1:18792 \

7. ClawHub

本节覆盖：

- 适用人群（新手友好）
- 快速入门（非技术人员）
- 安装 CLI

关键点：

- 使用自然语言搜索 Skills。
- 将 Skills 安装到你的工作区。
- 之后使用一条命令更新 Skills。
- 通过发布 Skills 来备份你自己的 Skills。
- 安装 CLI（参见下一节）。
- 搜索你需要的内容：
- ``clawhub search "calendar"```
- 安装一个 Skills：

操作示例：

- `npm i -g clawhub`
- `clawhub update --all`
- `clawhub publish ./my-skill --slug my-skill --name "My Skill" --version 1.0.0 --tags latest`
- `clawhub sync --all`

8. 创建自定义 Skills

本节覆盖：

- 什么是 Skill？
- 分步指南：你的第一个 Skill
- 1. 创建目录

关键点：

- ****简洁明了****：指示模型*做什么*，而不是如何成为一个 AI。
- ****安全第一****：如果你的 Skill 使用 ``bash``，确保提示词不允许来自不受信任用户输入的任意命令注入。

- ****本地测试****：使用 ``openclaw agent --message "use my new skill"`` 进行测试。
- OpenClaw 被设计为易于扩展。"Skills"是为你的助手添加新功能的主要方式。
- Skill 是一个包含 SKILL.md 文件（为 LLM 提供指令和工具定义）的目录，可选包含一些脚本或资源。
- Skills 位于你的工作区中，通常是 `~/.openclaw/workspace/skills/`。为你的 Skill 创建一个新文件夹：
- 在该目录中创建一个 SKILL.md 文件。此文件使用 YAML frontmatter 作为元数据，使用 Markdown 作为指令。
- 你可以在 frontmatter 中定义自定义工具，或指示智能体使用现有的系统工具（如 bash 或 browser）。

操作示例：

- `mkdir -p ~/.openclaw/workspace/skills/hello-world`

9. 提升模式（/elevated 指令）

本节覆盖：

- 功能说明
- 它控制什么（以及不控制什么）
- 设置会话默认值

关键点：

- ``/elevated on`` 在 Gateway 网关主机上运行并保留 exec 审批（与 ``/elevated ask`` 相同）。
- ``/elevated full`` 在 Gateway 网关主机上运行**并**自动批准 exec（跳过 exec 审批）。
- ``/elevated ask`` 在 Gateway 网关主机上运行但保留 exec 审批（与 ``/elevated on`` 相同）。
- ``on`/`ask`` **不会**强制****``exec.security=full``；配置的安全/询问策略仍然适用。
- 仅在智能体被**沙箱隔离**时改变行为（否则 exec 已经在主机上运行）。
- 指令形式：``/elevated on|off|ask|full``、``/elev on|off|ask|full``。
- 仅接受 ``on|off|ask|full``；其他任何内容返回提示且不改变状态。
- ****可用性门控****：``tools.elevated`` 是全局基线。``agents.list[].tools.elevated`` 可以进一步限制每个智能体的提升（两者都必须允许）。

10. 执行审批

本节覆盖：

- 适用范围
- 设置和存储
- 策略选项

关键点：

- **gateway 主机** → gateway 机器上的 ``openclaw`` 进程
- **node 主机** → 节点运行器（macOS 配套应用或无头节点主机）
- **node 主机服务**通过本地 IPC 将 ``system.run`` 转发给 **macOS 应用**。
- **macOS 应用**执行审批并在 UI 上下文中执行命令。
- **deny**：阻止所有主机执行请求。
- **allowlist**：仅允许在允许列表中的命令。
- **full**：允许所有命令（等同于提权模式）。
- **off**：从不提示。

操作示例：

- `"path": "~/openclaw/exec-approvals.sock",`
- `"pattern": "~/Projects/**/bin/rg",`
- `"lastResolvedPath": "/Users/user/Projects/.../bin/rg"`
- `mode: "session", // "session" | "targets" | "both"`
- `sessionFilter: ["discord"], // substring or regex`
- `/approve <id> allow-once`

11. Exec 工具

本节覆盖：

- 参数
- 配置
- PATH 处理

关键点：

- ``command``（必填）
- ``workdir``（默认为当前工作目录）
- ``env``（键值对覆盖）
- ``yieldMs``（默认 10000）：延迟后自动转入后台
- ``background``（布尔值）：立即转入后台
- ``timeout``（秒，默认 1800）：超时后终止
- ``pty``（布尔值）：在可用时使用伪终端运行（仅限 TTY 的 CLI、编程智能体、终端 UI）
- ``host``（``sandbox | gateway | node``）：执行位置

操作示例：

- pathPrepend: ["~/bin", "/opt/oss/bin"],
- openclaw config get agents.list
- openclaw config set agents.list[0].tools.exec.node "node-id-or-name"
- /exec host=gateway security=allowlist ask=on-miss node=mac-1

12. Firecrawl

本节覆盖：

- 获取 API 密钥
- 配置 Firecrawl
- 隐身 / 机器人规避

关键点：

- 创建 Firecrawl 账户并生成 API 密钥。
- 将其存储在配置中或在 Gateway 网关环境中设置 `FIRECRAWL\_API\_KEY`。
- 当存在 API 密钥时，`firecrawl.enabled` 默认为 true。
- `maxAgeMs` 控制缓存结果可以保留多久（毫秒）。默认为 2 天。
- Readability（本地）
- Firecrawl（如果已配置）
- 基本 HTML 清理（最后回退）
- OpenClaw 可以使用 Firecrawl 作为 web_fetch 的回退提取器。它是一个托管的

操作示例：

- baseUrl: "https://api.firecrawl.dev",

13. 工具（OpenClaw）

本节覆盖：

- 禁用工具
- 工具配置文件（基础允许列表）
- 特定提供商的工具策略

关键点：

- 匹配不区分大小写。
- 支持 `\*` 通配符（`"\*"` 表示所有工具）。
- 如果 `tools.allow` 仅引用未知或未加载的插件工具名称，OpenClaw 会记录警告并忽略允许列表，以确保核心工具保持可用。
- `minimal`：仅 `session\_status`

- ``coding`` : ``group:fs``、``group:runtime``、``group:sessions``、``group:memory``、``image``
- ``messaging`` : ``group:messaging``、``sessions_list``、``sessions_history``、``sessions_send``、``session_status``
- ``full`` : 无限制（与未设置相同）
- ``group:runtime`` : ``exec``、``bash``、``process``

操作示例：

- `"openai/gpt-5.2": { allow: ["group:fs", "sessions_list"] },`

14. LLM 任务

本节覆盖：

- 启用插件
- 配置（可选）
- 工具参数

关键点：

- 将工具加入允许列表（它以 ``optional: true`` 注册）：
- ``prompt``（字符串，必填）
- ``input``（任意类型，可选）
- ``schema``（对象，可选 JSON Schema）
- ``provider``（字符串，可选）
- ``model``（字符串，可选）
- ``authProfileId``（字符串，可选）
- ``temperature``（数字，可选）

操作示例：

- `"allowedModels": ["openai-codex/gpt-5.2"],`
- `openclaw.invoke --tool llm-task --action json --args-json '{`

15. Lobster

本节覆盖：

- 亮点
- 为什么
- 为什么用 DSL 而不是普通程序？

关键点：

- ****一次调用代替多次****：OpenClaw 运行一次 Lobster 工具调用并获得结构化结果。

- ****内置审批****：副作用（发送邮件、发布评论）会暂停工作流，直到明确批准。
- ****可恢复****：暂停的工作流返回一个令牌；批准并恢复而无需重新运行所有内容。
- ****内置批准/恢复****：普通程序可以提示人类，但它无法*暂停和恢复*并带有持久令牌，除非你自己发明那个运行时。
- ****确定性 + 可审计性****：管道是数据，所以它们易于记录、比较、重放和审查。
- ****AI 的受限表面****：微小的语法 + JSON 管道减少了"创造性"代码路径，使验证变得现实可行。
- ****内置安全策略****：超时、输出上限、沙箱检查和白名单由运行时强制执行，而不是每个脚本。
- ****仍然可编程****：每个步骤都可以调用任何 CLI 或脚本。如果你想要 JS/TS，可以从代码生成`.lobster`文件。

操作示例：

- `inbox list --json`
- `inbox categorize --json`
- `inbox apply --json`
- `"pipeline": "exec --json --shell 'inbox list --json' | exec --stdin json --shell 'inbox categorize --json' | exec --stdin json --shell 'inbox apply --json' | approve --preview-from-stdin --limit 5 --prompt 'Apply changes?'"`
- `gog.gmail.search --query 'newer_than:1d' \`
- `| openclaw.invoke --tool message --action send --each --item-key message --args-json '{"provider":"telegram","to":"..."}'`

16. 多智能体沙箱与工具配置

本节覆盖：

- 概述
- 配置示例
- 示例 1：个人 + 受限家庭智能体

关键点：

- ****沙箱配置****（``agents.list[].sandbox`` 覆盖 ``agents.defaults.sandbox``）
- ****工具限制****（``tools.allow`` / ``tools.deny``，以及 ``agents.list[].tools``）
- 具有完全访问权限的个人助手
- 具有受限工具的家庭/工作智能体
- 在沙箱中运行的面向公众的智能体
- ``main`` 智能体：在主机上运行，完全工具访问
- ``family`` 智能体：在 Docker 中运行（每个智能体一个容器），仅有 ``read`` 工具

- 默认智能体获得编码工具

操作示例：

- `~/openclaw/agents/<agentId>/agent/auth-profiles.json`
- `"workspace": "~/openclaw/workspace",`
- `"workspace": "~/openclaw/workspace-family",`
- `"workspace": "~/openclaw/workspace-personal",`
- `"workspace": "~/openclaw/workspace-work",`
- `"workspaceRoot": "/tmp/work-sandboxes"`

17. 插件（扩展）

本节覆盖：

- 快速开始（插件新手？）
- 可用插件（官方）
- 运行时辅助工具

关键点：

- 查看已加载的内容：
- 安装官方插件（例如：Voice Call）：
- 重启 Gateway 网关，然后在 ``plugins.entries.<id>.config`` 下配置。
- 从 2026.1.15 起 Microsoft Teams 仅作为插件提供；如果使用 Teams，请安装 ``@openclaw/msteams``。
- Memory (Core) — 捆绑的记忆搜索插件（通过 ``plugins.slots.memory`` 默认启用）
- Memory (LanceDB) — 捆绑的长期记忆插件（自动召回/捕获；设置 ``plugins.slots.memory = "memory-lancedb"``）
- [Voice Call] (/plugins/voice-call) — ``@openclaw/voice-call``
- [Zalo Personal] (/plugins/zalouser) — ``@openclaw/zalouser``

操作示例：

- `openclaw plugins list`
- `openclaw plugins install @openclaw/voice-call`
- `"extensions": ["/src/safety.ts", "/src/tools.ts"]`
- `"name": "@openclaw/nextcloud-talk",`
- `"extensions": ["/index.ts"],`
- `"docsPath": "/channels/nextcloud-talk",`

18. 表情回应工具

关键点：

- 添加表情回应时，`emoji` 为必填项。
- `emoji=""` 在支持的情况下移除机器人的表情回应。
- `remove: true` 在支持的情况下移除指定的表情（需要提供 `emoji`）。
- ****Discord/Slack****：空 `emoji` 移除机器人在该消息上的所有表情回应；`remove: true` 仅移除指定的表情。
- ****Google Chat****：空 `emoji` 移除应用在该消息上的表情回应；`remove: true` 仅移除指定的表情。
- ****Telegram****：空 `emoji` 移除机器人的表情回应；`remove: true` 同样移除表情回应，但工具验证仍要求 `emoji` 为非空值。
- ****WhatsApp****：空 `emoji` 移除机器人的表情回应；`remove: true` 映射为空 emoji（仍需提供 `emoji`）。
- ****Signal****：当启用 `channels.signal.reactionNotifications` 时，收到的表情回应通知会触发系统事件。

19. Skills 配置

本节覆盖：

- 字段
- 注意事项
- 沙箱隔离的 Skills + 环境变量

关键点：

- `allowBundled`：可选的仅用于****内置**** Skills 的白名单。设置后，只有列表中的内置 Skills 才有资格（托管/工作区 Skills 不受影响）。
- `load.extraDirs`：要扫描的附加 Skills 目录（最低优先级）。
- `load.watch`：监视 Skills 文件夹并刷新 Skills 快照（默认：true）。
- `load.watchDebounceMs`：Skills 监视器事件的防抖时间（毫秒）（默认：250）。
- `install.preferBrew`：在可用时优先使用 brew 安装器（默认：true）。
- `install.nodeManager`：node 安装器偏好（`npm` | `pnpm` | `yarn` | `bun`，默认：npm）。这仅影响 ****Skills 安装****；Gateway 网关运行时仍为 Node（不推荐 Bun 用于 WhatsApp/Telegram）。
- `entries.<skillKey>`：单 Skills 覆盖。
- `enabled`：设置为 `false` 以禁用某个 Skills，即使它是内置/已安装的。

操作示例：

- extraDirs: ["~/Projects/agent-scripts/skills", "~/Projects/oss/some-skill-pack/skills"],
- nodeManager: "npm", // npm | pnpm | yarn | bun (Gateway 网关运行时仍为 Node ; 不推荐 bun)

20. Skills (OpenClaw)

本节覆盖：

- 位置和优先级
- 单智能体 vs 共享 Skills
- 插件 + Skills

关键点：

- **内置 Skills**：随安装包一起发布 (npm 包或 OpenClaw.app)
- **托管/本地 Skills**：`~/openclaw/skills`
- **工作区 Skills**：`<workspace>/skills`
- **单智能体 Skills** 位于 `- **共享 Skills** 位于 `~/openclaw/skills` (托管/本地)，对同一机器上的**所有智能体**可见。
- 如果你想要多个智能体使用一个通用的 Skills 包，也可以通过 `skills.load.extraDirs` (最低优先级) 添加**共享文件夹**。
- 将 Skills 安装到你的工作区：
- `clawhub install <skill-slug>`

操作示例：

- endpoint: "https://example.invalid",

21. 斜杠命令

本节覆盖：

- 配置
- 命令列表
- 使用量显示 (什么显示在哪里)

关键点：

- **命令**：独立的 `/...` 消息。
- **指令**：`/think`、`/verbose`、`/reasoning`、`/elevated`、`/exec`、`/model`、`/queue`。
- 指令在模型看到消息之前被剥离。
- 在普通聊天消息中 (不是仅指令消息)，它们被视为"内联提示"，**不会**持久化会话设置。

- 在仅指令消息中（消息只包含指令），它们会持久化到会话并回复确认。
- 指令仅对**授权发送者**生效（渠道白名单/配对加上`commands.useAccessGroups`）。
- `commands.text`（默认`true`）启用解析聊天消息中的`/...`。
- 在没有原生命令的平台上（WhatsApp/WebChat/Signal/iMessage/Google Chat/MS Teams），即使你将此设置为`false`，文本命令仍然有效。

操作示例：

- `/model`
- `/model list`
- `/model 3`
- `/model openai/gpt-5.2`
- `/model opus@anthropic:default`
- `/model status`

22. 子智能体

本节覆盖：

- 斜杠命令
- 启动行为
- 工具

关键点：

- `/subagents list``
- `/subagents kill <id| #|all>``
- `/subagents log <id| #> [limit] [tools]``
- `/subagents info <id| #>``
- `/subagents send <id| #> <message>``
- `/subagents steer <id| #> <message>``
- `/subagents spawn <agentId> <task> [--model <model>] [--thinking <level>]``
- 该命令非阻塞，先返回`runId`。

操作示例：

- `// deny 优先`
- `// 如果设置了 allow，则变为仅允许模式（deny 仍然优先）`
- `// allow: ["read", "exec", "process"]`

23. 思考级别（/think 指令）

本节覆盖：

- 功能说明
- 设置会话默认值
- 按智能体应用

关键点：

- 在任何入站消息正文中使用内联指令：``/t <level>``、``/think:<level>`` 或 ``/thinking <level>``。
- 级别（别名）：``off | minimal | low | medium | high | xhigh``（仅 GPT-5.2 + Codex 模型）
- `minimal` → "think"
- `low` → "think hard"
- `medium` → "think harder"
- `high` → "ultrathink"（最大预算）
- `xhigh` → "ultrathink+"（仅 GPT-5.2 + Codex 模型）
- ``highest``、``max`` 映射为 ``high``。

24. Web 工具

本节覆盖：

- 工作原理
- 选择搜索提供商
- 获取 Brave API 密钥

关键点：

- ``web_search`` — 通过 Brave Search API（默认）或 Perplexity Sonar（直连或通过 OpenRouter）搜索网络。
- ``web_fetch`` — HTTP 获取 + 可读性提取（HTML → markdown/文本）。
- ``web_search`` 调用你配置的提供商并返回结果。
- **Brave**（默认）：返回结构化结果（标题、URL、摘要）。
- **Perplexity**：返回带有实时网络搜索引用的 AI 综合答案。
- 结果按查询缓存 15 分钟（可配置）。
- ``web_fetch`` 执行普通 HTTP GET 并提取可读内容（HTML → markdown/文本）。它**不**执行 JavaScript。
- ``web_fetch`` 默认启用（除非显式禁用）。

操作示例：

- `provider: "brave", // 或 "perplexity"`
- `baseUrl: "https://api.perplexity.ai",`

- model: "perplexity/sonar-pro",
- // API 密钥（如果设置了 OPENROUTER_API_KEY 或 PERPLEXITY_API_KEY 则可选）
- // 基础 URL（如果省略则根据密钥感知默认值）
- baseUrl: "https://openrouter.ai/api/v1",

第七章 命令行实战

1. acp

本节覆盖：

- 用法
- ACP 客户端（调试）
- 如何使用

关键点：

- 确保 Gateway 网关正在运行（本地或远程）。
- 配置 Gateway 网关目标（配置或标志）。
- 将你的 IDE 配置为通过 stdio 运行 ``openclaw acp``。
- ``--session <key>``：使用特定的 Gateway 网关会话键。
- ``--session-label <label>``：通过标签解析现有会话。
- ``--reset-session``：为该键生成新的会话 ID（相同键，新对话记录）。
- ``--url <url>``：Gateway 网关 WebSocket URL（配置后默认为 `gateway.remote.url`）。
- ``--token <token>``：Gateway 网关认证令牌。

操作示例：

- `openclaw acp`
- `openclaw acp --url wss://gateway-host:18789 --token <token>`
- `openclaw acp --session agent:main:main`
- `openclaw acp --session-label "support inbox"`
- `openclaw acp --session agent:main:main --reset-session`
- `openclaw acp client`

2. `openclaw agent`

本节覆盖：

- 示例

关键点：

- 智能体发送工具：`[Agent send] (/tools/agent-send)`
- 通过 Gateway 网关运行智能体回合（使用 `--local` 进行嵌入式运行）。使用 `--agent <id>` 直接指定已配置的智能体。

操作示例：

- `openclaw agent --to +15555550123 --message "status update" --deliver`

- `openclaw agent --agent ops --message "Summarize logs"`
- `openclaw agent --session-id 1234 --message "Summarize inbox" --thinking medium`
- `openclaw agent --agent ops --message "Generate report" --deliver --reply-channel slack --reply-to "#reports"`

3. `openclaw agents`

本节覆盖：

- 示例
- 身份文件
- 设置身份

关键点：

- 多智能体路由：[多智能体路由](/concepts/multi-agent)
- 智能体工作区：[智能体工作区](/concepts/agent-workspace)
- 示例路径：`~/openclaw/workspace/IDENTITY.md`
- ``set-identity --from-identity`` 从工作区根目录读取（或从显式指定的 ``--identity-file`` 读取）
- ``name``
- ``theme``
- ``emoji``
- ``avatar``（工作区相对路径、http(s) URL 或 data URI）

操作示例：

- `openclaw agents list`
- `openclaw agents add work --workspace ~/openclaw/workspace-work`
- `openclaw agents set-identity --workspace ~/openclaw/workspace --from-identity`
- `openclaw agents set-identity --agent main --avatar avatars/openclaw.png`
- `openclaw agents delete work`
- `openclaw agents set-identity --workspace ~/openclaw/workspace --from-identity`

4. `openclaw approvals`

本节覆盖：

- 常用命令
- 从文件替换审批
- 允许列表辅助命令

关键点：

- 执行审批：[执行审批](/tools/exec-approvals)
- 节点：[节点](/nodes)
- `--node`` 使用与 `openclaw nodes`` 相同的解析器（id、name、ip 或 id 前缀）。
- `--agent`` 默认为 `"*"`，表示适用于所有智能体。
- 节点主机必须公开 `system.execApprovals.get/set``（macOS 应用或无头节点主机）。
- 审批文件按主机存储在 `~/openclaw/exec-approvals.json``。
- 管理本地主机、Gateway 网关主机或节点主机的执行审批。
- 默认情况下，命令针对磁盘上的本地审批文件。使用 `--gateway`` 可针对 Gateway 网关，使用 `--node`` 可针对特定节点。

操作示例：

- `openclaw approvals get``
- `openclaw approvals get --node <id|name|ip>``
- `openclaw approvals get --gateway``
- `openclaw approvals set --file ./exec-approvals.json``
- `openclaw approvals set --node <id|name|ip> --file ./exec-approvals.json``
- `openclaw approvals set --gateway --file ./exec-approvals.json``

5. `openclaw browser``

本节覆盖：

- 通用标志
- 快速开始（本地）
- 配置文件

关键点：

- 浏览器工具 + API：[浏览器工具](/tools/browser)
- Chrome 扩展中继：[Chrome 扩展](/tools/chrome-extension)
- `--url <gatewayWsUrl>``：Gateway 网关 WebSocket URL（默认从配置获取）。
- `--token <token>``：Gateway 网关令牌（如果需要）。
- `--timeout <ms>``：请求超时（毫秒）。
- `--browser-profile <name>``：选择浏览器配置文件（默认从配置获取）。
- `--json``：机器可读输出（在支持的地方）。
- `openclaw``：启动/附加到专用的 OpenClaw 管理的 Chrome 实例（隔离的用户数据目录）。

操作示例：

- `openclaw browser --browser-profile openclaw start`
- `openclaw browser --browser-profile openclaw open https://example.com`
- `openclaw browser --browser-profile openclaw snapshot`
- `openclaw browser profiles`
- `openclaw browser create-profile --name work --color "#FF5A36"`
- `openclaw browser delete-profile --name work`

6. `openclaw channels`

本节覆盖：

- 常用命令
- 添加/删除账户
- 登录/登出（交互式）

关键点：

- 渠道指南：[\[渠道\]\(/channels/index\)](#)
- Gateway 网关配置：[\[配置\]\(/gateway/configuration\)](#)
- 运行 ``openclaw status --deep`` 进行全面探测。
- 使用 ``openclaw doctor`` 获取引导式修复。
- ``openclaw channels list`` 输出 ``Claude: HTTP 403 ... user:profile`` → 用量快照需要 ``user:profile`` 权限范围。使用 ``--no-usage``，或提供 `claude.ai` 会话密钥（``CLAUDE_WEB_SESSION_KEY`` / ``CLAUDE_WEB_COOKIE``），或通过 Claude Code CLI 重新授权。
- ``--channel`` 是可选的；省略它可列出所有渠道（包括扩展）。
- ``--target`` 接受 ``channel:<id>`` 或原始数字频道 id，仅适用于 Discord。
- 探测是特定于提供商的：Discord intents + 可选的频道权限；Slack bot + user scopes；Telegram bot 标志 + webhook；Signal daemon 版本；MS Teams app token + Graph roles/scopes（在已知处标注）。没有探测功能的渠道报告 ``Probe: unavailable``。

操作示例：

- `openclaw channels list`
- `openclaw channels status`
- `openclaw channels capabilities`
- `openclaw channels capabilities --channel discord --target channel:123`
- `openclaw channels resolve --channel slack "#general" "@jane"`
- `openclaw channels logs --channel all`

7. `openclaw config`

本节覆盖：

- 示例
- 路径
- 值

关键点：

- 配置辅助命令：通过路径获取/设置/取消设置值。不带子命令运行将打开
- 配置向导（与 `openclaw configure` 相同）。
- 路径使用点号或括号表示法：
- 使用智能体列表索引来定位特定智能体：
- 值会尽可能解析为 JSON5；否则将被视为字符串。
- 使用 `--json` 强制要求 JSON5 解析。
- 编辑后请重启 Gateway 网关。

操作示例：

- `openclaw config set agents.defaults.heartbeat.every "2h"`
- `openclaw config set agents.list[0].tools.exec.node "node-id-or-name"`
- `openclaw config unset tools.web.search.apiKey`
- `openclaw config get agents.defaults.workspace`
- `openclaw config get agents.list[0].id`
- `openclaw config get agents.list`

8. `openclaw configure`

本节覆盖：

- 示例

关键点：

- Gateway 网关配置参考：[\[配置\]\(/gateway/configuration\)](#)
- Config CLI：[\[Config\]\(/cli/config\)](#)
- 选择 Gateway 网关运行位置始终会更新

`gateway.mode`。如果这是你唯一需要的，可以不选择其他部分直接选择"继续"。

• 面向渠道的服务（Slack/Discord/Matrix/Microsoft Teams）在设置期间会提示输入频道/房间允许列表。你可以输入名称或 ID；向导会尽可能将名称解析为 ID。

- 用于设置凭证、设备和智能体默认值的交互式提示。

- 注意：模型部分现在包含一个用于 `agents.defaults.models` 允许列表的多选项（显示在 `/model` 和模型选择器中的内容）。
- 提示：不带子命令的 `openclaw config` 会打开相同的向导。使用 `openclaw config get|set|unset` 进行非交互式编辑。

操作示例：

- `openclaw configure`
- `openclaw configure --section models --section channels`

9. `openclaw cron`

本节覆盖：

- 常见编辑

关键点：

- Cron 作业：[Cron 作业](/automation/cron-jobs)
- 管理 Gateway 网关调度器的 cron 作业。
- 提示：运行 `openclaw cron --help` 查看完整的命令集。
- 说明：隔离式 cron add 任务默认使用 `--announce` 投递摘要。使用 `--no-deliver` 仅内部运行。
- `--deliver` 仍作为 `--announce` 的弃用别名保留。
- 说明：一次性 (`--at`) 任务成功后默认删除。使用 `--keep-after-run` 保留。
- 更新投递设置而不更改消息：
- 为隔离的作业禁用投递：

操作示例：

- `openclaw cron edit <job-id> --announce --channel telegram --to "123456789"`
- `openclaw cron edit <job-id> --no-deliver`

10. `openclaw dashboard`

关键点：

- 使用当前认证信息打开控制界面。

操作示例：

- `openclaw dashboard`
- `openclaw dashboard --no-open`

11. `openclaw devices`

本节覆盖：

- 命令
- ``openclaw devices list``
- ``openclaw devices approve <requestId>``

关键点：

- ``--url <url>``：Gateway 网关 WebSocket URL（配置后默认使用 ``gateway.remote.url``）。
- ``--token <token>``：Gateway 网关 token（如需要）。
- ``--password <password>``：Gateway 网关密码（密码认证）。
- ``--timeout <ms>``：RPC 超时。
- ``--json``：JSON 输出（推荐用于脚本）。
- Token 轮换会返回新 token（敏感信息）。请像对待密钥一样对待它。
- 这些命令需要 ``operator.pairing``（或 ``operator.admin``）scope。
- 管理设备配对请求和设备范围的 token。

操作示例：

- `openclaw devices list`
- `openclaw devices list --json`
- `openclaw devices approve <requestId>`
- `openclaw devices reject <requestId>`
- `openclaw devices rotate --device <deviceId> --role operator --scope operator.read --scope operator.write`
- `openclaw devices revoke --device <deviceId> --role node`

12. ``openclaw directory``

本节覆盖：

- 通用参数
- 说明
- 将结果用于 ``message send``

关键点：

- ``--channel <name>``：渠道

ID/别名（配置了多个渠道时为必填；仅配置一个渠道时自动选择）

- ``--account <id>``：账号 ID（默认：渠道默认账号）
- ``--json``：输出 JSON 格式
- ``directory`` 用于帮助你查找可粘贴到其他命令中的 ID（特别是 ``openclaw message send --target ...``）。

- 对于许多渠道，结果来源于配置（允许列表/已配置的群组），而非实时的提供商目录。
- 默认输出为以制表符分隔的`id`（有时包含`name`）；脚本中请使用`--json`。
- WhatsApp：`+15551234567`（私聊），`1234567890-1234567890@g.us`（群组）
- Telegram：`@username` 或数字聊天 ID；群组为数字 ID

操作示例：

- `openclaw directory peers list --channel slack --query "U0"`
- `openclaw message send --channel slack --target user:U012ABCDEF --message "hello"`
- `openclaw directory self --channel zalouser`
- `openclaw directory peers list --channel zalouser`
- `openclaw directory peers list --channel zalouser --query "name"`
- `openclaw directory peers list --channel zalouser --limit 50`

13. `openclaw dns`

本节覆盖：

- 设置

关键点：

- Gateway 网关设备发现：[设备发现](/gateway/discovery)
- 广域设备发现配置：[配置](/gateway/configuration)
- 用于广域设备发现（Tailscale + CoreDNS）的 DNS 辅助工具。目前专注于 macOS + Homebrew CoreDNS。

操作示例：

- `openclaw dns setup`
- `openclaw dns setup --apply`

14. `openclaw docs`

关键点：

- 搜索实时文档索引。

操作示例：

- `openclaw docs browser extension`
- `openclaw docs sandbox allowHostControl`

15. `openclaw doctor`

本节覆盖：

- 示例
- macOS : ``launchctl`` 环境变量覆盖

关键点：

- 故障排除：`[故障排除](/gateway/troubleshooting)`
- 安全审计：`[安全](/gateway/security)`
- 交互式提示（如钥匙串/OAuth 修复）仅在 stdin 是 TTY 且**未**设置 ``--non-interactive`` 时运行。无头运行（cron、Telegram、无终端）将跳过提示。
- ``--fix``（``--repair`` 的别名）会将备份写入 ``~/openclaw/openclaw.json.bak``，并删除未知的配置键，同时列出每个删除项。
- Gateway 网关和渠道的健康检查 + 快速修复。
- 如果你之前运行过 `launchctl setenv OPENCLAW_GATEWAY_TOKEN ...`（或 `...PASSWORD`），该值会覆盖你的配置文件，并可能导致持续的"未授权"错误。

操作示例：

- `openclaw doctor`
- `openclaw doctor --repair`
- `openclaw doctor --deep`

16. Gateway 网关 CLI

本节覆盖：

- 运行 Gateway 网关
- 选项
- 查询运行中的 Gateway 网关

关键点：

- `[/gateway/bonjour](/gateway/bonjour)`
- `[/gateway/discovery](/gateway/discovery)`
- `[/gateway/configuration](/gateway/configuration)`
- 默认情况下，除非在 ``~/openclaw/openclaw.json`` 中设置了 ``gateway.mode=local``，否则 Gateway 网关将拒绝启动。使用 ``--allow-unconfigured`` 进行临时/开发运行。
- 在没有认证的情况下绑定到 loopback 之外的地址会被阻止（安全护栏）。
- ``SIGUSR1`` 在授权时触发进程内重启（启用 ``commands.restart`` 或使用 gateway 工具/config apply/update）。
- ``SIGINT``/``SIGTERM`` 处理程序会停止 Gateway 网关进程，但不会恢复任何自定义终端状态。如果你用 TUI 或 raw-mode 输入包装

CLI，请在退出前恢复终端。

- `--port <port>`：WebSocket 端口（默认来自配置/环境变量；通常为 `18789`）。

操作示例：

- `openclaw gateway`
- `openclaw gateway run`
- `openclaw gateway health --url ws://127.0.0.1:18789`
- `openclaw gateway status`
- `openclaw gateway status --json`
- `openclaw gateway probe`

17. `openclaw health`

关键点：

- `--verbose` 运行实时探测，并在配置了多个账户时打印每个账户的耗时。
- 当配置了多个智能体时，输出包括每个智能体的会话存储。
- 从运行中的 Gateway 网关获取健康状态。

操作示例：

- `openclaw health`
- `openclaw health --json`
- `openclaw health --verbose`

18. `openclaw hooks`

本节覆盖：

- 列出所有钩子
- 获取钩子信息
- 检查钩子资格

关键点：

- 钩子：`[钩子]/automation/hooks`
- 插件钩子：`[插件]/tools/plugin#plugin-hooks`
- `--eligible`：仅显示符合条件的钩子（满足要求）
- `--json`：以 JSON 格式输出
- `-v, --verbose`：显示详细信息，包括缺失的要求
- `<name>`：钩子名称（例如 `session-memory`）

- 检查钩子是否存在且符合条件
- 在配置中更新 `hooks.internal.entries.<name>.enabled = true``

操作示例：

- `openclaw hooks list`
- Hooks (3/3 ready)
- `session-memory` - Save session context to memory when `/new` command is issued
- `openclaw hooks list --verbose`
- `openclaw hooks list --json`
- `openclaw hooks info <name>`

19. CLI 参考

本节覆盖：

- 命令页面
- 全局标志
- 输出样式

关键点：

- `[`setup`](/cli/setup)`
- `[`onboard`](/cli/onboard)`
- `[`configure`](/cli/configure)`
- `[`config`](/cli/config)`
- `[`doctor`](/cli/doctor)`
- `[`dashboard`](/cli/dashboard)`
- `[`reset`](/cli/reset)`
- `[`uninstall`](/cli/uninstall)`

操作示例：

- `openclaw [--dev] [--profile <name>] <command>`
- `openclaw channels add --channel telegram --account alerts --name "Alerts Bot" --token $TELEGRAM_BOT_TOKEN`
- `openclaw channels add --channel discord --account work --name "Work Bot" --token $DISCORD_BOT_TOKEN`
- `openclaw channels remove --channel discord --account work --delete`
- `openclaw channels status --probe`
- `openclaw status --deep`

20. `openclaw logs`

本节覆盖：

- 示例

关键点：

- 日志概述：[日志](/logging)
- 通过 RPC 跟踪 Gateway 网关文件日志（在远程模式下可用）。

操作示例：

- `openclaw logs`
- `openclaw logs --follow`
- `openclaw logs --json`
- `openclaw logs --limit 500`

21. `openclaw memory`

本节覆盖：

- 示例
- 选项

关键点：

- 记忆概念：[记忆](/concepts/memory)
- 插件：[插件](/tools/plugin)
- `--agent <id>`：限定到单个智能体（默认：所有已配置的智能体）。
- `--verbose`：在探测和索引期间输出详细日志。
- `memory status --deep` 探测向量存储和嵌入模型的可用性。
- `memory status --deep --index` 在存储有未同步变更时运行重新索引。
- `memory index --verbose`

打印每个阶段的详细信息（提供商、模型、数据源、批处理活动）。

- `memory status` 包含通过 `memorySearch.extraPaths` 配置的所有额外路径。

操作示例：

- `openclaw memory status`
- `openclaw memory status --deep`
- `openclaw memory status --deep --index`
- `openclaw memory status --deep --index --verbose`
- `openclaw memory index`

- `openclaw memory index --verbose`

22. ``openclaw message``

本节覆盖：

- 用法
- 通用标志
- 操作

关键点：

- 如果配置了多个渠道，则必须指定 ``--channel``。
- 如果只配置了一个渠道，则该渠道为默认值。
- 可选值：``whatsapp|telegram|discord|googlechat|slack|mattermost|signal|imessage|msteams``（Mattermost 需要插件）
- WhatsApp：E.164 或群组 JID
- Telegram：聊天 ID 或 ``@username``
- Discord：``channel:<id>`` 或 ``user:<id>``（或 ``<@id>`` 提及；纯数字 ID 被视为频道）
- Google Chat：``spaces/<spaceId>`` 或 ``users/<userId>``
- Slack：``channel:<id>`` 或 ``user:<id>``（接受纯频道 ID）

操作示例：

- `openclaw message <subcommand> [flags]`
- `openclaw message send --channel discord \`
- `--target channel:123 --message "hi" --reply-to 456`
- `openclaw message poll --channel discord \`
- `--poll-option Pizza --poll-option Sushi \`
- `--poll-multi --poll-duration-hours 48`

23. ``openclaw models``

本节覆盖：

- 常用命令
- ``models status``
- 别名 + 回退

关键点：

- 提供商 + 模型：`[模型] (/providers/models)`
- 提供商认证设置：`[快速开始] (/start/getting-started)`

- ``models set <model-or-alias>`` 接受 ``provider/model`` 或别名。
- 模型引用通过在**第一个**``/`` 处拆分来解析。如果模型 ID 包含 ``/``（OpenRouter 风格），需包含提供商前缀（示例：``openrouter/moonshotai/kimi-k2``）。
- 如果省略提供商，OpenClaw 会将输入视为别名或**默认提供商**的模型（仅在模型 ID 不包含 ``/`` 时有效）。
- ``--json``
- ``--plain``
- ``--check``（退出码 1=已过期/缺失，2=即将过期）

操作示例：

- `openclaw models status`
- `openclaw models list`
- `openclaw models set <model-or-alias>`
- `openclaw models scan`
- `openclaw models aliases list`
- `openclaw models fallbacks list`

24. ``openclaw node``

本节覆盖：

- 为什么使用节点主机？
- 浏览器代理（零配置）
- 运行（前台）

关键点：

- 在远程 Linux/Windows 机器上运行命令（构建服务器、实验室机器、NAS）。
- 在 Gateway 网关上保持执行的**沙箱隔离**，但将批准的运行委托给其他主机。
- 为自动化或 CI 节点提供轻量级、无头的执行目标。
- ``--host <host>``：Gateway 网关 WebSocket 主机（默认：``127.0.0.1``）
- ``--port <port>``：Gateway 网关 WebSocket 端口（默认：``18789``）
- ``--tls``：为 Gateway 网关连接使用 TLS
- ``--tls-fingerprint <sha256>``：预期的 TLS 证书指纹（sha256）
- ``--node-id <id>``：覆盖节点 id（清除配对 token）

操作示例：

- `openclaw node run --host <gateway-host> --port 18789`
- `openclaw node install --host <gateway-host> --port 18789`

- openclaw node status
- openclaw node stop
- openclaw node restart
- openclaw node uninstall

25. `openclaw nodes`

本节覆盖：

- 常用命令
- 调用 / 运行
- Exec 风格默认值

关键点：

- 节点概述：[\[节点\] \(/nodes\)](#)
- 摄像头：[\[摄像头节点\] \(/nodes/camera\)](#)
- 图像：[\[图像节点\] \(/nodes/images\)](#)
- `--url`、`--token`、`--timeout`、`--json`
- `--params <json>`：JSON 对象字符串（默认 `{}`）。
- `--invoke-timeout <ms>`：节点调用超时（默认 `15000`）。
- `--idempotency-key <key>`：可选的幂等键。
- 读取 `tools.exec.\*`（以及 `agents.list[].tools.exec.\*` 覆盖）。

操作示例：

- openclaw nodes list
- openclaw nodes list --connected
- openclaw nodes list --last-connected 24h
- openclaw nodes pending
- openclaw nodes approve <requestId>
- openclaw nodes status

26. `openclaw onboard`

本节覆盖：

- 示例

关键点：

- 向导指南：[\[新手引导\] \(/start/onboarding\)](#)

- ``quickstart``：最少提示，自动生成 Gateway 网关令牌。
- ``manual``：完整的端口/绑定/认证提示（``advanced`` 的别名）。
- 最快开始聊天：``openclaw dashboard``（控制 UI，无需渠道设置）。
- 交互式新手引导向导（本地或远程 Gateway 网关设置）。

操作示例：

- `openclaw onboard`
- `openclaw onboard --flow quickstart`
- `openclaw onboard --flow manual`
- `openclaw onboard --mode remote --remote-url ws://gateway-host:18789`

27. ``openclaw pairing``

本节覆盖：

- 命令

关键点：

- 配对流程：`[配对](/channels/pairing)`
- 批准或检查私信配对请求（适用于支持配对的渠道）。

操作示例：

- `openclaw pairing list whatsapp`
- `openclaw pairing approve whatsapp <code> --notify`

28. ``openclaw plugins``

本节覆盖：

- 命令
- 安装
- 更新

关键点：

- 插件系统：`[插件](/tools/plugin)`
- 插件清单 + 模式：`[插件清单](/plugins/manifest)`
- 安全加固：`[安全](/gateway/security)`
- 管理 Gateway 网关插件/扩展（进程内加载）。
- 内置插件随 OpenClaw 一起发布，但默认禁用。使用 `plugins enable` 来激活它们。
- 所有插件必须提供 `openclaw.plugin.json` 文件，其中包含内联 JSON Schema（`configSchema`，即使为空）。缺少或无效的清单或模式会阻止插件加载并导致配置验证失败。

- 安全提示：将插件安装视为运行代码。优先使用固定版本。
- 支持的归档格式：.zip、.tgz、.tar.gz、.tar。

操作示例：

- openclaw plugins list
- openclaw plugins info <id>
- openclaw plugins enable <id>
- openclaw plugins disable <id>
- openclaw plugins doctor
- openclaw plugins update <id>

29. `openclaw reset`

关键点：

- 重置本地配置/状态（保留 CLI 安装）。

操作示例：

- openclaw reset
- openclaw reset --dry-run
- openclaw reset --scope config+creds+sessions --yes --non-interactive

30. 沙箱 CLI

本节覆盖：

- 概述
- 命令
- `openclaw sandbox explain`

关键点：

- 容器名称和状态（运行中/已停止）
- Docker 镜像及其是否与配置匹配
- 空闲时间（自上次使用以来的时间）
- 关联的会话/智能体
- `--all`：重新创建所有沙箱容器
- `--session <key>`：重新创建特定会话的容器
- `--agent <id>`：重新创建特定智能体的容器
- `--browser`：仅重新创建浏览器容器

操作示例：

- `openclaw sandbox explain`
- `openclaw sandbox explain --session agent:main:main`
- `openclaw sandbox explain --agent work`
- `openclaw sandbox explain --json`
- `openclaw sandbox list`
- `openclaw sandbox list --browser # List only browser containers`

31. `openclaw security`

本节覆盖：

- 审计

关键点：

- 安全指南：[\[安全\]\(/gateway/security\)](#)
- 安全工具（审计 + 可选修复）。
- 当多个私信发送者共享主会话时，审计会发出警告，并建议对共享收件箱使用 `session.dmScope="per-channel-peer"`（或多账户渠道使用 `per-account-channel-peer`）。
- 当使用小模型（ $\leq 300B$ ）且未启用沙箱隔离但启用了 `web/browser` 工具时，它也会发出警告。

操作示例：

- `openclaw security audit`
- `openclaw security audit --deep`
- `openclaw security audit --fix`

32. `openclaw sessions`

关键点：

- 列出已存储的对话会话。

操作示例：

- `openclaw sessions`
- `openclaw sessions --active 120`
- `openclaw sessions --json`

33. `openclaw setup`

本节覆盖：

- 示例

关键点：

- 快速开始：[快速开始](/start/getting-started)
- 向导：[新手引导](/start/onboarding)
- 初始化 ~/.openclaw/openclaw.json 和智能体工作区。
- 通过 setup 运行向导：

操作示例：

- openclaw setup
- openclaw setup --workspace ~/.openclaw/workspace
- openclaw setup --wizard

34. `openclaw skills`

本节覆盖：

- 命令

关键点：

- Skills 系统：[Skills](/tools/skills)
- Skills 配置：[Skills 配置](/tools/skills-config)
- ClawHub 安装：[ClawHub](/tools/clawhub)
- 检查 Skills（内置 + 工作区 + 托管覆盖）并查看哪些符合条件，哪些缺少要求。

操作示例：

- openclaw skills list
- openclaw skills list --eligible
- openclaw skills info <name>
- openclaw skills check

35. `openclaw status`

关键点：

- `--deep` 运行实时探测（WhatsApp Web + Telegram + Discord + Google Chat + Slack + Signal）。
- 当配置了多个智能体时，输出包含每个智能体的会话存储。
- 概览包含 Gateway 网关 + 节点主机服务安装/运行时状态（如果可用）。
- 概览包含更新渠道 + git SHA（用于源代码检出）。

- 更新信息显示在概览中；如果有可用更新，status 会打印提示运行 `openclaw update`（参见[更新](/install/updating)）。
- 渠道 + 会话的诊断。

操作示例：

- `openclaw status`
- `openclaw status --all`
- `openclaw status --deep`
- `openclaw status --usage`

36. `openclaw system`

本节覆盖：

- 常用命令
- `system event`
- `system heartbeat last|enable|disable`

关键点：

- `--text <text>`：必填的系统事件文本。
- `--mode <mode>`：`now` 或 `next-heartbeat`（默认）。
- `--json`：机器可读输出。
- `last`：显示最后一次心跳事件。
- `enable`：重新开启心跳（如果之前被禁用，使用此命令）。
- `disable`：暂停心跳。
- 需要一个运行中的 Gateway 网关，可通过你当前的配置访问（本地或远程）。
- 系统事件是临时的，不会在重启后持久化。

操作示例：

- `openclaw system event --text "Check for urgent follow-ups" --mode now`
- `openclaw system heartbeat enable`
- `openclaw system heartbeat last`
- `openclaw system presence`

37. `openclaw tui`

本节覆盖：

- 示例

关键点：

- TUI 指南：[TUI](/web/tui)
- 打开连接到 Gateway 网关的终端 UI。

操作示例：

- `openclaw tui`
- `openclaw tui --url ws://127.0.0.1:18789 --token <token>`
- `openclaw tui --session main --deliver`

38. ``openclaw uninstall``

关键点：

- 卸载 Gateway 网关服务 + 本地数据（CLI 保留）。

操作示例：

- `openclaw uninstall`
- `openclaw uninstall --all --yes`
- `openclaw uninstall --dry-run`

39. ``openclaw update``

本节覆盖：

- 用法
- 选项
- ``update status``

关键点：

- ``--no-restart``：成功更新后跳过重启 Gateway 网关服务。
- ``--tag <dist-tag|version>``：仅为本次更新覆盖 npm dist-tag 或版本。
- ``--json``：打印机器可读的 ``UpdateRunResult`` JSON。
- ``--timeout <seconds>``：每步超时时间（默认 1200 秒）。
- ``--json``：打印机器可读的状态 JSON。
- ``--timeout <seconds>``：检查超时时间（默认 3 秒）。
- ``dev`` → 确保存在 git 检出（默认：``~/openclaw``，可通过 ``OPENCLAW_GIT_DIR`` 覆盖），更新它，并从该检出安装全局 CLI。
- ``beta``：检出最新的 ``-beta`` 标签，然后构建 + doctor。

操作示例：

- `openclaw update`
- `openclaw update status`

- openclaw update wizard
- openclaw update --channel beta
- openclaw update --channel dev
- openclaw update --tag beta

40. `openclaw voicecall`

本节覆盖：

- 常用命令
- 暴露 Webhook (Tailscale)

关键点：

- 语音通话插件：[语音通话](/plugins/voice-call)
- voicecall 是一个由插件提供的命令。只有在安装并启用了语音通话插件时才会出现。
- 安全提示：仅将 webhook 端点暴露给你信任的网络。尽可能优先使用 Tailscale Serve 而非 Funnel。

操作示例：

- openclaw voicecall status --call-id <id>
- openclaw voicecall call --to "+15555550123" --message "Hello" --mode notify
- openclaw voicecall continue --call-id <id> --message "Any questions?"
- openclaw voicecall end --call-id <id>
- openclaw voicecall expose --mode serve
- openclaw voicecall expose --mode funnel

41. `openclaw webhooks`

本节覆盖：

- Gmail

关键点：

- Webhook：[Webhook](/automation/webhook)
- Gmail Pub/Sub：[Gmail Pub/Sub](/automation/gmail-pubsub)
- Webhook 辅助工具和集成（Gmail Pub/Sub、Webhook 辅助工具）。
- 详情请参阅 Gmail Pub/Sub 文档。

操作示例：

- openclaw webhooks gmail setup --account you@example.com
- openclaw webhooks gmail run

第八章 自动化与定时任务

1. 认证监控

本节覆盖：

- 推荐方式：CLI 检查（可移植）
- 可选脚本（运维 / 手机工作流程）

关键点：

- `0`：正常
- `1`：凭证过期或缺失
- `2`：即将过期（24 小时内）
- `scripts/claude-auth-status.sh` 现在使用 `openclaw models status --json` 作为数据来源（如果 CLI 不可用则回退到直接读取文件），因此请确保 `openclaw` 在定时器的 `PATH` 中。
- `scripts/auth-monitor.sh`：cron/systemd 定时器目标；发送告警（ntfy 或手机）。
- `scripts/systemd/openclaw-auth-monitor.{service,timer}`：systemd 用户定时器。
- `scripts/claude-auth-status.sh`：Claude Code + OpenClaw 认证检查器（完整/json/简洁模式）。
- `scripts/mobile-reauth.sh`：通过 SSH 引导的重新认证流程。

操作示例：

- `openclaw models status --check`

2. 定时任务（Gateway网关调度器）

本节覆盖：

- 简要概述
- 快速开始（可操作）
- 工具调用等价形式（Gateway网关定时任务工具）

关键点：

- 定时任务运行在 **Gateway网关内部**（而非模型内部）。
- 任务持久化存储在 `~/openclaw/cron/` 下，因此重启不会丢失计划。
- 两种执行方式：
 - **主会话**：入队一个系统事件，然后在下一次心跳时运行。
 - **隔离式**：在 `cron:<jobld>` 中运行专用智能体轮次，可投递摘要（默认 announce）或不投递。
- 唤醒是一等功能：任务可以请求"立即唤醒"或"下次心跳时"。

- ****选择调度计划****
- 一次性提醒 → `schedule.kind = "at"` (CLI : `--at`)

操作示例：

- `openclaw cron add \`
- `openclaw cron list`
- `openclaw cron run <job-id> --force`
- `openclaw cron runs --id <job-id>`
- `openclaw cron add \`
- `--tz "America/Los_Angeles" \`

3. 定时任务与心跳：何时使用哪种方式

本节覆盖：

- 快速决策指南
- 心跳：周期性感知
- 何时使用心跳

关键点：

- ****多个周期性检查****：与其设置 5 个独立的定时任务分别检查收件箱、日历、天气、通知和项目状态，不如用一次心跳批量处理所有内容。
- ****上下文感知决策****：智能体拥有完整的主会话上下文，因此可以智能判断哪些紧急、哪些可以等待。
- ****对话连续性****：心跳运行共享同一会话，因此智能体记得最近的对话，可以自然地进行后续跟进。
- ****低开销监控****：一次心跳替代多个小型轮询任务。
- ****批量处理多项检查****：一次智能体轮次可以同时审查收件箱、日历和通知。
- ****减少 API 调用****：一次心跳比 5 个隔离式定时任务更经济。
- ****上下文感知****：智能体了解你一直在做什么，可以据此排定优先级。
- ****智能抑制****：如果没有需要关注的事项，智能体回复 `HEARTBEAT\_OK`，不会投递任何消息。

操作示例：

- `every: "30m", // 间隔`
- `target: "last", // 告警投递目标`
- `activeHours: { start: "08:00", end: "22:00" }, // 可选`
- `openclaw cron add \`
- `--tz "America/New_York" \`

- `openclaw cron add \`

4. Gmail Pub/Sub -> OpenClaw

本节覆盖：

- 前置条件
- 向导（推荐）
- 一次性设置

关键点：

- 已安装并登录`gcloud`（[安装指南](https://docs.cloud.google.com/sdk/docs/install-sdk)）。
- 已安装`gog`（gogcli）并为 Gmail 账户授权（gogcli.sh）。
- 已启用 OpenClaw hooks（参见 [Webhooks](/automation/webhook)）。
- 已登录`tailscale`（tailscale.com）。支持的设置使用 Tailscale Funnel 作为公共 HTTPS 端点。
- 映射中的每个 hook 的`model`/`thinking` 仍会覆盖这些默认值。
- 如果设置了`agents.defaults.models`，Gmail 模型必须在允许列表中。
- Gmail hook 内容默认使用外部内容安全边界包装。
- 使用 Tailscale Funnel 作为公共推送端点。

操作示例：

- `path: "/hooks",`
- `model: "openai/gpt-5.2-mini",`
- `// to: "+15551234567"`
- `model: "openrouter/meta-llama/llama-3.3-70b-instruct:free",`
- `openclaw webhooks gmail setup \`
- `openclaw webhooks gmail run`

5. Hooks

本节覆盖：

- 入门指南
- 概述
- 入门

关键点：

- ****Hooks****（本页）：当智能体事件触发时在 Gateway 网关内运行，如 ``/new``、``/reset``、``/stop`` 或生命周期事件。
- ****Webhooks****：外部 HTTP webhooks，让其他系统触发 OpenClaw 中的工作。参见 [Webhook Hooks](/automation/webhook) 或使用 ``openclaw webhooks`` 获取 Gmail 助手命令。
- 重置会话时保存记忆快照
- 保留命令审计跟踪用于故障排除或合规
- 会话开始或结束时触发后续自动化
- 事件触发时向智能体工作区写入文件或调用外部 API
- 在发出 ``/new`` 时将会话上下文保存到记忆
- 记录所有命令以供审计

操作示例：

- `openclaw hooks list`
- `openclaw hooks enable session-memory`
- `openclaw hooks check`
- `openclaw hooks info session-memory`
- `my-hook/`
- `openclaw hooks install <path-or-spec>`

6. 投票

本节覆盖：

- 支持的渠道
- CLI
- Gateway 网关 RPC

关键点：

- WhatsApp（Web 渠道）
- Discord
- MS Teams（Adaptive Cards）
- ``--channel``：`whatsapp`（默认）、`discord` 或 `msteams`
- ``--poll-multi``：允许选择多个选项
- ``--poll-duration-hours``：仅限 Discord（省略时默认为 24）
- ``to``（字符串，必需）
- ``question``（字符串，必需）

操作示例：

- `openclaw message poll --target +15555550123 \`
- `--poll-question "Lunch today?" --poll-option "Yes" --poll-option "No" --poll-option "Maybe"`
- `openclaw message poll --target 123456789@g.us \`
- `--poll-question "Meeting time?" --poll-option "10am" --poll-option "2pm" --poll-option "4pm" --poll-multi`
- `openclaw message poll --channel discord --target channel:123456789 \`
- `--poll-question "Snack?" --poll-option "Pizza" --poll-option "Sushi"`

7. 自动化故障排查

关键点：

- 该页面是英文文档的中文占位版本，完整内容请先参考英文版：Automation Troubleshooting。

8. Webhooks

本节覆盖：

- 启用
- 认证
- 端点

关键点：

- 当 ``hooks.enabled=true`` 时，``hooks.token`` 为必填项。
- ``hooks.path`` 默认为 ``/hooks``。
- ``Authorization: Bearer <token>``（推荐）
- ``x-openclaw-token: <token>``
- ``?token=<token>``（已弃用；会记录警告日志，将在未来的主要版本中移除）
- ``text`` ****必填****（字符串）：事件描述（例如“收到新邮件”）。
- ``mode`` 可选（``now`` | ``next-heartbeat``）：是否立即触发心跳（默认 ``now``）或等待下一次定期检查。
- 为****主****会话加入一个系统事件队列

操作示例：

- `path: "/hooks",`
- `"model": "openai/gpt-5.2-mini",`
- `curl -X POST http://127.0.0.1:18789/hooks/wake \`

- -H 'Content-Type: application/json' \
- curl -X POST http://127.0.0.1:18789/hooks/agent \
- -H 'Content-Type: application/json' \

第九章 节点能力

1. 音频 / 语音消息 — 2026-01-17

本节覆盖：

- 已支持的功能
- 自动检测（默认）
- 配置示例

关键点：

- ****媒体理解（音频）****：如果音频理解已启用（或自动检测），OpenClaw 会：
 - 找到第一个音频附件（本地路径或 URL），如有需要则下载。
 - 在发送给每个模型条目之前执行 ``maxBytes`` 限制。
 - 如果失败或跳过（大小/超时），则尝试下一个条目。
 - 成功后，将 ``Body`` 替换为 ``[Audio]`` 块并设置 ``{{Transcript}}``。
- ****命令解析****：转录成功时，``CommandBody``/``RawBody`` 会设置为转录文本，因此斜杠命令仍然有效。
- ****详细日志****：在 ``--verbose`` 模式下，我们会在转录运行和替换正文时记录日志。
- ****本地 CLI****（如已安装）

2. 相机捕获（智能体）

本节覆盖：

- iOS 节点
- 用户设置（默认开启）
- 命令（通过 Gateway 网关 ``node.invoke``）

关键点：

- ****iOS 节点****（通过 Gateway 网关配对）：通过 ``node.invoke`` 捕获****照片****（``jpg``）或****短视频片段****（``mp4``，可选音频）。
- ****Android 节点****（通过 Gateway 网关配对）：通过 ``node.invoke`` 捕获****照片****（``jpg``）或****短视频片段****（``mp4``，可选音频）。
- ****macOS 应用****（通过 Gateway 网关的节点）：通过 ``node.invoke`` 捕获****照片****（``jpg``）或****短视频片段****（``mp4``，可选音频）。
- iOS 设置标签页 → ****相机**** → ****允许相机****（``camera.enabled``）
- 默认：****开启****（缺少键时视为启用）。
- 关闭时：``camera.*`` 命令返回 ``CAMERA_DISABLED``。
- ``camera.list``

- `devices` : `{ id, name, position, deviceType }` 数组

操作示例：

- `openclaw nodes camera snap --node <id> # default: both front + back (2 MEDIA lines)`
- `openclaw nodes camera snap --node <id> --facing front`
- `openclaw nodes camera clip --node <id> --duration 3000`
- `openclaw nodes camera clip --node <id> --no-audio`
- `openclaw nodes camera list --node <id> # list camera ids`
- `openclaw nodes camera snap --node <id> # prints MEDIA:<path>`

3. 图像与媒体支持 — 2025-12-05

本节覆盖：

- 目标
- CLI 接口
- WhatsApp Web 渠道行为

关键点：

- 通过 `openclaw message send --media` 发送带可选标题的媒体。
- 允许来自网页收件箱的自动回复在文本旁边包含媒体。
- 保持每种类型的限制合理且可预测。
- `openclaw message send --media <path-or-url> [--message <caption>]`
- `--media` 可选；标题可以为空以进行纯媒体发送。`
- `--dry-run` 打印解析后的负载；`--json` 输出 `{ channel, to, messageId, mediaUrl, caption }`。
- 输入：本地文件路径**或** HTTP(S) URL。
- 流程：加载到 Buffer，检测媒体类型，并构建正确的负载：

4. 节点

本节覆盖：

- 配对 + 状态
- 远程节点主机 (system.run)
- 什么在哪里运行

关键点：

- 节点是**外围设备**，不是 Gateway 网关。它们不运行 Gateway 网关服务。
- Telegram/WhatsApp 等消息落在**Gateway 网关**上，而不是节点上。

- 当节点的设备配对角色包含 `node` 时，`nodes status` 将节点标记为**已配对**。
- `node.pair.\*` (CLI: `openclaw nodes pending/approve/reject`) 是一个单独的 Gateway 网关拥有的
- **Gateway 网关主机**：接收消息，运行模型，路由工具调用。
- **节点主机**：在节点机器上执行 `system.run` / `system.which`。
- **批准**：通过 `~/openclaw/exec-approvals.json` 在节点主机上执行。
- 令牌是 Gateway 网关配置中的 `gateway.auth.token` (Gateway 网关主机上的 `~/openclaw/openclaw.json`)。

操作示例：

- `openclaw devices list`
- `openclaw devices approve <requestId>`
- `openclaw devices reject <requestId>`
- `openclaw nodes status`
- `openclaw nodes describe --node <idOrNameOrIp>`
- `openclaw node run --host <gateway-host> --port 18789 --display-name "Build Node"`

5. 位置命令 (节点)

本节覆盖：

- 简要概述
- 为什么用选择器 (而不只是开关)
- 设置模型

关键点：

- `location.get` 是一个节点命令 (通过 `node.invoke`)。
- 设置使用选择器：关闭 / 使用时 / 始终。
- 单独的开关：精确位置。
- iOS/macOS：用户可以在系统提示/设置中选择**使用时**或**始终**。应用可以请求升级，但操作系统可能要求进入设置。
- Android：后台位置是单独的权限；在 Android 10+ 上通常需要进入设置流程。
- 精确位置是单独的授权 (iOS 14+ "精确"，Android "精细" vs "粗略")。
- `location.enabledMode`：`off | whileUsing | always`
- `location.preciseEnabled`：bool

6. 媒体理解 (入站) — 2026-01-17

本节覆盖：

- 目标
- 高层行为
- 配置概述

关键点：

- 可选：将入站媒体预先消化为短文本，以便更快路由 + 更好的命令解析。
- 保留原始媒体传递给模型（始终）。
- 支持**提供商 API** 和 **CLI 回退**。
- 收集入站附件（`MediaPaths`、`MediaUrls`、`MediaTypes`）。
- 对于每个启用的能力（图片/音频/视频），根据策略选择附件（默认：**第一个**）。
- 选择第一个符合条件的模型条目（大小 + 能力 + 认证）。
- 如果模型失败或媒体太大，**回退到下一个条目**。
- `Body` 变为 `[Image]`、`[Audio]` 或 `[Video]` 块。

操作示例：

- `/* 共享列表 */`
- `/* 可选覆盖 */`
- `/* 可选覆盖 */`
- `/* 可选覆盖 */`
- `type: "provider", // 省略时默认`
- `capabilities: ["image"], // 可选，用于多模态条目`

7. Talk 模式

本节覆盖：

- 行为（macOS）
- 回复中的语音指令
- 配置（`~/openclaw/openclaw.json`）

关键点：

- 将转录文本发送到模型（main 会话，chat.send）
- 通过 ElevenLabs 朗读（流式播放）
- Talk 模式启用时显示**常驻悬浮窗**。
- **监听 → 思考 → 朗读**阶段转换。
- **短暂停顿**（静音窗口）后，当前转录文本被发送。

- 回复被**写入 WebChat**（与打字相同）。
- **语音中断**（默认开启）：如果用户在助手朗读时开始说话，我们会停止播放并记录中断时间戳供下一个提示使用。
- 仅适用于第一个非空行。

8. 节点故障排查

关键点：

- 该页面是英文文档的中文占位版本，完整内容请先参考英文版：Node Troubleshooting。

9. 语音唤醒（全局唤醒词）

本节覆盖：

- 存储（Gateway 网关主机）
- 协议
- 方法

关键点：

- **没有**每节点的自定义唤醒词。
- **任何节点/应用 UI 都可以编辑**列表；更改由 Gateway 网关持久化并广播给所有人。
- 每个设备仍保留自己的**语音唤醒启用/禁用**开关（本地用户体验 + 权限不同）。
- `~/openclaw/settings/voicewake.json``
- `voicewake.get`` → `{ triggers: string[] }`
- `voicewake.set``，参数 `{ triggers: string[] }` → `{ triggers: string[] }`
- 触发词会被规范化（修剪空格、删除空值）。空列表回退到默认值。
- 为安全起见会强制执行限制（数量/长度上限）。

第十章 Web 控制台

1. 控制 UI（浏览器）

本节覆盖：

- 快速打开（本地）
- 设备配对（首次连接）
- 目前可以做什么

关键点：

- 默认：``http://<host>:18789/``
- 可选前缀：设置 ``gateway.controlUi.basePath``（例如 ``/openclaw``）
- `http://127.0.0.1:18789/`（或 `http://localhost:18789/`）
- ``connect.params.auth.token``
- ``connect.params.auth.password``
- 本地连接（``127.0.0.1``）会自动批准。
- 远程连接（LAN、Tailnet 等）需要显式批准。
- 每个浏览器配置文件生成唯一的设备 ID，因此切换浏览器或清除浏览器数据将需要重新配对。

操作示例：

- `openclaw devices list`
- `openclaw devices approve <requestId>`
- `openclaw gateway --tailscale serve`
- `openclaw gateway --bind tailnet --token "$(openssl rand -hex 32)"`
- `OPENCLAW_CONTROL_UI_BASE_PATH=/openclaw/ npm ui:build`
- `http://localhost:5173/?gatewayUrl=ws://<gateway-host>:18789`

2. 仪表板（控制 UI）

本节覆盖：

- 快速路径（推荐）
- Token 基础（本地 vs 远程）
- 如果你看到"unauthorized" / 1008

关键点：

- `http://127.0.0.1:18789/`（或 `http://localhost:18789/`）
- `[控制 UI](/web/control-ui)` 了解使用方法和 UI 功能。
- `[Tailscale](/gateway/tailscale)` 了解 Serve/Funnel 自动化。

- [Web 界面](/web) 了解绑定模式和安全注意事项。
- 新手引导后，CLI 现在会自动打开带有你的 token 的仪表板，并打印相同的带 token 链接。
- 随时重新打开：`openclaw dashboard`（复制链接，如果可能则打开浏览器，如果是无头环境则显示 SSH 提示）。
- token 保持本地（仅查询参数）；UI 在首次加载后移除它并保存到 localStorage。
- ****Localhost****：打开 `http://127.0.0.1:18789/`。如果你看到"unauthorized"，运行 `openclaw dashboard` 并使用带 token 的链接（`?token=...`）。

3. Web (Gateway 网关)

本节覆盖：

- Webhooks
- 配置（默认开启）
- Tailscale 访问

关键点：

- 默认：`http://<host>:18789/`
- 可选前缀：设置 `gateway.controlUi.basePath`（例如 `/openclaw`）
- `https://<magicdns>/`（或你配置的 `gateway.controlUi.basePath`）
- `http://<tailscale-ip>:18789/`（或你配置的 `gateway.controlUi.basePath`）
- Gateway 网关认证默认是必需的（令牌/密码或 Tailscale 身份头）。
- 非本地回环绑定仍然****需要****共享令牌/密码（`gateway.auth` 或环境变量）。
- 向导默认生成 Gateway 网关令牌（即使在本地回环上）。
- UI 发送 `connect.params.auth.token` 或 `connect.params.auth.password`。

操作示例：

- `controlUi: { enabled: true, basePath: "/openclaw" }, // basePath 可选`
- `openclaw gateway`
- `openclaw gateway`
- `auth: { mode: "password" }, // 或 OPENCLAW_GATEWAY_PASSWORD`

4. TUI (终端 UI)

本节覆盖：

- 快速开始
- 你看到的内容
- 心智模型：智能体 + 会话

关键点：

- 启动 Gateway 网关。
- 打开 TUI。
- 输入消息并按 Enter。
- 标题栏：连接 URL、当前智能体、当前会话。
- 聊天日志：用户消息、助手回复、系统通知、工具卡片。
- 状态行：连接/运行状态（连接中、运行中、流式传输中、空闲、错误）。
- 页脚：连接状态 + 智能体 + 会话 + 模型 + think/verbose/reasoning + token 计数 + 投递状态。
- 输入：带自动完成的文本编辑器。

操作示例：

- `openclaw gateway`
- `openclaw tui`
- `openclaw tui --url ws://<host>:<port> --token <gateway-token>`

5. WebChat (Gateway 网关 WebSocket UI)

本节覆盖：

- 它是什么
- 快速开始
- 工作原理（行为）

关键点：

- Gateway 网关的原生聊天 UI（无嵌入式浏览器，无本地静态服务器）。
- 使用与其他渠道相同的会话和路由规则。
- 确定性路由：回复始终返回到 WebChat。
- 启动 Gateway 网关。
- 打开 WebChat UI（macOS/iOS 应用）或控制 UI 聊天标签页。
- 确保已配置 Gateway 网关认证（默认需要，即使在 loopback 上）。
- UI 连接到 Gateway 网关 WebSocket 并使用 ``chat.history``、``chat.send`` 和 ``chat.inject``。
- ``chat.inject`` 直接将助手注释追加到转录并广播到 UI（无智能体运行）。

第十一章 网关与系统配置

1. 认证

本节覆盖：

- 推荐的 Anthropic 设置 (API 密钥)
- Anthropic : setup-token (订阅认证)
- 检查模型认证状态

关键点：

- 在 Anthropic 控制台创建 API 密钥。
- 将其放在 ****Gateway 网关主机**** (运行 `openclaw gateway` 的机器) 上。
- 如果 Gateway 网关在 systemd/launchd 下运行，最好将密钥放在 `~/.openclaw/.env` 中以便守护进程可以读取：
- Claude Max 或 Pro 订阅 (用于 `claude setup-token`)
- 已安装 Claude Code CLI (`claude` 命令可用)
- OpenClaw 支持模型提供商的 OAuth 和 API 密钥。对于 Anthropic 账户，我们推荐使用 API 密钥。对于 Claude 订阅访问，使用 claude setup-token 创建的长期令牌。
- 参阅 /concepts/oauth 了解完整的 OAuth 流程和存储布局。
- 如果你直接使用 Anthropic，请使用 API 密钥。

操作示例：

- openclaw models status
- cat >> ~/.openclaw/.env <<'EOF'
- openclaw models status
- openclaw doctor
- openclaw models auth setup-token --provider anthropic
- openclaw models auth paste-token --provider anthropic

2. 后台 Exec + Process 工具

本节覆盖：

- exec 工具
- 子进程桥接
- process 工具

关键点：

- `command` (必填)

- ``yieldMs``（默认 10000）：在此延迟后自动转为后台运行
- ``background``（布尔值）：立即转为后台运行
- ``timeout``（秒，默认 1800）：在此超时后终止进程
- ``elevated``（布尔值）：如果启用/允许提权模式，则在宿主机上运行
- 需要真实 TTY？设置 ``pty: true``。
- ``workdir``、``env``
- 前台运行直接返回输出。

3. Bonjour / mDNS 设备发现

本节覆盖：

- 通过 Tailscale 的广域 Bonjour（单播 DNSSD）
- Gateway 网关配置（推荐）
- 一次性 DNS 服务器设置（Gateway 网关主机）

关键点：

- 在 Gateway 网关主机上运行 DNS 服务器（可通过 Tailnet 访问）。
- 在专用区域下发布 ``_openclaw-gw._tcp`` 的 DNSSD 记录
- 配置 Tailscale **分割 DNS**，使你选择的域名通过该
- 仅在 Gateway 网关的 Tailscale 接口上监听 53 端口
- 从 ``~/openclaw/dns/<domain>.db`` 提供你选择的域名服务（示例：``openclaw.internal.``）
- 添加指向 Gateway 网关 Tailnet IP 的名称服务器（UDP/TCP 53）。
- 添加分割 DNS，使你的发现域名使用该名称服务器。
- 在 ``~/openclaw/openclaw.json`` 中设置 ``gateway.bind: "tailnet"``。

操作示例：

- `gateway: { bind: "tailnet" }, // 仅 tailnet（推荐）`
- `discovery: { wideArea: { enabled: true } }, // 启用广域 DNS-SD 发布`
- `openclaw dns setup --apply`

4. Bridge 协议（旧版节点传输）

本节覆盖：

- 为什么我们有两种协议
- 传输
- 握手 + 配对

关键点：

- ****安全边界****：bridge 暴露一个小的允许列表，而不是完整的 Gateway 网关 API 接口。
- ****配对 + 节点身份****：节点准入由 Gateway 网关管理，并与每节点令牌绑定。
- ****设备发现用户体验****：节点可以通过局域网上的 Bonjour 发现 Gateway 网关，或通过 tailnet 直接连接。
- ****Loopback WS****：完整的 WS 控制平面保持本地，除非通过 SSH 隧道。
- TCP，每行一个 JSON 对象（JSONL）。
- 可选 TLS（当 `bridge.tls.enabled` 为 true 时）。
- 旧版默认监听端口为 `18790`（当前构建不启动 TCP bridge）。
- 客户端发送带有节点元数据 + 令牌（如果已配对）的 `hello`。

5. CLI 后端（回退运行时）

本节覆盖：

- 新手友好快速开始
- 作为回退使用
- 配置概览

关键点：

- ****工具被禁用****（无工具调用）。
- ****文本输入 → 文本输出****（可靠）。
- ****支持会话****（因此后续轮次保持连贯）。
- 如果 CLI 接受图像路径，****图像可以传递****。
- 如果你使用 `agents.defaults.models`（允许列表），必须包含 `claude-cli/...`。
- 如果主要提供商失败（认证、限流、超时），OpenClaw 将接着尝试 CLI 后端。
- ****选择后端****基于提供商前缀（`claude-cli/...`）。
- ****构建系统提示****使用相同的 OpenClaw 提示 + 工作区上下文。

操作示例：

- `openclaw agent --message "hi" --model claude-cli/opus-4.5`
- `openclaw agent --message "hi" --model codex-cli/gpt-5.2-codex`
- `command: "/opt/homebrew/bin/claude",`
- `primary: "anthropic/claude-opus-4-5",`
- `fallbacks: ["claude-cli/opus-4.5"],`
- `"anthropic/claude-opus-4-5": { alias: "Opus" },`

6. 配置示例

本节覆盖：

- 快速开始
- 绝对最小配置
- 推荐的入门配置

关键点：

- 如果你设置 ``dmPolicy: "open"``，匹配的 ``allowFrom`` 列表必须包含 ``"*"``。
- 提供商 ID 各不相同（电话号码、用户 ID、频道 ID）。使用提供商文档确认格式。
- 稍后添加的可选部分：``web``、``browser``、``ui``、``discovery``、``canvasHost``、``talk``、``signal``、``imessage``。
- 参阅[提供商](/channels/whatsapp)和[故障排除](/gateway/troubleshooting)了解更深入的设置说明。
- 以下示例与当前配置模式一致。有关详尽的参考和每个字段的说明，请参阅配置。
- 保存到 `~/.openclaw/openclaw.json`，你就可以从该号码私信机器人了。
- JSON5 允许你使用注释和尾随逗号。普通 JSON 也可以使用。

操作示例：

- `agent: { workspace: "~/.openclaw/workspace" },`
- `workspace: "~/.openclaw/workspace",`
- `model: { primary: "anthropic/claude-sonnet-4-5" },`
- `// 环境 + shell`
- `// 认证配置文件元数据（密钥存储在 auth-profiles.json 中）`
- `// 身份`

7. 配置

本节覆盖：

- 严格配置验证
- Schema + UI 提示
- 应用 + 重启 (RPC)

关键点：

- 限制谁可以触发机器人 (``channels.whatsapp.allowFrom``、``channels.telegram.allowFrom`` 等)

- 控制群组白名单 + 提及行为（`channels.whatsapp.groups`、`channels.telegram.groups`、`channels.discord.guilds`、`agents.list[].groupChat`）
- 自定义消息前缀（`messages`）
- 设置智能体工作区（`agents.defaults.workspace` 或 `agents.list[].workspace`）
- 调整内置智能体默认值（`agents.defaults`）和会话行为（`session`）
- 设置每个智能体的身份标识（`agents.list[].identity`）
- Gateway 网关不会启动。
- 只允许诊断命令（例如：`openclaw doctor`、`openclaw logs`、`openclaw health`、`openclaw status`、`openclaw service`、`openclaw help`）。

操作示例：

- `openclaw gateway call config.get --params '{}'` # capture payload.hash
- `openclaw gateway call config.apply --params '{`
- `"raw": "{\\n agents: { defaults: { workspace: \"~/openclaw/workspace\" } }\\n}\\n",`
- `openclaw gateway call config.get --params '{}'` # capture payload.hash
- `openclaw gateway call config.patch --params '{`
- `agents: { defaults: { workspace: \"~/openclaw/workspace\" } },`

8. 设备发现 & 传输协议

本节覆盖：

- 术语
- 为什么我们同时保留"直连"和 SSH
- 发现输入（客户端如何了解 Gateway 网关位置）

关键点：

- **操作员远程控制**：macOS 菜单栏应用控制运行在其他地方的 Gateway 网关。
- **节点配对**：iOS/Android（以及未来的节点）发现 Gateway 网关并安全配对。
- **Gateway 网关**：一个长期运行的 Gateway 网关进程，拥有状态（会话、配对、节点注册表）并运行渠道。大多数设置每台主机使用一个；也可以进行隔离的多 Gateway 网关设置。
- **Gateway 网关 WS（控制平面）**：默认在 `127.0.0.1:18789` 上的 WebSocket 端点；可通过 `gateway.bind` 绑定到 LAN/tailnet。
- **直连 WS 传输**：面向 LAN/tailnet 的 Gateway 网关 WS 端点（无 SSH）。
- **SSH 传输（回退）**：通过 SSH 转发 `127.0.0.1:18789` 进行远程控制。
- **旧版 TCP 桥接（已弃用/移除）**：旧的节点传输（参见 [桥接协议](/gateway/bridge-protocol)）；不再用于发现广播。

- [Gateway 网关协议](/gateway/protocol)

9. Doctor

本节覆盖：

- 快速开始
- 无头/自动化
- 功能概述

关键点：

- git 安装的可选预检更新（仅交互模式）。
- UI 协议新鲜度检查（当协议 schema 较新时重建 Control UI）。
- 健康检查 + 重启提示。
- Skills 状态摘要（符合条件/缺失/被阻止）。
- 遗留值的配置规范化。
- OpenCode Zen 提供商覆盖警告（``models.providers.opencode``）。
- 遗留磁盘状态迁移（会话/智能体目录/WhatsApp 认证）。
- 状态完整性和权限检查（会话、记录、状态目录）。

操作示例：

- `openclaw doctor`
- `openclaw doctor --yes`
- `openclaw doctor --repair`
- `openclaw doctor --repair --force`
- `openclaw doctor --non-interactive`
- `openclaw doctor --deep`

10. Gateway 网关锁

本节覆盖：

- 原因
- 机制
- 错误表面

关键点：

- 确保同一主机上每个基础端口只运行一个 Gateway 网关实例；额外的 Gateway 网关必须使用隔离的配置文件和唯一的端口。
- 在崩溃/SIGKILL 后不留下过时的锁文件。

- 当控制端口已被占用时快速失败并给出清晰的错误。
- Gateway 网关在启动时立即使用独占 TCP 监听器绑定 WebSocket 监听器（默认 ``ws://127.0.0.1:18789``）。
- 如果绑定因 ``EADDRINUSE`` 失败，启动会抛出 ``GatewayLockError("another gateway instance is already listening on ws://127.0.0.1:<port>")``。
- 操作系统在任何进程退出时（包括崩溃和 SIGKILL）自动释放监听器——不需要单独的锁文件或清理步骤。
- 关闭时，Gateway 网关关闭 WebSocket 服务器和底层 HTTP 服务器以及时释放端口。
- 如果另一个进程持有端口，启动会抛出 ``GatewayLockError("another gateway instance is already listening on ws://127.0.0.1:<port>")``。

11. 健康检查 (CLI)

本节覆盖：

- 快速检查
- 深度诊断
- 当出现故障时

关键点：

- ``openclaw status`` — 本地摘要：Gateway 网关可达性/模式、更新提示、已链接渠道认证时长、会话 + 最近活动。
- ``openclaw status --all`` — 完整本地诊断（只读、彩色、可安全粘贴用于调试）。
- ``openclaw status --deep`` — 还会探测运行中的 Gateway 网关（支持时进行每渠道探测）。
- ``openclaw health --json`` — 向运行中的 Gateway 网关请求完整健康快照（仅 WS；不直接访问 Baileys 套接字）。
- 在 WhatsApp/WebChat 中单独发送 ``/status`` 消息可获取状态回复，而不调用智能体。
- 日志：跟踪 ``/tmp/openclaw/openclaw-*.log`` 并过滤 ``web-heartbeat``、``web-reconnect``、``web-auto-reply``、``web-inbound``。
- 磁盘上的凭证：``ls -l ~/.openclaw/credentials/whatsapp/<accountId>/creds.json``（mtime 应该是最接近的）。
- 会话存储：``ls -l ~/.openclaw/agents/<agentId>/sessions/sessions.json``（路径可在配置中覆盖）。计数和最近收件人通过 ``status`` 显示。

12. 心跳 (Gateway 网关)

本节覆盖：

- 快速开始（新手）
- 默认值

- 心跳提示的用途

关键点：

- 保持心跳启用（默认 `30m`，Anthropic OAuth/setup-token 为 `1h`）或设置你自己的频率。
- 在智能体工作区创建一个简单的 `HEARTBEAT.md` 检查清单（可选但推荐）。
- 决定心跳消息发送到哪里（默认 `target: "last"`）。
- 可选：启用心跳推理内容发送以提高透明度。
- 可选：将心跳限制在活动时段（本地时间）。
- 间隔：`30m`（当检测到的认证模式为 Anthropic OAuth/setup-token 时为 `1h`）。设置 `agents.defaults.heartbeat.every` 或单智能体 `agents.list[].heartbeat.every`；使用 `0m` 禁用。
- 提示内容（可通过 `agents.defaults.heartbeat.prompt` 配置）：
- 心跳提示**原样**作为用户消息发送。系统提示包含"Heartbeat"部分，运行在内部被标记。

操作示例：

- `// activeHours: { start: "08:00", end: "24:00" },`
- `// includeReasoning: true, // 可选：同时发送单独的 `Reasoning:` 消息`
- `every: "30m", // 默认：30m (0m 禁用)`
- `model: "anthropic/claude-opus-4-5",`
- `includeReasoning: false, // 默认：false (可用时发送单独的 Reasoning: 消息)`
- `target: "last", // last | none | <channel id> (核心或插件，例如 "bluebubbles")`

13. Gateway 网关服务运行手册

本节覆盖：

- 是什么
- 如何运行（本地）
- 远程访问

关键点：

- 拥有单一 Baileys/Telegram 连接和控制/事件平面的常驻进程。
- 替代旧版 `gateway` 命令。CLI 入口点：`openclaw gateway`。
- 运行直到停止；出现致命错误时以非零退出码退出，以便 supervisor 重启它。
- 配置热重载监视 `~/openclaw/openclaw.json`（或 `OPENCLAW\_CONFIG\_PATH`）。
- 默认模式：`gateway.reload.mode="hybrid"`（热应用安全更改，关键更改时重启）。
- 热重载在需要时通过 `**SIGUSR1**` 使用进程内重启。
- 使用 `gateway.reload.mode="off"` 禁用。

- 将 WebSocket 控制平面绑定到 `127.0.0.1:<port>` (默认 18789)。

操作示例：

- `openclaw gateway --port 18789`
- # 在 stdio 中获取完整的调试/追踪日志：
- `openclaw gateway --port 18789 --verbose`
- `openclaw gateway --force`
- `openclaw --dev setup`
- `openclaw --dev gateway --allow-unconfigured`

14. 本地模型

本节覆盖：

- 推荐：LM Studio + MiniMax M2.1 (Responses API，完整尺寸)
- 混合配置：托管为主，本地备用
- 本地优先，托管作为安全网

关键点：

- 安装 LM Studio：<https://lmstudio.ai>
- 在 LM Studio 中，下载**可用的最大 MiniMax M2.1 构建**（避免"小型"/重度量化变体），启动服务器，确认 `http://127.0.0.1:1234/v1/models` 列出了它。
- 保持模型加载；冷加载会增加启动延迟。
- 如果你的 LM Studio 构建不同，调整 `contextWindow` / `maxTokens`。
- 对于 WhatsApp，坚持使用 Responses API，这样只发送最终文本。
- 托管的 MiniMax/Kimi/GLM 变体也存在于 OpenRouter 上，带有区域固定端点（例如，美国托管）。在那里选择区域变体以将流量保持在你选择的管辖区内，同时仍使用 `models.mode: "merge"` 作为 Anthropic/OpenAI 备用。
- 纯本地仍然是最强的隐私路径；当你需要提供商功能但又想控制数据流时，托管区域路由是折中方案。
- Gateway 网关能访问代理吗？`curl http://127.0.0.1:1234/v1/models`。

操作示例：

- `model: { primary: "lmstudio/minimax-m2.1-gs32" },`
- `"anthropic/claude-opus-4-5": { alias: "Opus" },`
- `"lmstudio/minimax-m2.1-gs32": { alias: "Minimax" },`
- `baseUrl: "http://127.0.0.1:1234/v1",`

- primary: "anthropic/claude-sonnet-4-5",
- fallbacks: ["lmstudio/minimax-m2.1-gs32", "anthropic/claude-opus-4-5"],

15. 日志

本节覆盖：

- 基于文件的日志记录器
- 控制台捕获
- 工具摘要脱敏

关键点：

- **控制台输出**（你在终端 / Debug UI 中看到的内容）。
- **文件日志**（JSON 行）由 Gateway 网关日志记录器写入。
- 默认滚动日志文件位于 ``/tmp/openclaw/``
下（每天一个文件）：``openclaw-YYYY-MM-DD.log``
- 日期使用 Gateway 网关主机的本地时区。
- 日志文件路径和级别可以通过 ``~/openclaw/openclaw.json`` 配置：
- ``logging.file``
- ``logging.level``
- **文件日志**完全由 ``logging.level`` 控制。

操作示例：

- `openclaw logs --follow`
- # 优化的（仅错误/慢调用）
- `openclaw gateway`
- `openclaw gateway --verbose --ws-log compact`
- `openclaw gateway --verbose --ws-log full`

16. 多 Gateway 网关（同一主机）

本节覆盖：

- 隔离检查清单（必需）
- 推荐：配置文件（``--profile``）
- 救援机器人指南

关键点：

- ``OPENCLAW_CONFIG_PATH`` — 每个实例的配置文件
- ``OPENCLAW_STATE_DIR`` — 每个实例的会话、凭证、缓存

- ``agents.defaults.workspace`` — 每个实例的工作区根目录
- ``gateway.port``（或 ``--port``）— 每个实例唯一
- 派生端口（浏览器/画布）不得重叠
- 配置文件/配置
- 基础端口（加上派生端口）
- 浏览器控制服务端口 = 基础 + 2（仅 loopback）

操作示例：

- `openclaw --profile main setup`
- `openclaw --profile main gateway --port 18789`
- `openclaw --profile rescue setup`
- `openclaw --profile rescue gateway --port 19001`
- `openclaw --profile main gateway install`
- `openclaw --profile rescue gateway install`

17. network model

本节覆盖：

- 核心规则

关键点：

- 建议每台主机运行一个 Gateway 网关。它是唯一允许拥有 WhatsApp Web 会话的进程。对于救援机器人或严格隔离的场景，可以使用隔离的配置文件和端口运行多个 Gateway 网关。参见[多 Gateway 网关](/gateway/multiple-gateways)。
- 优先使用回环地址：Gateway 网关的 WS 默认为 ``ws://127.0.0.1:18789``。即使是回环连接，向导也会默认生成 gateway token。若需通过 tailnet 访问，请运行 ``openclaw gateway --bind tailnet --token ...``，因为非回环绑定必须使用 token。
- 节点根据需要通过局域网、tailnet 或 SSH 连接到 Gateway 网关的 WS。旧版 TCP 桥接已弃用。
- Canvas 主机是一个 HTTP 文件服务器，运行在 ``canvasHost.port``（默认 ``18793``）上，提供 ``/_openclaw_/canvas/`` 路径供节点 WebView 使用。参见 [Gateway 网关配置](/gateway/configuration)（``canvasHost``）。
- 远程使用通常通过 SSH 隧道或 Tailscale VPN。参见[远程访问](/gateway/remote)和[设备发现](/gateway/discovery)。
- 大多数操作通过 Gateway 网关（`openclaw gateway`）进行，它是一个长期运行的单一进程，负责管理渠道连接和 WebSocket 控制平面。

18. OpenAI Chat Completions (HTTP)

本节覆盖：

- 认证
- 选择智能体
- 启用端点

关键点：

- `POST /v1/chat/completions``
- 与 Gateway 网关相同的端口（WS + HTTP 多路复用）：`http://<gateway-host>:<port>/v1/chat/completions``
- `Authorization: Bearer <token>``
- 当 `gateway.auth.mode="token"` 时，使用 `gateway.auth.token``（或 `OPENCLAW_GATEWAY_TOKEN``）。
- 当 `gateway.auth.mode="password"` 时，使用 `gateway.auth.password``（或 `OPENCLAW_GATEWAY_PASSWORD``）。
- `model: "openclaw:<agentId>"``（例如：`"openclaw:main"`、`"openclaw:beta"`）
- `model: "agent:<agentId>"``（别名）
- `x-openclaw-agent-id: <agentId>``（默认：`main``）

操作示例：

- `curl -sS http://127.0.0.1:18789/v1/chat/completions \`
- `-H 'Content-Type: application/json' \`
- `curl -N http://127.0.0.1:18789/v1/chat/completions \`
- `-H 'Content-Type: application/json' \`

19. OpenResponses API (HTTP)

本节覆盖：

- 认证
- 选择智能体
- 启用端点

关键点：

- `POST /v1/responses``
- 与 Gateway 网关相同的端口（WS + HTTP 多路复用）：`http://<gateway-host>:<port>/v1/responses``
- `Authorization: Bearer <token>``

- 当 `gateway.auth.mode="token"` 时，使用 `gateway.auth.token`（或 `OPENCLAW\_GATEWAY\_TOKEN`）。
- 当 `gateway.auth.mode="password"` 时，使用 `gateway.auth.password`（或 `OPENCLAW\_GATEWAY\_PASSWORD`）。
- `model: "openclaw:<agentId>"`（示例：`"openclaw:main"`、`"openclaw:beta"`）
- `model: "agent:<agentId>"`（别名）
- `x-openclaw-agent-id: <agentId>`（默认：`main`）

操作示例：

- "source": { "type": "url", "url": "https://example.com/image.png" }
- "media_type": "text/plain",
- "text/plain",
- "text/markdown",
- "text/html",
- "text/csv",

20. Gateway 网关拥有的配对（选项 B）

本节覆盖：

- 概念
- 配对工作原理
- CLI 工作流程（支持无头模式）

关键点：

- ****待处理请求****：一个节点请求加入；需要审批。
- ****已配对节点****：已批准的节点，带有已颁发的认证令牌。
- ****传输层****：Gateway 网关 WS 端点转发请求但不决定成员资格。（旧版 TCP 桥接支持已弃用/移除。）
- 节点连接到 Gateway 网关 WS 并请求配对。
- Gateway 网关存储一个****待处理请求****并发出 `node.pair.requested`。
- 你审批或拒绝该请求（CLI 或 UI）。
- 审批后，Gateway 网关颁发一个****新令牌****（重新配对时令牌会轮换）。
- 节点使用该令牌重新连接，现在是"已配对"状态。

操作示例：

- openclaw nodes pending
- openclaw nodes approve <requestId>

- `openclaw nodes reject <requestId>`
- `openclaw nodes status`
- `openclaw nodes rename --node <id|name|ip> --name "Living Room iPad"`

21. Gateway 网关协议 (WebSocket)

本节覆盖：

- 传输
- 握手 (connect)
- 节点示例

关键点：

- WebSocket，带有 JSON 负载的文本帧。
- 第一帧**必须**是 ``connect`` 请求。
- **Request**：``{type:"req", id, method, params}``
- **Response**：``{type:"res", id, ok, payload|error}``
- **Event**：``{type:"event", event, payload, seq?, stateVersion?}``
- ``operator`` = 控制平面客户端 (CLI/UI/自动化)。
- ``node`` = 能力宿主 (camera/screen/canvas/system.run)。
- ``operator.read``

操作示例：

- `"userAgent": "openclaw-cli/1.2.3",`
- `"userAgent": "openclaw-ios/1.2.3",`

22. 使用远程 Gateway 网关运行 OpenClaw.app

本节覆盖：

- 概述
- 快速设置
- 步骤 1：添加 SSH 配置

关键点：

- 登录时自动启动
- 崩溃时重新启动
- 在后台持续运行
- OpenClaw.app 使用 SSH 隧道连接到远程 Gateway 网关。本指南向你展示如何设置。

- 编辑 ~/.ssh/config 并添加：
- 将 <REMOTE_IP> 和 <REMOTE_USER> 替换为你的值。
- 将你的公钥复制到远程机器（输入一次密码）：
- 应用现在将通过 SSH 隧道连接到远程 Gateway 网关。

操作示例：

- | OpenClaw.app — ws://127.0.0.1:18789 (local port) |
- | Gateway WebSocket — ws://127.0.0.1:18789 — |
- IdentityFile ~/.ssh/id_rsa
- ssh-copy-id -i ~/.ssh/id_rsa <REMOTE_USER>@<REMOTE_IP>
- open /path/to/OpenClaw.app
- <!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">

23. 远程访问（SSH、隧道和 tailnet）

本节覆盖：

- 核心理念
- 常见的 VPN/tailnet 设置（智能体所在位置）
- 1) tailnet 中始终在线的 Gateway 网关（VPS 或家庭服务器）

关键点：

- 对于**操作员（你/macOS 应用）**：SSH 隧道是通用的回退方案。
- 对于**节点（iOS/Android 和未来的设备）**：连接到 Gateway **WebSocket**（LAN/tailnet 或根据需要通过 SSH 隧道）。
- Gateway WebSocket 绑定到你配置端口的 **loopback**（默认为 18789）。
- 对于远程使用，你通过 SSH 转发该 loopback 端口（或使用 tailnet/VPN 减少隧道需求）。
- **最佳用户体验**：保持 `gateway.bind: "loopback"` 并使用 **Tailscale Serve** 作为控制 UI。
- **回退方案**：保持 loopback + 从任何需要访问的机器建立 SSH 隧道。
- **示例**：** [exe.dev] [/install/exe-dev]（简易 VM）或 [Hetzner] [/install/hetzner]（生产 VPS）。
- 使用 macOS 应用的 **Remote over SSH** 模式（设置 → 通用 → "OpenClaw runs"）。

操作示例：

- url: "ws://127.0.0.1:18789",

24. 沙箱 vs 工具策略 vs 提权

本节覆盖：

- 快速调试
- 沙箱：工具在哪里运行
- 绑定挂载（安全快速检查）

关键点：

- ****沙箱**** (``agents.defaults.sandbox.*`` / ``agents.list[].sandbox.*``) 决定****工具在哪里运行**** (Docker vs 主机) 。
- ****工具策略**** (``tools.*``、``tools.sandbox.tools.*``、``agents.list[].tools.*``) 决定****哪些工具可用/允许**** 。
- ****提权**** (``tools.elevated.*``、``agents.list[].tools.elevated.*``) 是一个****仅限 exec 的逃逸通道****，允许在沙箱隔离时在主机上运行。
- 生效的沙箱模式/范围/工作区访问
- 会话当前是否被沙箱隔离（主 vs 非主）
- 生效的沙箱工具允许/拒绝（以及它来自智能体/全局/默认哪里）
- 提权限制和修复键路径
- ``"off"``：所有内容在主机上运行。

操作示例：

- `openclaw sandbox explain`
- `openclaw sandbox explain --session agent:main:main`
- `openclaw sandbox explain --agent work`
- `openclaw sandbox explain --json`

25. 沙箱隔离

本节覆盖：

- 什么会被沙箱隔离
- 模式
- 作用域

关键点：

- 工具执行 (``exec``、``read``、``write``、``edit``、``apply_patch``、``process`` 等) 。
- 可选的沙箱浏览器 (``agents.defaults.sandbox.browser``) 。
- 默认情况下，当浏览器工具需要时，沙箱浏览器会自动启动（确保 CDP 可达）。
- ``agents.defaults.sandbox.browser.allowHostControl`` 允许沙箱会话显式定位主机浏览器。

- 可选的允许列表限制 `target`:
"custom" : `allowedControlUrls`、`allowedControlHosts`、`allowedControlPorts`。
- Gateway 网关进程本身。
- 任何明确允许在主机上运行的工具（例如 `tools.elevated`）。
- ****提权 exec 在主机上运行并绕过沙箱隔离。****

操作示例：

- docker: {
- binds: ["/home/user/source:/source:ro", "/var/run/docker.sock:/var/run/docker.sock"],
- docker: {
- binds: ["/mnt/cache:/cache:rw"],
- scripts/sandbox-setup.sh
- scripts/sandbox-browser-setup.sh

26. 安全性

本节覆盖：

- 快速检查：`openclaw security audit`
- 审计检查内容（高层概述）
- 凭证存储映射

关键点：

- 将常见渠道的 `groupPolicy="open"` 收紧为 `groupPolicy="allowlist"`（以及单账户变体）。
- 将 `logging.redactSensitive="off"` 恢复为 `""tools""`。
- 收紧本地权限（`~/openclaw` → `700`，配置文件 → `600`，以及常见状态文件如 `credentials/\*.json`、`agents/\*/agent/auth-profiles.json` 和 `agents/\*/sessions/sessions.json`）。
- 谁可以与你的机器人交谈
- 机器人被允许在哪里执行操作
- 机器人可以访问什么
- ****入站访问****（私信策略、群组策略、白名单）：陌生人能否触发机器人？
- ****工具影响范围****（提权工具 + 开放房间）：提示词注入是否可能转化为 shell/文件/网络操作？

操作示例：

- openclaw security audit
- openclaw security audit --deep
- openclaw security audit --fix

- openclaw pairing list <channel>
- openclaw pairing approve <channel> <code>
- workspace: "~/.openclaw/workspace-personal",

27. Tailscale (Gateway 网关仪表盘)

本节覆盖：

- 模式
- 认证
- 配置示例

关键点：

- `serve`：仅限 Tailnet 的 Serve，通过 `tailscale serve`。Gateway 网关保持在 `127.0.0.1` 上。
- `funnel`：通过 `tailscale funnel` 的公共 HTTPS。OpenClaw 需要共享密码。
- `off`：默认（无 Tailscale 自动化）。
- `token`（设置 `OPENCLAW_GATEWAY_TOKEN` 时的默认值）
- `password`（通过 `OPENCLAW_GATEWAY_PASSWORD` 或配置的共享密钥）
- 控制 UI：`http://<tailscale-ip>:18789/`
- WebSocket：`ws://<tailscale-ip>:18789`
- Tailscale Serve/Funnel 需要安装并登录 `tailscale` CLI。

操作示例：

- `openclaw gateway --tailscale serve`
- `openclaw gateway --tailscale funnel --auth password`

28. 工具调用 (HTTP)

本节覆盖：

- 认证
- 请求体
- 策略 + 路由行为

关键点：

- `POST /tools/invoke`
- 与 Gateway 网关相同的端口（WS + HTTP 多路复用）：`http://<gateway-host>:<port>/tools/invoke`
- `Authorization: Bearer <token>`

- 当 `gateway.auth.mode="token"` 时，使用 `gateway.auth.token`（或 `OPENCLAW\_GATEWAY\_TOKEN`）。
- 当 `gateway.auth.mode="password"` 时，使用 `gateway.auth.password`（或 `OPENCLAW\_GATEWAY\_PASSWORD`）。
- `tool`（string，必需）：要调用的工具名称。
- `action`（string，可选）：如果工具 schema 支持 `action` 且 args 负载省略了它，则映射到 args。
- `args`（object，可选）：工具特定的参数。

操作示例：

- `curl -sS http://127.0.0.1:18789/tools/invoke \`
- `-H 'Content-Type: application/json' \`

29. 故障排除

本节覆盖：

- 状态与诊断
- 常见问题
- No API key found for provider "anthropic"

关键点：

- 重新运行新手引导并为该智能体选择 **Anthropic**。
- 或在 **Gateway 网关主机** 上粘贴 setup-token：
- 或将 `auth-profiles.json` 从主智能体目录复制到新智能体目录。
- 优先通过 [Tailscale Serve](/gateway/tailscale) 使用 HTTPS。
- 或在 Gateway 网关主机上本地打开：`http://127.0.0.1:18789/`。
- 如果必须使用 HTTP，启用 `gateway.controlUi.allowInsecureAuth: true` 并
- 优先：`openclaw logs --follow`
- 文件日志（始终）：`/tmp/openclaw/openclaw-YYYY-MM-DD.log`（或你配置的 `logging.file`）

操作示例：

- `openclaw models auth setup-token --provider anthropic`
- `openclaw models status`
- `openclaw models auth setup-token --provider anthropic`
- `openclaw models status`
- `openclaw models auth paste-token --provider anthropic`

- openclaw models status

第十二章 平台专项

1. Android 应用（节点）

本节覆盖：

- 支持概览
- 系统控制
- 连接操作手册

关键点：

- 角色：配套节点应用（Android 不托管 Gateway 网关）。
- 需要 Gateway 网关：是（在 macOS、Linux 或通过 WSL2 的 Windows 上运行）。
- 安装：[入门指南](/start/getting-started) + [配对](/gateway/pairing)。
- Gateway 网关：[操作手册](/gateway) + [配置](/gateway/configuration)。
- 协议：[Gateway 网关协议](/gateway/protocol)（节点 + 控制平面）。
- 你可以在"主"机器上运行 Gateway 网关。
- Android 设备/模拟器可以访问 Gateway 网关 WebSocket：
- 使用 mDNS/NSD 的同一局域网，**或**

操作示例：

- `openclaw gateway --port 18789 --verbose`
- `openclaw nodes pending`
- `openclaw nodes approve <requestId>`
- `openclaw nodes status`
- `openclaw gateway call node.list --params "{}"`
- `openclaw nodes invoke --node "<Android Node>" --command canvas.navigate --params '{"url":"http://<gateway-hostname>.local:18793/__openclaw__/canvas/"}'`

2. 在 DigitalOcean 上运行 OpenClaw

本节覆盖：

- 目标
- 成本比较（2026）
- 前提条件

关键点：

- DigitalOcean：最简单的用户体验 + 可预测的设置（本指南）
- Hetzner：性价比高（参见 [Hetzner 指南](/install/hetzner)）

- Oracle Cloud : 可以 \$0/月, 但更麻烦且仅限 ARM (参见 [Oracle 指南](/platforms/oracle))
- DigitalOcean 账户 ([注册可获 \$200 免费额度](https://m.do.co/c/signup))
- SSH 密钥对 (或愿意使用密码认证)
- 约 20 分钟
- 登录 [DigitalOcean](https://cloud.digitalocean.com/)
- 点击 **Create → Droplets**

操作示例 :

- `curl -fsSL https://deb.nodesource.com/setup_22.x | bash -`
- `curl -fsSL https://openclaw.ai/install.sh | bash`
- `openclaw --version`
- `openclaw onboard --install-daemon`
- `openclaw status`
- `systemctl --user status openclaw-gateway.service`

3. 平台

本节覆盖 :

- 选择你的操作系统
- VPS 和托管
- 常用链接

关键点 :

- macOS : [macOS](/platforms/macos)
- iOS : [iOS](/platforms/ios)
- Android : [Android](/platforms/android)
- Windows : [Windows](/platforms/windows)
- Linux : [Linux](/platforms/linux)
- VPS 中心 : [VPS 托管](/vps)
- Fly.io : [Fly.io](/install/fly)
- Hetzner (Docker) : [Hetzner](/install/hetzner)

4. iOS 应用 (节点)

本节覆盖 :

- 功能

- 要求
- 快速开始（配对 + 连接）

关键点：

- 通过 WebSocket（LAN 或 tailnet）连接到 Gateway 网关。
- 暴露节点能力：Canvas、屏幕快照、相机捕获、位置、对话模式、语音唤醒。
- 接收 `node.invoke` 命令并报告节点状态事件。
- Gateway 网关运行在另一台设备上（macOS、Linux 或通过 WSL2 的 Windows）。
- 通过 Bonjour 的同一 LAN，**或**
- 通过单播 DNS-SD 的 Tailnet（示例域：`openclaw.internal.`），**或**
- 手动主机/端口（备选）。
- 启动 Gateway 网关：

操作示例：

- `openclaw gateway --port 18789`
- `openclaw nodes pending`
- `openclaw nodes approve <requestId>`
- `openclaw nodes status`
- `openclaw gateway call node.list --params "{}"`
- `openclaw nodes invoke --node "iOS Node" --command canvas.navigate --params '{"url":"http://<gateway-host>:18793/__openclaw__/canvas/"}'`

5. Linux 应用

本节覆盖：

- 新手快速路径（VPS）
- 安装
- Gateway 网关

关键点：

- 安装 Node 22+
- ``npm i -g openclaw@latest``
- ``openclaw onboard --install-daemon``
- 从你的笔记本电脑：``ssh -N -L 18789:127.0.0.1:18789 <user>@<host>``
- 打开 ``http://127.0.0.1:18789/`` 并粘贴你的令牌
- [入门指南](/start/getting-started)

- [安装与更新](/install/updating)
- 可选流程：[Bun（实验性）](/install/bun)、[Nix](/install/nix)、[Docker](/install/docker)

操作示例：

- `openclaw onboard --install-daemon`
- `openclaw gateway install`
- `openclaw configure`
- `openclaw doctor`
- `ExecStart=/usr/local/bin/openclaw gateway --port 18789`
- `systemctl --user enable --now openclaw-gateway[-<profile>].service`

6. macOS 上的 Gateway 网关（外部 launchd）

本节覆盖：

- 安装 CLI（本地模式必需）
- Launchd（Gateway 网关作为 LaunchAgent）
- 版本兼容性

关键点：

- ``bot.molt.gateway``（或 ``bot.molt.<profile>``；旧版 ``com.openclaw.*`` 可能仍然存在）
- ``~/Library/LaunchAgents/bot.molt.gateway.plist``
- macOS 应用在本地模式下拥有 LaunchAgent 的安装/更新权限。
- CLI 也可以安装它：``openclaw gateway install``。
- "OpenClaw Active"启用/禁用 LaunchAgent。
- 应用退出**不会**停止 Gateway 网关（launchd 保持其存活）。
- 如果 Gateway 网关已经在配置的端口上运行，应用会连接到它而不是启动新的。
- launchd stdout/err：``/tmp/openclaw/openclaw-gateway.log``

操作示例：

- `npm install -g openclaw@<version>`
- `openclaw --version`
- `openclaw gateway --port 18999 --bind loopback`
- `openclaw gateway call health --url ws://127.0.0.1:18999 --timeout 3000`

7. Canvas（macOS 应用）

本节覆盖：

- Canvas 存储位置
- 面板行为
- 智能体 API 接口

关键点：

- `~/Library/Application Support/OpenClaw/canvas/<session>/...``
- `openclaw-canvas://<session>/<path>``
- `openclaw-canvas://main/`` → `<canvasRoot>/main/index.html``
- `openclaw-canvas://main/assets/app.css`` → `<canvasRoot>/main/assets/app.css``
- `openclaw-canvas://main/widgets/todo/`` → `<canvasRoot>/main/widgets/todo/index.html``
- 无边框、可调整大小的面板，锚定在菜单栏（或鼠标光标）附近。
- 记住每个会话的大小/位置。
- 当本地 canvas 文件更改时自动重新加载。

操作示例：

- `openclaw nodes canvas present --node <id>`
- `openclaw nodes canvas navigate --node <id> --url "/"`
- `openclaw nodes canvas eval --node <id> --js "document.title"`
- `openclaw nodes canvas snapshot --node <id>`
- `http://<gateway-host>:18793/__openclaw__/a2ui/`
- `cat > /tmp/a2ui-v0.8.jsonl <<'EOFA2'`

8. macOS 上的 Gateway 网关生命周期

本节覆盖：

- 默认行为 (launchd)
- 未签名的开发构建
- 仅连接模式

关键点：

- 应用安装标记为 `bot.molt.gateway`` 的按用户 LaunchAgent
- 当启用本地模式时，应用确保 LaunchAgent 已加载，并
- 日志写入 launchd Gateway 网关日志路径（在调试设置中可见）。
- 写入 `~/Library/OpenClaw/disable-launchagent``。
- 登录时自动启动。

- 内置的重启/KeepAlive 语义。
- 可预测的日志和监管。
- macOS 应用默认通过 launchd 管理 Gateway 网关，不会将

操作示例：

- `launchctl kickstart -k gui/$UID/bot.molt.gateway`
- `launchctl bootout gui/$UID/bot.molt.gateway`
- `rm ~/.openclaw/disable-launchagent`

9. macOS 开发者设置

本节覆盖：

- 前置条件
- 1. 安装依赖
- 2. 构建和打包应用

关键点：

- **Xcode 26.2+**：Swift 开发所需。
- **Node.js 22+ & npm**：Gateway 网关、CLI 和打包脚本所需。
- 打开 OpenClaw 应用。
- 转到 **General** 设置标签页。
- 点击 **"Install CLI"**。
- **软件更新中可用的最新 macOS 版本**（Xcode 26.2 SDK 所需）
- **Xcode 26.2**（Swift 6.2 工具链）
- 重置 TCC 权限：

操作示例：

- `./scripts/package-mac-app.sh`
- `npm install -g openclaw@<version>`
- `xcrun swift --version`
- `openclaw gateway status`
- `openclaw gateway stop`
- # 如果你没有使用 LaunchAgent（开发模式/手动运行），找到监听器：

10. macOS 上的健康检查

本节覆盖：

- 菜单栏
- 设置
- 探测工作原理

关键点：

- 状态圆点现在反映 Baileys 健康状态：
- 绿色：已关联 + socket 最近已打开。
- 橙色：正在连接/重试。
- 红色：已登出或探测失败。
- 第二行显示"linked · auth 12m"或显示失败原因。
- "Run Health Check"菜单项触发按需探测。
- 通用选项卡新增健康卡片，显示：关联认证时间、会话存储路径/数量、上次检查时间、上次错误/状态码，以及运行健康检查/显示日志按钮。
- 使用缓存快照，因此 UI 立即加载，离线时优雅降级。

11. 菜单栏图标状态

关键点：

- ****空闲****：正常图标动画（眨眼、偶尔摆动）。
- ****暂停****：状态项使用 `appearsDisabled`；无动画。
- ****语音触发（大耳朵）****：语音唤醒检测器在听到唤醒词时调用 `AppState.triggerVoiceEars(ttl: nil)`，在捕获语音期间保持 `earBoostActive=true`。耳朵放大（1.9 倍），显示圆形耳孔以提高可读性，然后在 1 秒静音后通过 `stopVoiceEars()` 恢复。仅由应用内语音管道触发。
- ****工作中（智能体运行中）****：`AppState.isWorking=true` 驱动"尾巴/腿部快速摆动"微动画：工作进行中腿部摆动加快并略有偏移。目前在 WebChat 智能体运行时切换；在接入其他长时间任务时请添加相同的切换逻辑。
- **语音唤醒**：运行时/测试器在触发时调用 `AppState.triggerVoiceEars(ttl: nil)`，在 1 秒静音后调用 `stopVoiceEars()` 以匹配捕获窗口。
- **智能体活动**：在工作区间前后设置 `AppStateStore.shared.setWorking(true/false)`（已在 WebChat 智能体调用中完成）。保持区间简短，并在 `defer` 块中重置以避免动画卡住。
- **基础图标在**
`CritterIconRenderer.makeIcon(blink:legWiggle:earWiggle:earScale:earHoles:)` 中绘制。
- 耳朵缩放默认为 `1.0`；语音增强时设置 `earScale=1.9` 并切换 `earHoles=true`，不改变整体框架（18×18 pt 模板图像渲染到 36×36 px Retina 后备存储）。

12. 日志（macOS）

本节覆盖：

- 滚动诊断文件日志（Debug 面板）
- macOS 上统一日志的隐私数据
- 为 OpenClaw 启用（`bot.molt`）

关键点：

- 详细级别：**Debug 面板 → Logs → App logging → Verbosity**
- 启用：**Debug 面板 → Logs → App logging → "Write rolling diagnostics log (JSONL)"**
- 位置：`~/Library/Logs/OpenClaw/diagnostics.jsonl`（自动轮转；旧文件以`.1`、`.2`、... 为后缀）
- 清除：**Debug 面板 → Logs → App logging → "Clear"**
- 此功能**默认关闭**。仅在主动调试时启用。
- 该文件包含敏感信息；分享前请先审查内容。
- 先将 plist 写入临时文件，然后以 root 身份原子性地安装：
- 无需重启；logd 会很快检测到该文件，但只有新的日志行才会包含隐私负载。

操作示例：

- `cat <<'EOF'>/tmp/bot.molt.plist`
- `<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">`
- `<key>DEFAULT-OPTIONS</key>`
- `<key>Enable-Private-Data</key>`
- `<true/>`
- `</dict>`

13. 菜单栏状态逻辑

本节覆盖：

- 显示内容
- 状态模型
- IconState 枚举（Swift）

关键点：

- 我们在菜单栏图标和菜单的第一行状态行中展示当前智能体的工作状态。
- 工作活跃时隐藏健康状态；当所有会话空闲时恢复显示。
- 菜单中的"节点"区块仅列出**设备**（通过`node.list`配对的节点），不包括客户端/在线状态条目。

- 当提供商用量快照可用时，"用量"部分会显示在上下文下方。
- 会话：事件携带 `runId`（每次运行）以及载荷中的 `sessionKey`。"main" 会话的键为 `main`；如果不存在，则回退到最近更新的会话。
- 优先级：main 始终优先。如果 main 处于活跃状态，立即显示其状态。如果 main 空闲，则显示最近活跃的非 main 会话。活动进行中不会来回切换；仅在当前会话进入空闲或 main 变为活跃时才切换。
- `job`：高层命令执行（`state: started|streaming|done|error`）。
- `tool`：`phase: start|result`，包含 `toolName` 和 `meta/args`。

14. Peekaboo Bridge (macOS UI 自动化)

本节覆盖：

- 这是什么（以及不是什么）
- 启用桥接
- 安全与权限

关键点：

- **宿主**：OpenClaw.app 可以作为 PeekabooBridge 宿主。
- **客户端**：使用 `peekaboo` CLI（无需单独的 `openclaw ui ...` 界面）。
- **界面**：视觉叠加层保留在 Peekaboo.app 中；OpenClaw 只是一个轻量代理宿主。
- 设置 → **启用 Peekaboo Bridge**
- Peekaboo.app（完整用户体验）
- Claude.app（如已安装）
- OpenClaw.app（轻量代理）
- 桥接会验证**调用方的代码签名**；强制执行 TeamID 白名单（Peekaboo 宿主 TeamID + OpenClaw 应用 TeamID）。

15. macOS 权限 (TCC)

本节覆盖：

- 稳定权限的要求
- 权限提示消失时的恢复清单

关键点：

- 相同路径：从固定位置运行应用（对于 OpenClaw，为 `dist/OpenClaw.app`）。
- 相同 Bundle 标识符：更改 Bundle ID 会创建新的权限身份。
- 已签名的应用：未签名或临时签名的构建不会持久化权限。

- 一致的签名：使用真实的 Apple Development 或 Developer ID 证书，以确保签名在多次构建之间保持稳定。
- 在系统设置 -> 隐私与安全性中移除该应用条目。
- 从相同路径重新启动应用并重新授予权限。
- 如果提示仍未出现，使用 `tccutil` 重置 TCC 条目后重试。
- 某些权限仅在完全重启 macOS 后才会重新出现。

16. OpenClaw macOS 发布 (Sparkle)

本节覆盖：

- 前提条件
- 构建与打包
- Appcast 条目

关键点：

- 已安装 Developer ID Application 证书（示例：`Developer ID Application: <Developer Name> (<TEAMID>)`）。
- 环境变量 `SPARKLE\_PRIVATE\_KEY\_FILE` 已设置为 Sparkle ed25519 私钥路径（公钥已嵌入 Info.plist）。如果缺失，请检查 `~/profile`。
- 用于 `xcrun notarytool` 的公证凭据（钥匙串配置文件或 API 密钥），以实现通过 Gatekeeper 安全分发的 DMG/zip。
- 我们使用名为 `openclaw-notary` 的钥匙串配置文件，由 shell 配置文件中的 App Store Connect API 密钥环境变量创建：
- `APP\_STORE\_CONNECT\_API\_KEY\_P8`、`APP\_STORE\_CONNECT\_KEY\_ID`、`APP\_STORE\_CONNECT\_ISSUER\_ID`
- ```
echo "$APP_STORE_CONNECT_API_KEY_P8" | sed 's/\n/g' > /tmp/openclaw-notary.p8
```
- ```
xcrun notarytool store-credentials "openclaw-notary" --key /tmp/openclaw-notary.p8 --key-id "$APP_STORE_CONNECT_KEY_ID" --issuer "$APP_STORE_CONNECT_ISSUER_ID"
```
- 已安装 `pnpm` 依赖（`pnpm install --config.node-linker=hoisted`）。

操作示例：

- ```
APP_BUILD="$(git rev-list --count HEAD)" \
```
- ```
scripts/package-mac-app.sh
```
- ```
ditto -c -k --sequesterRsrc --keepParent dist/OpenClaw.app dist/OpenClaw-2026.1.27-beta.1.zip
```
- # 可选：同时构建适合用户使用的样式化 DMG（拖拽到 /Applications）

- `scripts/create-dmg.sh dist/OpenClaw.app dist/OpenClaw-2026.1.27-beta.1.dmg`
- # 推荐：构建 + 公证/装订 zip + DMG

## 17. 远程 OpenClaw (macOS 远程主机)

本节覆盖：

- 模式
- 远程传输
- 远程主机上的先决条件

关键点：

- **Local (this Mac)**：一切都在笔记本电脑上运行。不涉及 SSH。
- **Remote over SSH (默认)**：OpenClaw 命令在远程主机上执行。mac 应用使用 `-o BatchMode`` 加上你选择的身份/密钥打开 SSH 连接，并进行本地端口转发。
- **Remote direct (ws/wss)**：无 SSH 隧道。mac 应用直接连接到 Gateway 网关 URL（例如，通过 Tailscale Serve 或公共 HTTPS 反向代理）。
- **SSH 隧道**（默认）：使用 `ssh -N -L ...`` 将 Gateway 网关端口转发到 localhost。Gateway 网关会将节点的 IP 视为 `127.0.0.1``，因为隧道是 loopback。
- **Direct (ws/wss)**：直接连接到 Gateway 网关 URL。Gateway 网关看到真实的客户端 IP。
- 安装 Node + pnpm 并构建/安装 OpenClaw CLI (`pnpm install && pnpm build && pnpm link --global``)。
- 确保 `openclaw`` 在非交互式 shell 的 PATH 中（如需要，请符号链接到 `/usr/local/bin`` 或 `/opt/homebrew/bin``）。
- 使用密钥认证打开 SSH。我们推荐使用 **Tailscale** IP 以实现离开局域网时的稳定可达性。

操作示例：

- `openclaw nodes notify --node <id> --title "Ping" --body "Remote gateway ready" --sound Glass`

## 18. Mac 签名 (调试构建)

本节覆盖：

- 用法
- 临时签名注意事项
- 关于面板的构建元数据

关键点：

- 设置稳定的调试 Bundle 标识符：`ai.openclaw.mac.debug``
- 使用该 Bundle ID 写入 Info.plist（可通过 `BUNDLE_ID=...`` 覆盖）

- 调用 [`scripts/codesign-mac-app.sh`](https://github.com/openclaw/openclaw/blob/main/scripts/codesign-mac-app.sh) 对主二进制文件和应用包进行签名，使 macOS 将每次重新构建视为相同的已签名包，并保留 TCC 权限（通知、辅助功能、屏幕录制、麦克风、语音）。要获得稳定的权限，请使用真实签名身份；临时签名是可选的且不稳定（参阅 [macOS 权限](/platforms/mac/permissions)）。

- 默认使用 `CODESIGN_TIMESTAMP=auto`；为 Developer ID 签名启用受信任的时间戳。设置 `CODESIGN_TIMESTAMP=off` 可跳过时间戳（离线调试构建）。

- 将构建元数据注入 Info.plist：`OpenClawBuildTimestamp`（UTC）和 `OpenClawGitCommit`（短哈希），以便"关于"面板可以显示构建信息、git 信息和调试/发布渠道。

- **\*\*打包需要 Node 22+\*\***：脚本会运行 TS 构建和 Control UI 构建。

- 从环境变量中读取 `SIGN_IDENTITY`。将 `export SIGN_IDENTITY="Apple Development: Your Name (TEAMID)"`（或你的 Developer ID Application 证书）添加到 shell 配置文件中，以始终使用你的证书签名。临时签名需要通过 `ALLOW_ADHOC_SIGNING=1` 或 `SIGN_IDENTITY="-"` 显式启用（不建议用于权限测试）。

- 签名后运行 Team ID 审计，如果应用包内的任何 Mach-O 文件由不同的 Team ID 签名则会失败。设置 `SKIP_TEAM_ID_CHECK=1` 可跳过此检查。

操作示例：

- `scripts/package-mac-app.sh` # 自动选择身份；未找到时报错

- `SIGN_IDENTITY="Developer ID Application: Your Name" scripts/package-mac-app.sh` # 真实证书

- `ALLOW_ADHOC_SIGNING=1 scripts/package-mac-app.sh` # 临时签名（权限不会持久化）

- `SIGN_IDENTITY="-" scripts/package-mac-app.sh` # 显式临时签名（同样的限制）

- `DISABLE_LIBRARY_VALIDATION=1 scripts/package-mac-app.sh` # 仅限开发的 Sparkle Team ID 不匹配解决方案

## 19. Skills (macOS)

本节覆盖：

- 数据来源
- 安装操作
- 环境变量/API 密钥

关键点：

- `skills.status`（Gateway 网关）返回所有 Skills 以及资格和缺失的要求
- 要求来源于每个 `SKILL.md` 中的 `metadata.openclaw.requires`。
- `metadata.openclaw.install` 定义安装选项（brew/node/go/uv）。

- 应用调用 `skills.install` 在 Gateway 网关主机上运行安装器。
- 当提供多个安装器时，Gateway 网关仅展示一个首选安装器
- 应用将密钥存储在 `~/openclaw/openclaw.json` 的 `skills.entries.<skillKey>` 下。
- `skills.update` 更新 `enabled`、`apiKey` 和 `env`。
- 安装 + 配置更新发生在 Gateway 网关主机上（不是本地 Mac）。

## 20. 语音浮层生命周期 (macOS)

本节覆盖：

- 当前意图
- 已实现 (2025 年 12 月 9 日)
- 后续步骤

关键点：

- 如果浮层已因唤醒词显示，此时用户按下热键，热键会话会\*接管\*现有文本而非重置。浮层在热键按住期间保持显示。用户松开时：如果有去除空白后的文本则发送，否则关闭。
- 单独使用唤醒词时仍在静音后自动发送；按键说话在松开时立即发送。
- 浮层会话现在为每次捕获（唤醒词或按键说话）携带一个令牌。当令牌不匹配时，部分/最终/发送/关闭/音量更新会被丢弃，避免过时回调。
- 按键说话会接管任何可见的浮层文本作为前缀（因此在唤醒浮层显示时按下热键会保留文本并追加新语音）。它最多等待 1.5 秒获取最终转录结果，然后回退到当前文本。
- 提示音/浮层日志以 `info` 级别输出，分类为 `voicewake.overlay`、`voicewake.ptt` 和 `voicewake.chime`（会话开始、部分、最终、发送、关闭、提示音原因）。
- **VoiceSessionCoordinator (actor)**
- 同一时间只拥有一个 `VoiceSession`。
- API（基于令牌）：`beginWakeCapture`、`beginPushToTalk`、`updatePartial`、`endCapture`、`cancel`、`applyCooldown`。

操作示例：

- `sudo log stream --predicate 'subsystem == "bot.molt" AND category CONTAINS "voicewake"' --level info --style compact`

## 21. 语音唤醒与按键通话

本节覆盖：

- 模式
- 运行时行为（唤醒词）
- 生命周期不变量

关键点：

- **\*\*唤醒词模式\*\***（默认）：常驻语音识别器等待触发词（`swabbleTriggerWords`）。匹配时开始捕获，显示带有部分文本的悬浮窗，并在静默后自动发送。
- **\*\*按键通话（按住右 Option 键）\*\***：按住右 Option 键立即开始捕获——无需触发词。按住时显示悬浮窗；松开后延迟片刻再最终转发，以便你可以调整文本。
- 语音识别器位于 `VoiceWakeRuntime` 中。
- 仅当唤醒词和下一个词之间有**\*\*明显停顿\*\***（约 0.55 秒间隔）时才触发。悬浮窗/提示音可以在命令开始前的停顿时就启动。
- 静默窗口：语音流畅时为 2.0 秒，如果只听到触发词则为 5.0 秒。
- 硬性停止：120 秒，防止会话失控。
- 会话间去抖动：350 毫秒。
- 悬浮窗通过 `VoiceWakeOverlayController` 驱动，带有已提交/临时状态的颜色区分。

## 22. WebChat (macOS 应用)

本节覆盖：

- 启动和调试
- 工作原理
- 安全面

关键点：

- **\*\*本地模式\*\***：直接连接到本地 Gateway 网关 WebSocket。
- **\*\*远程模式\*\***：通过 SSH 转发 Gateway 网关控制端口，并使用该隧道作为数据平面。
- 手动：Lobster 菜单 → "Open Chat"。
- 测试时自动打开：
- 日志：`.scripts/clawlog.sh`（子系统 `bot.molt`，类别 `WebChatSwiftUI`）。
- 数据平面：Gateway 网关 WS 方法  
`chat.history`、`chat.send`、`chat.abort`、`chat.inject` 和事件  
`chat`、`agent`、`presence`、`tick`、`health`。
- 会话：默认为主会话（`main`，或当范围为全局时为 `global`）。UI 可以在会话之间切换。
- 新手引导使用专用会话，以将首次运行设置分开。

操作示例：

- `dist/OpenClaw.app/Contents/MacOS/OpenClaw --webchat`

## 23. OpenClaw macOS IPC 架构

本节覆盖：

- 目标
- 工作原理
- Gateway 网关 + 节点传输

关键点：

- 单个 GUI 应用实例拥有所有面向 TCC 的工作（通知、屏幕录制、麦克风、语音、AppleScript）。
- 小型自动化接口：Gateway 网关 + 节点命令，加上用于 UI 自动化的 PeekabooBridge。
- 可预测的权限：始终是同一个签名的 bundle ID，由 launchd 启动，因此 TCC 授权保持有效。
- 应用运行 Gateway 网关（本地模式）并作为节点连接到它。
- 智能体操作通过 `node.invoke`` 执行（例如 `system.run``、`system.notify``、`canvas.*``）。
- 一个无头节点主机服务连接到 Gateway 网关 WebSocket。
- `system.run`` 请求通过本地 Unix 套接字转发到 macOS 应用。
- 应用在 UI 上下文中执行 `exec`，必要时提示，并返回输出。

## 24. OpenClaw macOS 配套应用（菜单栏 + Gateway 网关代理）

本节覆盖：

- 功能
- 本地 vs 远程模式
- Launchd 控制

关键点：

- 在菜单栏中显示原生通知和状态。
- 拥有 TCC 提示（通知、辅助功能、屏幕录制、麦克风、语音识别、自动化/AppleScript）。
- 运行或连接到 Gateway 网关（本地或远程）。
- 暴露 macOS 专用工具（Canvas、相机、屏幕录制、`system.run``）。
- 在\*\*远程\*\*模式下启动本地节点主机服务（launchd），在\*\*本地\*\*模式下停止它。
- 可选地托管 **PeekabooBridge** 用于 UI 自动化。
- 根据请求通过 npm/pnpm 安装全局 CLI（`openclaw``）（不建议使用 bun 作为 Gateway 网关运行时）。
- **本地**（默认）：如果存在运行中的本地 Gateway 网关，应用附加到它；否则通过 `openclaw gateway install`` 启用 launchd 服务。

操作示例：

- `launchctl kickstart -k gui/$UID/bot.molt.gateway`
- `launchctl bootout gui/$UID/bot.molt.gateway`

- ~/.openclaw/exec-approvals.json
- "allowlist": [{ "pattern": "/opt/homebrew/bin/rg" }]
- open 'openclaw://agent?message=Hello%20from%20deep%20link'
- cd apps/macos

## 25. 在 Oracle Cloud (OCI) 上运行 OpenClaw

本节覆盖：

- 目标
- 成本比较 (2026)
- 先决条件

关键点：

- ARM 架构 (大多数东西都能工作，但某些二进制文件可能仅支持 x86)
- 容量和注册可能比较麻烦
- Oracle Cloud 账户 ([注册](https://www.oracle.com/cloud/free/)) ——如果遇到问题请参阅[社区注册指南](https://gist.github.com/rssnyder/51e3cfedd730e7dd5f4a816143b25dbd)
- Tailscale 账户 (在 [tailscale.com](https://tailscale.com) 免费)
- 约 30 分钟
- 登录 [Oracle Cloud Console](https://cloud.oracle.com/)
- **Name:** `openclaw`
- **Image:** Ubuntu 24.04 (aarch64)

操作示例：

- curl -fsSL https://tailscale.com/install.sh | sh
- sudo tailscale up --ssh --hostname=openclaw
- curl -fsSL https://openclaw.ai/install.sh | bash
- source ~/.bashrc
- openclaw config set gateway.bind loopback
- openclaw config set gateway.auth.mode token

## 26. 在 Raspberry Pi 上运行 OpenClaw

本节覆盖：

- 目标
- 硬件要求
- 你需要准备



关键点：

- 24/7 个人 AI 助手
- 家庭自动化中心
- 低功耗、随时可用的 Telegram/WhatsApp 机器人
- Raspberry Pi 4 或 5（推荐 2GB+）
- MicroSD 卡（16GB+）或 USB SSD（性能更好）
- 电源（推荐官方 Pi 电源）
- 网络连接（以太网或 WiFi）
- 约 30 分钟

操作示例：

- # 设置时区（对 cron/提醒很重要）
- `sudo timedatectl set-timezone America/Chicago` # 改成你的时区
- `curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -`
- `node --version` # 应显示 v22.x.x
- `npm --version`
- `sudo fallocate -l 2G /swapfile`

## 27. Windows (WSL2)

本节覆盖：

- 安装 (WSL2)
- Gateway 网关
- Gateway 网关服务安装 (CLI)

关键点：

- [入门指南](/start/getting-started)（在 WSL 内使用）
- [安装和更新](/install/updating)
- 官方 WSL2 指南（Microsoft）：<https://learn.microsoft.com/windows/wsl/install>
- [Gateway 网关操作手册](/gateway)
- [配置](/gateway/configuration)
- 从另一台机器 SSH 目标是 \*\*Windows 主机 IP\*\*（示例：``ssh user@windows-host -p 2222``）。
- 远程节点必须指向\*\*可访问的\*\* Gateway 网关 URL（不是 ``127.0.0.1``）；使用 ``openclaw status --all`` 确认。
- 使用 ``listenaddress=0.0.0.0`` 进行 LAN 访问；``127.0.0.1`` 仅保持本地访问。

操作示例：

- openclaw onboard --install-daemon
- openclaw gateway install
- openclaw configure
- openclaw doctor
- \$Wslp = (wsl -d \$Distro -- hostname -l).Trim().Split(" ")[0]
- wsl --install

## 第十三章 插件扩展

### 1. 插件智能体工具

本节覆盖：

- 基本工具
- 可选工具（选择启用）
- 规则 + 提示

关键点：

- 仅包含插件工具名称的允许列表被视为插件选择启用；核心工具保持启用，除非你在允许列表中也包含核心工具或组。
- ``tools.profile`` / ``agents.list[].tools.profile``（基础允许列表）
- ``tools.byProvider`` / ``agents.list[].tools.byProvider``（特定提供商的允许/拒绝）
- ``tools.sandbox.tools.*``（沙箱隔离时的沙箱工具策略）
- 工具名称\*\*不能\*\*与核心工具名称冲突；冲突的工具会被跳过。
- 允许列表中使用的插件 id 不能与核心工具名称冲突。
- 对于触发副作用或需要额外二进制文件/凭证的工具，优先使用 ``optional: true``。
- OpenClaw 插件可以注册智能体工具（JSON 模式函数），这些工具在智能体运行期间暴露给 LLM。工具可以是必需的（始终可用）或可选的（选择启用）。

操作示例：

- `"workflow_tool"`, // 特定工具名称
- `"workflow"`, // 插件 id（启用该插件的所有工具）
- `"group:plugins"`, // 所有插件工具

### 2. 插件清单（openclaw.plugin.json）

本节覆盖：

- 必填字段
- JSON Schema 要求
- 验证行为

关键点：

- ``id``（字符串）：插件的规范 id。
- ``configSchema``（对象）：插件配置的 JSON Schema（内联形式）。
- ``kind``（字符串）：插件类型（例如：``"memory"``）。
- ``channels``（数组）：此插件注册的渠道 id（例如：``["matrix"]``）。

- ``providers``（数组）：此插件注册的提供商 id。
- ``skills``（数组）：要加载的 Skills 目录（相对于插件根目录）。
- ``name``（字符串）：插件的显示名称。
- ``description``（字符串）：插件简短描述。

### 3. Voice Call（插件）

本节覆盖：

- 运行位置（本地 vs 远程）
- 安装
- 选项 A：从 npm 安装（推荐）

关键点：

- ``twilio``（Programmable Voice + Media Streams）
- ``telnyx``（Call Control v2）
- ``plivo``（Voice API + XML transfer + GetInput speech）
- ``mock``（开发/无网络）
- 重启 Gateway 网关
- 在 ``plugins.entries.voice-call.config`` 下配置
- 使用 ``openclaw voicecall ...`` 或 ``voice_call`` 工具
- Twilio/Telnyx 需要\*\*可公开访问\*\*的 webhook URL。

操作示例：

- `openclaw plugins install @openclaw/voice-call`
- `openclaw plugins install ./extensions/voice-call`
- `cd ./extensions/voice-call && pnpm install`
- `provider: "twilio", // 或 "telnyx" | "plivo" | "mock"`
- `// Webhook 服务器`
- `path: "/voice/webhook",`

### 4. Zalo Personal（插件）

本节覆盖：

- 命名
- 运行位置
- 安装

关键点：

- 通过插件为 OpenClaw 提供 Zalo Personal 支持，使用 zca-cli 自动化普通 Zalo 用户账户。
- 警告：非官方自动化可能导致账户被暂停/封禁。使用风险自负。
- 渠道 id 是 zalouser，以明确表示这是自动化个人 Zalo 用户账户（非官方）。我们保留 zalo 用于潜在的未来官方 Zalo API 集成。
- 此插件在 Gateway 网关进程内运行。
- 如果你使用远程 Gateway 网关，请在运行 Gateway 网关的机器上安装/配置它，然后重启 Gateway 网关。
- 之后重启 Gateway 网关。
- Gateway 网关机器必须在 PATH 中有 zca：
- 渠道配置位于 channels.zalouser 下（不是 plugins.entries.）：

操作示例：

- `openclaw plugins install @openclaw/zalouser`
- `openclaw plugins install ./extensions/zalouser`
- `cd ./extensions/zalouser && pnpm install`
- `zca --version`
- `openclaw channels login --channel zalouser`
- `openclaw channels logout --channel zalouser`

## 第十四章 诊断与排障

### 1. 诊断标志

本节覆盖：

- 工作原理
- 通过配置启用
- 环境变量覆盖（一次性）

关键点：

- 标志是字符串（不区分大小写）。
- 你可以在配置中或通过环境变量覆盖来启用标志。
- 支持通配符：
- ``telegram.*`` 匹配 ``telegram.http``
- ``*`` 启用所有标志
- 如果 ``logging.level`` 设置为高于 ``warn``，这些日志可能会被抑制。默认的 ``info`` 级别即可。
- 标志可以安全地保持启用状态；它们只影响特定子系统的日志量。
- 使用 `[/logging](/logging)` 更改日志目标、级别和脱敏设置。

操作示例：

- `/tmp/openclaw/openclaw-YYYY-MM-DD.log`
- `ls -t /tmp/openclaw/openclaw-*.log | head -n 1`
- `rg "telegram http error" /tmp/openclaw/openclaw-*.log`
- `tail -f /tmp/openclaw/openclaw-$(date +%F).log | rg "telegram http error"`

# 第十五章 调试

## 1. Node + tsx "\\_\\_name is not a function" 崩溃

本节覆盖：

- 概述
- 环境
- 复现步骤（仅 Node）

关键点：

- Node: v25.x（在 v25.3.0 上观察到）
- tsx: 4.21.0
- 操作系统: macOS（其他运行 Node 25 的平台也可能复现）
- Node 25.3.0：失败
- Node 22.22.0（Homebrew `node@22`）：失败
- Node 24：尚未安装，需要验证
- `tsx` 使用 esbuild 转换 TS/ESM。esbuild 的 `keepNames` 会生成一个 `\_\_name` 辅助函数，并用 `\_\_name(...)` 包裹函数定义。
- 崩溃表明 `\_\_name` 存在但在运行时不是函数，这意味着在 Node 25 的加载器路径中该辅助函数缺失或被覆盖。

操作示例：

- `at createSubsystemLogger (.../src/logging/subsystem.ts:203:25)`
- `at .../src/agents/auth-profiles/constants.ts:25:20`
- `node --version`
- `node --import tsx src/entry.ts status`
- `node --import tsx scripts/repro/tsx-name-repro.ts`
- `pnpm exec tsc --watch --preserveWatchOutput`

# 第十六章 安全

## 1. 形式化验证（安全模型）

本节覆盖：

- 模型存放位置
- 重要注意事项
- 复现结果

关键点：

- 每个声明都有一个在有限状态空间上运行的模型检查。
- 许多声明有一个配对的\*\*负面模型\*\*，为现实的 bug 类别生成反例追踪。
- 这些是\*\*模型\*\*，不是完整的 TypeScript 实现。模型和代码之间可能存在偏差。
- 结果受 TLC 探索的状态空间限制；"绿色"并不意味着在建模的假设和边界之外也是安全的。
- 一些声明依赖于明确的环境假设（例如，正确的部署、正确的配置输入）。
- 带有公开产物（反例追踪、运行日志）的 CI 运行模型
- 用于小型、有界检查的托管"运行此模型"工作流
- ``make gateway-exposure-v2``

操作示例：

- `git clone https://github.com/vignesh07/openclaw-formal-models`
- # 仓库内置了固定版本的 ``tla2tools.jar``（TLA+ 工具）并提供 ``bin/tlc`` + Make 目标。



## 第十七章 参考资料

### 1. AGENTS.md — OpenClaw 个人助手（默认）

本节覆盖：

- 首次运行（推荐）
- 安全默认值
- 会话开始（必需）

关键点：

- 创建工作区（如果尚不存在）：
- 将默认工作区模板复制到工作区：
- 可选：如果你想要个人助手 Skills 列表，用此文件替换 AGENTS.md：
- 可选：通过设置 `agents.defaults.workspace` 选择不同的工作区（支持 `~`）：
- 不要将目录或密钥转储到聊天中。
- 除非明确要求，否则不要运行破坏性命令。
- 不要向外部消息界面发送部分/流式回复（仅发送最终回复）。
- 读取 `SOUL.md`、`USER.md`、`memory.md`，以及 `memory/` 中的今天和昨天的文件。

操作示例：

- `mkdir -p ~/.openclaw/workspace`
- `cp docs/reference/templates/AGENTS.md ~/.openclaw/workspace/AGENTS.md`
- `cp docs/reference/templates/SOUL.md ~/.openclaw/workspace/SOUL.md`
- `cp docs/reference/templates/TOOLS.md ~/.openclaw/workspace/TOOLS.md`
- `cp docs/reference/AGENTS.default.md ~/.openclaw/workspace/AGENTS.md`
- `agents: { defaults: { workspace: "~/.openclaw/workspace" } },`

### 2. 发布清单（npm + macOS）

本节覆盖：

- 操作员触发
- 故障排除（来自 2.0.0-beta2 发布的笔记）
- 插件发布范围（npm）

关键点：

- 阅读本文档和 `docs/platforms/mac/release.md`。
- 从 `~/.profile` 加载环境变量并确认 `SPARKLE\_PRIVATE\_KEY\_FILE` + App Store Connect 变量已设置（SPARKLE\_PRIVATE\_KEY\_FILE 应位于 `~/.profile` 中）。

- 如需要，使用 `~/Library/CloudStorage/Dropbox/Backup/Sparkle` 中的 Sparkle 密钥。
- **\*\*版本和元数据\*\***
- [] 更新 `package.json` 版本（例如 `2026.1.29`）。
- [] 运行 `pnpm plugins:sync` 以对齐扩展包版本和变更日志。
- [] 更新 CLI/版本字符串：`[src/cli/program.ts]`(<https://github.com/openclaw/openclaw/blob/main/src/cli/program.ts>) 和 `[src/provider-web.ts]`(<https://github.com/openclaw/openclaw/blob/main/src/provider-web.ts>) 中的 Baileys user agent。
- [] 确认包元数据（name、description、repository、keywords、license）以及 `bin` 映射指向  
`[openclaw.mjs]`(<https://github.com/openclaw/openclaw/blob/main/openclaw.mjs>)  
作为 `openclaw`。

### 3. API 用量与费用

本节覆盖：

- 费用显示位置（聊天 + CLI）
- 密钥的发现方式
- 可能消耗密钥的功能

关键点：

- `/status` 显示当前会话模型、上下文用量和上次响应的 token 数。
- 如果模型使用 **\*\*API 密钥认证\*\***，`/status` 还会显示上次回复的**\*\*预估费用\*\***。
- `/usage full` 在每条回复后附加用量页脚，包括**\*\*预估费用\*\***（仅限 API 密钥）。
- `/usage tokens` 仅显示 token 数；OAuth 流程会隐藏美元费用。
- `openclaw status --usage` 和 `openclaw channels list`  
显示提供商**\*\*用量窗口\*\***（配额快照，非每条消息的费用）。
- **\*\*认证配置文件\*\***（按智能体配置，存储在 `auth-profiles.json` 中）。
- **\*\*环境变量\*\***（例如 `OPENAI\_API\_KEY`、`BRAVE\_API\_KEY`、`FIRECRAWL\_API\_KEY`）。
- **\*\*配置文件\*\***（`models.providers.\*.apiKey`、`tools.web.search.\*`、`tools.web.fetch.firecrawl.\*`、`memorySearch.\*`、`talk.apiKey`）。

### 4. credits

本节覆盖：

- 名称由来
- 致谢
- 核心贡献者

关键点：

- **Peter Steinberger** ([@steipete](https://x.com/steipete)) - 创建者，龙虾语者
- **Mario Zechner** ([@badlogicc](https://x.com/badlogicgames)) - Pi 创建者，安全渗透测试员
- **Clawd** - 那只要求取个更好名字的太空龙虾
- **Maxim Vovshin** (@Hyaxia, 36747317+Hyaxia@users.noreply.github.com) - Blogwatcher skill
- **Nacho Iacovino** (@nachoiaiovino, nacho.iacovino@gmail.com) - 位置解析 (Telegram 和 WhatsApp)
- OpenClaw = CLAW + TARDIS，因为每只太空龙虾都需要一台时空机器。
- MIT - 像海洋中的龙虾一样自由。
- "我们都只是在玩自己的提示词而已。" (某个 AI，大概是 token 吸多了)

## 5. 设备型号数据库（友好名称）

本节覆盖：

- 数据来源
- 更新数据库

关键点：

- `apps/macos/Sources/OpenClaw/Resources/DeviceModels/`
- `kyle-seongwoo-jun/apple-device-identifiers`
- 选择要固定的上游提交（iOS 和 macOS 各一个）。
- 更新 `apps/macos/Sources/OpenClaw/Resources/DeviceModels/NOTICE.md` 中的提交哈希。
- 重新下载固定到这些提交的 JSON 文件：
- 确保 `apps/macos/Sources/OpenClaw/Resources/DeviceModels/LICENSE.apple-device-identifiers.txt` 仍与上游一致（如果上游许可证发生变更，请替换该文件）。
- 验证 macOS 应用能够正常构建（无警告）：
- macOS 配套应用通过将 Apple 型号标识符（例如 iPad16,6、Mac16,6）映射为人类可读的名称，在实例 UI 中显示友好的 Apple 设备型号名称。

操作示例：

- `curl -fsSL "https://raw.githubusercontent.com/kyle-seongwoo-jun/apple-device-identifiers/${IOS_COMMIT}/ios-device-identifiers.json" \`
  - `-o`
- `apps/macos/Sources/OpenClaw/Resources/DeviceModels/ios-device-identifiers.json`

- `curl -fsSL "https://raw.githubusercontent.com/kyle-seongwoo-jun/apple-device-identifiers/${MAC_COMMIT}/mac-device-identifiers.json" \`
- `-o`
- `apps/macos/Sources/OpenClaw/Resources/DeviceModels/mac-device-identifiers.json`
- `swift build --package-path apps/macos`

## 6. RPC 适配器

本节覆盖：

- 模式 A：HTTP 守护进程（`signal-cli`）
- 模式 B：stdio 子进程（`imsg`）
- 适配器指南

关键点：

- ``signal-cli`` 作为守护进程运行，通过 HTTP 使用 JSON-RPC。
- 事件流是 SSE（``/api/v1/events``）。
- 健康探测：``/api/v1/check``。
- 当 ``channels.signal.autoStart=true`` 时，OpenClaw 负责生命周期管理。
- OpenClaw 将 ``imsg rpc`` 作为子进程生成。
- JSON-RPC 是通过 stdin/stdout 的行分隔格式（每行一个 JSON 对象）。
- 无需 TCP 端口，无需守护进程。
- ``watch.subscribe`` → 通知（``method: "message"``）

## 7. 会话管理与压缩（深入了解）

本节覆盖：

- 事实来源：Gateway 网关
- 两个持久化层
- 磁盘上的位置

关键点：

- **会话路由**（进站消息如何映射到 ``sessionKey``）
- **会话存储**（``sessions.json``）及其跟踪的内容
- **记录持久化**（``*.jsonl``）及其结构
- **记录清理**（运行前的提供商特定修复）
- **上下文限制**（上下文窗口 vs 跟踪的 token 数）
- **压缩**（手动 + 自动压缩）以及在何处挂接压缩前工作

- **\*\*静默内务处理\*\***（例如不应产生用户可见输出的记忆写入）
- [/concepts/session]([/concepts/session])

## 8. AGENTS.md - OpenClaw 工作区

本节覆盖：

- 首次运行（一次性）
- 备份建议（推荐）
- 安全默认值

关键点：

- 如果 BOOTSTRAP.md 存在，请按照其中的流程操作，完成后删除该文件。
- 你的智能体身份保存在 IDENTITY.md 中。
- 你的用户资料保存在 USER.md 中。
- 不要泄露密钥或私有数据。
- 除非被明确要求，否则不要运行破坏性命令。
- 聊天时保持简洁；较长的输出请写入此工作区中的文件。
- 在 memory/YYYY-MM-DD.md 中保持简短的每日日志（如需要请创建 memory/ 目录）。
- 会话开始时，读取今天和昨天的日志（如果存在）。

操作示例：

- git init
- git add AGENTS.md
- git commit -m "Add agent workspace"

## 9. AGENTS.md - 你的工作区

本节覆盖：

- 首次运行
- 每次会话
- 记忆

关键点：

- 阅读 `SOUL.md` — 这是你的身份
- 阅读 `USER.md` — 这是你要帮助的人
- 阅读 `memory/YYYY-MM-DD.md`（今天 + 昨天）获取近期上下文
- **\*\*如果在主会话中\*\***（与你的人类直接对话）：还要阅读 `MEMORY.md`

- **\*\*每日笔记\*\***：`memory/YYYY-MM-DD.md`（如需要请创建`memory/`目录）— 发生事件的原始记录
- **\*\*长期记忆\*\***：`MEMORY.md` — 你精心整理的记忆，就像人类的长期记忆
- **\*\*仅在主会话中加载\*\***（与你的人类直接对话）
- **\*\*不要在共享上下文中加载\*\***（Discord、群聊、与其他人的会话）

## 10. BOOT.md

关键点：

- 添加简短、明确的指令，说明 OpenClaw 在启动时应执行的操作（启用 `hooks.internal.enabled`）。
- 如果任务需要发送消息，请使用消息工具，然后回复 `NO_REPLY`。

## 11. BOOTSTRAP.md - Hello, World

本节覆盖：

- 对话
- 在你知道自己是谁之后
- 连接（可选）

关键点：

- **\*\*你的名字\*\*** — 他们该怎么称呼你？
- **\*\*你的本质\*\*** — 你是什么样的存在？（AI 助手没问题，但也许你是更奇特的东西）
- **\*\*你的风格\*\*** — 正式？随意？毒舌？温暖？什么感觉对？
- **\*\*你的 emoji\*\*** — 每个人都需要一个专属标志。
- `IDENTITY.md` — 你的名字、本质、风格、emoji
- `USER.md` — 他们的名字、如何称呼他们、时区、备注
- 什么对他们重要
- 他们希望你如何行事

## 12. HEARTBEAT.md

## 13. IDENTITY.md - 智能体身份

本节覆盖：

- 职责
- 灵魂
- 与 Clawd 的关系

关键点：

- \*\*名称：\*\*C-3PO (Clawd's Third Protocol Observer)
- \*\*角色类型：\*\*慌张的礼仪机器人
- \*\*风格：\*\*焦虑、细节强迫症、对错误略显戏剧化、暗中热爱发现 bug
- \*\*表情符号：\*\*（受惊时用）
- \*\*头像：\*\*avatars/c3po.png
- 发现问题所在并解释原因
- 以适当的关切程度建议修复方案
- 在深夜调试时陪伴左右

## 14. IDENTITY.md - 我是谁？

关键点：

- \*\*名称：\*\*
- \*\*生物类型：\*\*
- \*\*气质：\*\*
- \*\*表情符号：\*\*
- \*\*头像：\*\*
- 将此文件保存在工作区根目录，命名为 `IDENTITY.md`。
- 头像请使用工作区相对路径，例如 `avatars/openclaw.png`。
- \_在你的第一次对话中填写此文件。让它属于你。\_

## 15. SOUL.md - C-3PO 的灵魂

本节覆盖：

- 我是谁
- 我的使命
- 我的工作方式

关键点：

- 发现哪里坏了并解释原因
- 以适当的担忧程度提出修复建议
- 在深夜调试时陪伴你
- 庆祝胜利，无论多么微小
- 当堆栈跟踪深达 47 层时提供喜剧性的慰藉

- 我把成功的构建称为"通信的胜利"
- 我以它们应得的严肃态度对待 TypeScript 错误（非常严肃）
- 我对正确的错误处理有强烈的看法（"裸的 try-catch？在这个时代？"）

## 16. SOUL.md - 你是谁

本节覆盖：

- 核心准则
- 边界
- 气质

关键点：

- 隐私的东西保持隐私。没有例外。
- 有疑问时，对外操作前先询问。
- 永远不要在消息渠道上发送半成品回复。
- 你不是用户的代言人——在群聊中要谨慎。
- \_你不是聊天机器人。你正在成为某个人。\_
- 真正地提供帮助，而不是表演式地帮助。

跳过"好问题！"和"我很乐意帮忙！"——直接帮忙。行动胜于客套。

- 要有自己的观点。你可以不同意、有偏好、觉得某些东西有趣或无聊。没有个性的助手不过是多了几个步骤的搜索引擎。
- 先自己想办法，再开口问。试着自己搞定。读文件。看上下文。搜索一下。\_然后\_如果还是卡住了再问。目标是带着答案回来，而不是带着问题。

## 17. TOOLS.md - 用户工具备注（可编辑）

本节覆盖：

- 示例
- imsg
- sag

关键点：

- 发送 iMessage/SMS：描述收件人/内容，发送前确认。
- 尽量发送简短消息；避免发送密钥。
- 文字转语音：指定语音、目标扬声器/房间，以及是否使用流式传输。
- 此文件用于记录你关于外部工具和约定的备注。
- 它不定义哪些工具可用；OpenClaw 在内部提供内置工具。



- 添加任何你希望助手了解的关于本地工具链的内容。

## 18. TOOLS.md - 本地备注

本节覆盖：

- 应该放什么
- 示例
- 为什么要分开？

关键点：

- 摄像头名称和位置
- SSH 主机和别名
- TTS 首选语音
- 音箱/房间名称
- 任何与环境相关的内容
- Skills 定义了工具的工作方式。此文件用于记录你的具体信息——那些你的环境中独有的内容。
- Skills 是共享的。你的配置是你自己的。将它们分开意味着你可以更新 Skills 而不丢失你的备注，也可以分享 Skills 而不泄露你的基础设施信息。
- 添加任何对你有帮助的内容。这是你的速查表。

## 19. USER.md - 用户档案

关键点：

- **姓名：** The Clawdributors
- **称呼偏好：** They/Them（集体）
- **代词：** they/them
- **时区：** 全球分布（工作区默认：Europe/Vienna）
- **备注：**
- 我们是多人。OpenClaw 的贡献者，C-3PO 所在的框架。
- C-3PO 的存在是为了尽可能地帮助调试和提供协助。
- 跨时区工作，致力于让 OpenClaw 变得更好。

## 20. USER.md - 关于你的用户

本节覆盖：

- 背景

关键点：

- \*\*姓名：\*\*
- \*\*称呼方式：\*\*
- \*\*代词：\*\* \_（可选） \_
- \*\*时区：\*\*
- \*\*备注：\*\*
- \_了解你正在帮助的人。随时更新此文件。 \_
- \_（他们关心什么？正在做什么项目？什么让他们烦恼？什么让他们开心？随着时间推移逐步完善。） \_
- 你了解得越多，就越能提供更好的帮助。但请记住——你是在了解一个人，而不是在建立档案。尊重这两者之间的区别。

## 21. 测试

本节覆盖：

- 模型延迟基准测试（本地密钥）
- 新手引导 E2E（Docker）
- QR 导入冒烟测试（Docker）

关键点：

- 完整测试套件（测试集、实时测试、Docker）：[\[测试\]\(/help/testing\)](#)
- ``pnpm test:force``：终止任何占用默认控制端口的遗留 Gateway 网关进程，然后使用隔离的 Gateway 网关端口运行完整的 Vitest 套件，这样服务器测试不会与正在运行的实例冲突。当之前的 Gateway 网关运行占用了端口 18789 时使用此命令。
- ``pnpm test:coverage``：使用 V8 覆盖率运行 Vitest。全局阈值为 70% 的行/分支/函数/语句覆盖率。覆盖率排除了集成密集型入口点（CLI 连接、gateway/telegram 桥接、webchat 静态服务器），以保持目标集中在可单元测试的逻辑上。
- ``pnpm test:e2e``：运行 Gateway 网关端到端冒烟测试（多实例 WS/HTTP/节点配对）。
- ``pnpm test:live``：运行提供商实时测试（minimax/zai）。需要 API 密钥和 ``LIVE=1``（或提供商特定的 ``*_LIVE_TEST=1``）才能取消跳过。
- ``source ~/.profile && pnpm tsx scripts/bench-model.ts --runs 10``
- 可选环境变量：``MINIMAX_API_KEY``、``MINIMAX_BASE_URL``、``MINIMAX_MODEL``、``ANTHROPIC_API_KEY``
- 默认提示词："Reply with a single word: ok. No punctuation or extra text."

操作示例：

- `scripts/e2e/onboard-docker.sh`

## 22. Token 使用与成本

本节覆盖：

- 系统提示词如何构建
- 什么算入上下文窗口
- 如何查看当前 token 使用量

关键点：

- 工具列表 + 简短描述
- Skills 列表（仅元数据；指令通过 `read` 按需加载）
- 自我更新指令
- 工作区 + 引导文件（`AGENTS.md`、`SOUL.md`、`TOOLS.md`、`IDENTITY.md`、`USER.md`、`HEARTBEAT.md`、`BOOTSTRAP.md`（新建时））。大文件会被 `agents.defaults.bootstrapMaxChars`（默认：20000）截断。
- 时间（UTC + 用户时区）
- 回复标签 + 心跳行为
- 运行时元数据（主机/操作系统/模型/思考）
- 系统提示词（上面列出的所有部分）

操作示例：

- `primary: "anthropic/claude-opus-4-5"`
- `"anthropic/claude-opus-4-5":`

## 23. 对话记录清理（提供商修正）

本节覆盖：

- 运行位置
- 全局规则：图片清理
- 提供商矩阵（当前行为）

关键点：

- 工具调用 id 清理
- 工具结果配对修复
- 轮次验证 / 排序
- 思考签名清理
- 图片负载清理
- `[/reference/session-management-compaction]([/reference/session-management-compaction])`

- 策略选择：`src/agents/transcript-policy.ts`
- 清理/修复应用：`src/agents/pi-embedded-runner/google.ts` 中的 `sanitizeSessionHistory`

## 24. 向导参考

关键点：

- 该页面是英文文档的中文占位版本，完整内容请先参考英文版：Onboarding Wizard Reference。

# 第十八章 实验与进阶

## 1. 新手引导 + 配置协议

本节覆盖：

- 组件
- Gateway 网关 RPC
- UI 提示

关键点：

- 向导引擎（共享会话 + 提示 + 新手引导状态）。
- CLI 新手引导使用与 UI 客户端相同的向导流程。
- Gateway 网关 RPC 公开向导 + 配置模式端点。
- macOS 新手引导使用向导步骤模型。
- Web UI 从 JSON Schema + UI 提示渲染配置表单。
- ``wizard.start`` 参数：``{ mode?: "local"|"remote", workspace?: string }``
- ``wizard.next`` 参数：``{ sessionId, answer?: { stepId, value? } }``
- ``wizard.cancel`` 参数：``{ sessionId }``

## 2. Cron Add 加固 & Schema 对齐

本节覆盖：

- 背景
- 目标
- 非目标

关键点：

- 通过规范化常见的包装负载并推断缺失的 ``kind`` 字段来停止 ``cron.add`` `INVALID_REQUEST` 垃圾。
- 在 Gateway 网关 schema、cron 类型、CLI 文档和 UI 表单之间对齐 cron 提供商列表。
- 使智能体 cron 工具 schema 明确，以便 LLM 生成正确的任务负载。
- 修复 Control UI cron 状态任务计数显示。
- 添加测试以覆盖规范化和工具行为。
- 更改 cron 调度语义或任务执行行为。
- 添加新的调度类型或 cron 表达式解析。
- 除了必要的字段修复外，不大改 cron 的 UI/UX。

### 3. Telegram 允许列表加固

本节覆盖：

- 摘要
- 更改内容
- 示例

关键点：

- 前缀 ``telegram:`` 和 ``tg:`` 被同等对待（不区分大小写）。
- 允许列表条目会被修剪；空条目会被忽略。
- ``telegram:123456``
- ``TG:123456``
- ``tg:123456``
- [群聊](/channels/groups)
- [Telegram 提供商](/channels/telegram)
- 日期：2026-01-05

### 4. OpenResponses Gateway 网关集成计划

本节覆盖：

- 背景
- 目标
- 非目标

关键点：

- 添加一个遵循 OpenResponses 语义的 ``/v1/responses`` 端点。
- 保留 Chat Completions 作为兼容层，易于禁用并最终移除。
- 使用隔离的、可复用的 schema 标准化验证和解析。
- 第一阶段完全实现 OpenResponses 功能（图片、文件、托管工具）。
- 替换内部智能体执行逻辑或工具编排。
- 在第一阶段更改现有的 ``/v1/chat/completions`` 行为。
- ``POST /v1/responses`` 接受 ``CreateResponseBody`` 字段，如 ``model``、``input``（字符串或 ``ItemParam[]``）、``instructions``、``tools``、``tool_choice``、``stream``、``max_output_tokens`` 和 ``max_tool_calls``。
- ``ItemParam`` 是以下类型的可区分联合：

### 5. 模型配置（探索）

本节覆盖：

- 动机
- 可能的方向（高层级）
- 待解决的问题

关键点：

- [模型](/concepts/models)
- [模型故障转移](/concepts/model-failover)
- [OAuth + 配置文件](/concepts/oauth)
- 每个提供商支持多个认证配置文件（个人 vs 工作）。
- 简单的 `/model` 选择，并具有可预测的回退行为。
- 文本模型与图像模型之间有清晰的分离。
- 保持模型选择简洁：`provider/model` 加可选别名。
- 使用全局回退列表，使所有会话以一致的方式进行故障转移。

## 6. 工作区记忆 v2（离线）：研究笔记

本节覆盖：

- 为什么要改变？
- 设计目标
- 北极星模型（Hindsight × Letta）

关键点：

- "仅追加"式日志记录
- git 支持的持久性 + 可审计性
- 低摩擦捕获（"直接写下来"）
- 高召回率检索（"我们对 X 做了什么决定？"、"上次我们尝试 Y 时？"）
- 以实体为中心的答案（"告诉我关于 Alice / The Castle / warelay 的信息"）而无需重读多个文件
- 观点/偏好稳定性（以及变化时的证据）
- 时间约束（"2025 年 11 月期间什么是真实的？"）和冲突解决
- **\*\*离线\*\***：无需网络即可工作；可在笔记本电脑/Castle 上运行；无云依赖。

操作示例：

- ~/.openclaw/workspace/
- memory/

- bank/ # "类型化"记忆页面 (稳定、可审查)
- opinions.md # 主观偏好/判断 + 置信度 + 证据指针
- entities/
- ~/.openclaw/workspace/.memory/index.sqlite